Construindo um HSM de brinquedo

Como funciona um módulo criptográfico de hardware, aprendido de dentro para fora

Projeto hsm-a7 — Artix-7 XC7A35T

Sumário

Parte I — O que é um HSM

1. O problema que um HSM resolve
2. Modelo de ameaça
3. As três propriedades
4. Onde HSMs são usados

Parte II — Arquitetura interna

5. A fronteira criptográfica
6. Hierarquia de chaves
7. Cerimônia de chave, split knowledge e dual control
8. Key blocks: por que o formato é feio
9. Máquina de estados: a chave de switch
10. Aleatoriedade: onde HSM vira ciência
11. Self-test: por que o dispositivo se recusa a funcionar
12. Zeroização: apagar é mais difícil do que parece
13. Log de auditoria: por que antes, não depois
14. Ciclo de vida de uma chave

Parte III — Como HSMs são atacados

15. Ataques de API — o mais importante
16. Canais laterais
17. Injeção de falha
18. Ataques físicos e antitamper

Parte IV — Certificação

19. FIPS 140-3
20. PCI: as regras de pagamentos
21. Common Criteria
22. O que “certificado” significa — e o que não significa

Parte V — O equipamento hsm-a7

23. Objetivo, plataforma e restrições
24. A plataforma, peça por peça

Parte VI — As funções, fase a fase

25. Fase 2 — engines e self-test
26. Fase 3 — hierarquia de chaves
27. Fase 4 — armazenamento não-volátil
28. Fase 5 — API de host
29. Fase 6 — tamper e ataques
30. Fase 7 — assimétrico
31. Fase 8 — criptografia de pagamento
32. Ordem de trabalho, e a regra que a sustenta

Parte VII — Criptografia de pagamento

33. PIN blocks — por que um PIN não viaja sozinho
34. Tradução de PIN — o comando mais perigoso que existe
35. Verificação de PIN — PVV e IBM 3624
36. DUKPT — uma chave por transação
37. O que este projeto pode e o que não pode ensinar

Apêndices

A. Glossário
B. Especificação do protocolo
C. Pinagem
D. Reproduzir o trabalho
E. Operar o dispositivo
F. Leitura

Sobre este documento

Este é o material de um projeto didático: construir, do zero, um módulo criptográfico com hierarquia de chaves, cerimônia de carga, key blocks e API de host — reproduzindo a arquitetura de um HSM real em escala reduzida, num FPGA Artix-7.

O documento tem duas metades que se encontram no meio. As Partes I a IV explicam **como um HSM funciona e por quê**, independentemente deste projeto: arquitetura, ameaças, ataques históricos e certificação. A Parte V descreve **a plataforma do equipamento** e liga cada peça ao conceito correspondente. A Parte VI descreve **as funções que cada fase acrescentou**: primitivas e autoteste, hierarquia de chaves, armazenamento e log, API de host, resposta a violação e criptografia assimétrica. A Parte VII trata do domínio para o qual o projeto aponta: **criptografia de pagamento** — PIN blocks, tradução de PIN, verificação por PVV e 3624, DUKPT — e diz, sem rodeio, o que deste domínio é ensinável aqui e o que não é.

O equipamento tem AES-256, SHA-256, HMAC, CMAC, TRNG com health tests da SP 800-90B, CTR_DRBG, RSA-2048 e ECDSA P-256, todos conferidos contra vetores em autoteste, no boot ou antes do primeiro uso. Tem também key store, cerimônia de LMK com dual control, key blocks X9.143, zeroização com prova, armazenamento persistente, log de auditoria, porta host ASCII e resposta a violação. O histórico da construção e as medidas ficam nas notas de fase, fora deste manual.

Quem quer só entender o assunto pode parar na Parte IV — ou seguir para a VII, se o interesse for pagamento. Quem quer reproduzir o trabalho começa na Parte V e volta às anteriores conforme a dúvida aparecer. Quem tem a placa na mão e quer operá-la vá direto ao **apêndice E**, que percorre uma sessão inteira: ligar, autoteste, cerimônia, ativação e o trabalho com chaves.

> **AVISO — este dispositivo não é um HSM**
>
> Não há PUF, malha antitamper, invólucro com detecção de violação, gerador de números aleatórios certificado, nem qualquer validação FIPS, PCI ou Common Criteria. As chaves ficam numa BRAM que qualquer um com o bitstream pode inspecionar em simulação.
>
> O objetivo é **entender a arquitetura de dentro para fora**, não proteger material de chave real. Nenhuma chave de produção deve jamais ser carregada neste dispositivo.
>
> A honestidade sobre isso não é modéstia: saber exatamente **por que** este aparelho não é seguro é metade do aprendizado. Cada ausência listada na seção 22 corresponde a um mecanismo real que existe num HSM de verdade e custa dinheiro, tempo de projeto e meses de avaliação.

Licença. Este manual e o código do projeto estão sob a **Apache License 2.0**. O texto integral está em `LICENSE`, no repositório.

Independência. Este documento não nomeia fabricante nem produto de HSM, e não há vínculo, patrocínio, endosso ou alegação de compatibilidade com equipamento comercial algum. O conteúdo vem de normas públicas — ISO, ANSI, NIST, IETF — e de literatura acadêmica publicada; nada é

derivado de documentação proprietária. Ver `THIRD-PARTY.md`.

Parte I — O que é um HSM

1. O problema que um HSM resolve

Software comum guarda chaves na memória do processo. Quem tem acesso à máquina — o usuário `root`, um dump de memória, um bug de leitura fora de limites, um backup mal apagado — tem a chave. E há uma consequência mais sutil: **não existe forma de distinguir "usar a chave" de "copiar a chave"**. Se o programa consegue assinar, o programa consegue exportar o segredo que assina. Quem controla o programa herda essa capacidade.

Um HSM (*Hardware Security Module*) existe para tornar essas duas coisas diferentes. Ele oferece uma **interface de operações**, não de acesso:

```
    o host manda:  "assine este bloco com a chave 7"
    o host nunca manda:  "me devolva a chave 7"
```

A chave nasce dentro do dispositivo, vive dentro e morre dentro. Ela pode sair — mas só embrulhada (*wrapped*) por outra chave que também nunca saiu.

Essa separação parece pequena e reorganiza tudo. A pergunta de segurança deixa de ser *"quem consegue ler a memória?"* e passa a ser:

> ***O que a API permite deduzir sobre a chave, se for chamada um milhão de vezes com entradas escolhidas pelo atacante?***

Praticamente toda a arquitetura descrita neste documento decorre dessa pergunta. É também por isso que os ataques mais interessantes contra HSMs não quebram criptografia: eles conversam com a API.

Um exemplo concreto

Imagine um banco que precisa verificar PINs de cartão. O PIN correto é derivado do número da conta com uma chave secreta. Sem HSM:

- a chave de derivação está na memória do servidor de autorização;
- qualquer comprometimento desse servidor entrega a chave;
- com a chave, o atacante calcula o PIN de **qualquer** conta, offline, sem deixar rastro e sem limite de tentativas.

Com HSM:

- a chave está dentro do módulo e nunca esteve em outro lugar;
- o servidor manda "este PIN confere para esta conta?" e recebe sim ou não;
- comprometer o servidor dá ao atacante a capacidade de **perguntar**, não a chave. Perguntas são contadas, limitadas e registradas.

O atacante não é impedido — ele é **reduzido a um canal estreito, medido e auditável**. Essa redução é o produto que um HSM vende.

2. Modelo de ameaça

"Seguro" não significa nada sem dizer contra quem. HSMs são projetados contra uma escada de adversários, e cada degrau custa mais caro de defender.

Adversário	Capacidade	Defesa correspondente
Aplicação comprometida	Chama a API à vontade	Máquina de estados, atributos de chave, limites de uso
Administrador do host	Root na máquina, vê todo o tráfego do canal	Chaves nunca no host; autorização física no próprio módulo
Insider com acesso físico	Toca no equipamento, opera o painel	Dual control, split knowledge, log de auditoria
Atacante com o aparelho na mão	Bancada, osciloscópio, fonte de alimentação	Antitamper, sensores, canal lateral, injeção de falha
Laboratório especializado	Decapsulação, microssonda, FIB	Malha ativa, camadas de blindagem, zeroização por detecção

Duas observações que orientam o resto do documento.

O host é hostil por premissa. Não é que se desconfie do host — é que a arquitetura não pode *depender* de ele estar íntegro. É por isso que autorização de operações sensíveis é feita com **botões no próprio equipamento**, e não com um comando "eu autorizo" que chega pelo cabo. Um comando pelo cabo tem exatamente a confiabilidade do host que o enviou.

A escada é econômica, não absoluta. Nenhum HSM é inviolável. O objetivo é tornar o ataque mais caro que o valor protegido, e detectável quando tentado. Um módulo que resiste seis meses a um laboratório nacional e falha contra ele no sétimo mês pode estar perfeitamente adequado, se as chaves tiverem rotação trimestral.

3. As três propriedades

Tudo que um HSM faz se organiza em três propriedades. Elas são interdependentes: qualquer uma sozinha é inútil.

Confinamento

Existe uma fronteira física, e material de chave em claro não a atravessa. Dentro: chaves, estado do gerador aleatório, segredos derivados. Fora: criptogramas, chaves embrulhadas, valores de verificação, identificadores, log.

Confinamento sem mediação é um cofre com a porta aberta.

Mediação

Toda operação passa por verificação de política **antes** de executar: o dispositivo está no estado certo? Há autorização suficiente? Esta chave tem permissão para esta operação? O pedido é bem formado?

Mediação sem confinamento é uma sugestão — se a chave pode ser lida por fora, a verificação é decorativa.

Evidência

O que aconteceu fica registrado de forma que sobreviva à falha e não possa ser reescrito para omitir a tentativa.

Sem evidência, você nunca descobre se as outras duas falharam. E é a propriedade mais frequentemente subestimada: um log que registra apenas sucessos mostra um dispositivo saudável exatamente enquanto ele está sendo mapeado por um atacante, cujo trabalho produz muitos erros e poucos acertos.

4. Onde HSMs são usados

O termo cobre famílias de equipamento bastante diferentes, e saber qual é qual ajuda a entender por que este projeto modela LMK e TR-31 em vez de, digamos, PKCS#11 e certificados.

HSM de pagamentos

O mais antigo e o mais rígido. Protege PINs de cartão, chaves de terminal e criptogramas EMV. Há poucos fabricantes, e os equipamentos são certificados sob o PCI PTS HSM.

É onde nasceram os conceitos de **LMK** (chave mestra local), **key block** e a exigência operacional de dual control e split knowledge. As regras não são recomendações: PCI PIN Security e PCI PTS HSM são requisitos contratuais para quem processa transações de cartão.

O comando típico não é “assine” — é “verifique este PIN”, “traduza este PIN block desta chave para aquela”, “gere uma chave de terminal”.

HSM de propósito geral

Protege chaves de PKI, assinatura de código, TLS, bancos de dados, blockchain. Vários fabricantes, tipicamente validados em FIPS 140-3, YubiHSM.

A interface padrão é **PKCS#11** — `C_GenerateKey`, `C_Sign`, `C_WrapKey` — ou APIs próprias (JCE, CNG, KMIP). O modelo mental é o de um chaveiro com objetos que têm atributos, e os atributos é que definem o que se pode fazer com cada chave.

HSM em nuvem

AWS CloudHSM, Azure Managed HSM, Google Cloud HSM. Fisicamente são os mesmos módulos, operados pelo provedor, com o cliente controlando as chaves. O interesse conceitual está no problema novo que criam: **como você confia num módulo que não está na sua sala?** A resposta envolve atestação remota, e é uma área em movimento.

Elementos seguros e TPMs

Primos pequenos: TPM na placa-mãe, Secure Enclave, smartcard, YubiKey. Mesma ideia — chave que não sai, operação mediada — com orçamento de área, consumo e custo muito menores. Um cartão de crédito com chip é um HSM minúsculo cuja produção passa de bilhões de unidades por ano.

O que este projeto modela

O caminho de pagamentos: **LMK, key blocks TR-31, cerimônia de carga com componentes, dual control físico**. A razão é didática — é a família onde os mecanismos organizacionais (quem autoriza o quê, e como se prova) estão mais explícitos e mais bem documentados publicamente.

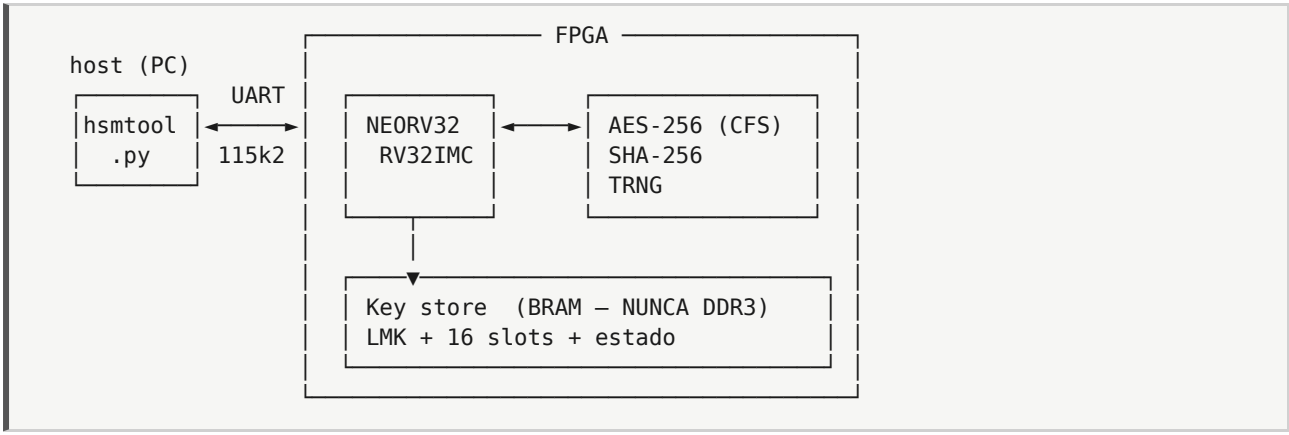
A Fase 5 do plano acrescenta a interface de host, e a escolha dela foi revista: em vez de um subconjunto de PKCS#11, um **command set ASCII** no estilo dos HSM de pagamento. Não é troca de sintaxe. PKCS#11 é uma API de *biblioteca* — objetos, sessões, atributos. Um command set de pagamento é um protocolo de *serviço*, sem sessão e sem estado do lado do cliente, e essa diferença muda como o dispositivo tem de se defender: cada comando chega sozinho e precisa se bastar.

Parte II — Arquitetura interna

5. A fronteira criptográfica

É o conceito central, e num projeto bem feito ele é literal e desenhável. Num documento de certificação FIPS existe um diagrama marcando o que está dentro, o que está fora, e uma lista de **todas** as portas que atravessam.

Neste projeto:



Dentro: LMK, chaves de trabalho em claro, estado do DRBG.

Fora: key blocks embrulhados, KCVs, criptogramas, *handles*, log.

Nada da primeira lista atravessa para a segunda. Todo comando novo é avaliado contra essa regra **antes** de ser implementado — é o único momento barato de perceber que um comando vaza.

Por que a fronteira precisa ser observável

Uma fronteira que você só pode *afirmar* não vale muito. Este projeto usa UART, e não Ethernet, precisamente por isso: a UART sai por dois pinos físicos identificáveis, e dá para prender um analisador lógico neles e capturar tudo que o dispositivo já disse.

O critério de aceitação da Fase 3 explora isso literalmente: rodar a suíte de testes inteira, capturar a UART, e fazer `grep` das chaves conhecidas do teste contra a captura. Se aparecer um byte, o projeto falhou — e você **sabe**, com evidência.

Com um stack de rede no meio, "nenhuma chave vazou" vira uma opinião sobre dezenas de milhares de linhas de código de terceiros. Com dois fios, vira uma medida.

Por que a memória importa mais do que parece

Neste projeto as chaves ficam em BRAM — memória interna ao die do FPGA. Lê-la exige decapsular o chip.

A DDR3 da placa é externa: o barramento passa por trilhas na PCB e chega a conectores de expansão. Uma ponta de prova resolve. Por isso a DDR3 não é apenas *evitada por convenção* — o barramento externo do processador está **desligado na configuração**, de modo que não existe caminho lógico do SoC até ela.

Essa é a diferença entre uma **política** e um **controle**: entre "o firmware não escreve chave na memória externa" e "não há circuito ligando o processador à memória externa". A primeira depende de todo programador futuro; a segunda, não.

6. Hierarquia de chaves

Um HSM com dezesseis slots de chave não guarda dezesseis segredos. Guarda **um** — e protege ou deriva todos os outros a partir dele.

```
LMK  (Local Master Key)
 │    nunca sai, nem embrulhada
 │
 ├── KBEK / KBAK ── derivadas por CMAC, uma por propósito
 │      │
 │      └── protegem key blocks TR-31
 │
 └── chaves de trabalho: BDK, KEK, chaves de dados, ZMK, ZPK...
```

Três razões, e vale entender cada uma porque elas explicam decisões aparentemente arbitrárias mais adiante.

Backup e migração. Se cada chave fosse independente, mover o serviço para outro HSM significaria mover N segredos, cada um com seu próprio procedimento. Com a hierarquia, você move a LMK — por cerimônia, em componentes — e todos os key blocks existentes continuam válidos, porque eles sempre foram apenas dados cifrados sob ela.

Revogação em bloco. Zeroizar a LMK invalida instantaneamente **todo** key block protegido por ela, onde quer que esteja: em backup, em fita, no servidor de outra filial. É a única operação de destruição que se consegue executar em tempo constante sobre dados que não estão na sua posse.

Separação de propósito. KBEK (chave de cifra) e KBAK (chave de autenticação) são derivadas da LMK por CMAC com rótulos distintos. A chave que cifra o corpo do key block nunca é a mesma que autentica seu cabeçalho. Se fosse, quebrar uma quebraria as duas — e existem ataques que exploram exatamente o reuso de chave entre modos de operação.

Atributos de chave

Cada chave carrega metadados que **fazem parte** dela, no sentido de que alterá-los invalida a autenticação. Estrutura usada neste projeto:

```c
typedef struct {
    uint8_t  in_use;
    uint8_t  usage[2];      // TR-31: 'B0'=BDK, 'K0'=KEK, 'D0'=dados...
    uint8_t  algorithm;     // 'A'=AES, 'T'=3DES
    uint8_t  mode_of_use;   // 'E'=cifrar, 'D'=decifrar, 'B'=ambos, 'N'=nenhum
    uint8_t  exportability; // 'E'=exportável, 'N'=não, 'S'=sensível
    uint8_t  key_len;
    uint8_t  key[32];       // BRAM. Nunca sai daqui em claro.
    uint8_t  kcv[3];
    uint32_t use_count;
} key_slot_t;
```

`exportability` é o campo que mais ensina. Uma chave marcada `'N'` não pode sair do módulo **por caminho nenhum** — nem embrulhada. Se algum comando conseguir exportá-la, o comando está errado. É exatamente o tipo de propriedade que se testa exaustivamente, porque ela vale zero se houver um único caminho esquecido.

`use_count` existe porque uso é informação: uma chave que devia ser usada mil vezes por dia e foi usada um milhão indica alguém explorando a API.

7. Cerimônia de chave, split knowledge e dual control

A LMK não é gerada num lugar e copiada para outro. Ela é **montada dentro do dispositivo**, a partir de componentes que nenhuma pessoa vê juntos.

```
    LMK = componente_A ⊕ componente_B ⊕ componente_C
```

Cada custodiante carrega apenas o seu componente. Nenhum componente isolado revela nada sobre a LMK — é o argumento do one-time pad: XOR com material desconhecido e uniforme não vaza informação. Isso é **split knowledge**.

KCV: verificar sem revelar

A cada passo, o dispositivo devolve um **KCV** (*Key Check Value*): os três bytes mais significativos do resultado de cifrar um bloco de zeros com a chave.

É um resumo pequeno o suficiente para não ajudar um atacante — 24 bits não permitem recuperar uma chave de 256 — e grande o suficiente para pegar digitação errada, componente trocado ou cartão corrompido. O host calcula o KCV esperado de forma independente e compara.

Repare no padrão: o dispositivo precisa provar que recebeu a coisa certa, sem revelar a coisa. Essa tensão reaparece o tempo todo em criptografia aplicada.

Dual control

Split knowledge diz que ninguém *sabe* a chave inteira. Dual control diz que ninguém *age* sozinho.

Neste projeto, `LMK_LOAD_COMPONENT` só é aceito com **dois botões físicos pressionados simultaneamente**. É simplório e é exatamente o ponto: separa fisicamente *quem digita* de *quem autoriza*. Um operador sozinho, mesmo com acesso administrativo total ao host, não carrega componente nenhum.

Por que isso existe, além de desconfiança: um operador que **não pode** agir sozinho não pode ser coagido com sucesso individual, não pode ser subornado individualmente, e não pode cometer o erro sozinho. Remove-se a possibilidade, não a tentação.

Num HSM comercial o mecanismo é mais rico — cada custodiante se autentica com smartcard e PIN, e o log registra *quem* autorizou. Mas a estrutura é a mesma, e a versão com dois botões torna visível o que a versão com smartcards esconde atrás de conveniência.

Por que a integridade do botão é código de segurança

Um contato mecânico produz dezenas de transições elétricas em poucos milissegundos ao ser pressionado ou solto. Sem filtragem, um único toque parece vários pressionamentos.

Se o firmware ler "pressionado" num ressalto que ocorreu **depois** de o custodiante soltar o botão, ele registrou uma autorização que ninguém deu. O mecanismo de autorização precisa refletir **intenção humana**, não estado elétrico de contato — e a distância entre as duas coisas é uns 10 ms de filtro.

8. Key blocks: por que o formato é feio

TR-31 (hoje ANSI X9.143) é o formato de troca de chaves da indústria de pagamentos. Um key block é ASCII, tem cabeçalho legível, e parece desnecessariamente complicado. Cada complicação existe por causa de um incidente.

```
D0112K0AB00N0000  <corpo cifrado em AES-CBC>  <CMAC>
^^^^^^^^^^^^^^^^
cabeçalho, em claro, mas COBERTO PELO MAC
```

O cabeçalho declara o que a chave é: versão do formato, comprimento, **uso** (`K0` = KEK, `B0` = BDK, `D0` = dados), algoritmo, **modo de uso** (só cifrar, só decifrar, ambos) e **exportabilidade**.

O ataque que o formato existe para impedir

Se o cabeçalho não fosse autenticado junto com o corpo, um atacante poderia pegar um key block legítimo e:

- trocar `N` (não exportável) por `E` (exportável), e depois exportar a chave em claro por um caminho autorizado;
- trocar o uso de "só verificar PIN" para "cifrar dados arbitrários", e usar a chave de PIN como oráculo de cifra;
- trocar o algoritmo declarado, induzindo o módulo a usar a chave num modo para o qual ela não foi projetada.

A chave continuaria criptograficamente válida, o HSM a importaria sem reclamar, e ela passaria a fazer algo que o dono nunca autorizou.

Isso **não é hipotético**. É a razão histórica pela qual os formatos anteriores — os chamados *variant key blocks*, em que o tipo era codificado por XOR com um vetor de controle — foram abandonados pela indústria. Bond e Clulow mostraram famílias inteiras de ataques que consistiam em manipular esses vetores para converter uma chave de um tipo em outro.

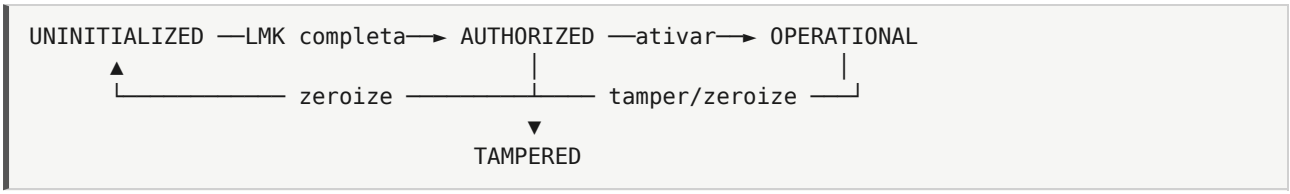
Consequência de projeto: **um byte errado no cabeçalho explode o MAC inteiro**. É o comportamento desejado, e é o que torna a implementação irritante de acertar.

Por que escrever o parser duas vezes

Neste projeto o TR-31 é implementado em C no firmware e em Python no host, e os dois validam-se mutuamente sobre blocos aleatórios.

É de longe a forma mais rápida de aprender o padrão de verdade, porque a menor divergência — um byte de preenchimento, uma ordem de campo, um comprimento incluído ou não na contagem — aparece imediatamente como MAC inválido. Não dá para "quase" implementar TR-31: ou os dois lados concordam em cada byte, ou nada funciona.

9. Máquina de estados: a chave de switch



HSMs comerciais têm uma chave física de três posições no painel. Este é o modelo.

Comandos sensíveis — carga de componente, exportação de material derivado da LMK, mudança de política — existem **apenas** em `AUTHORIZED`. E `AUTHORIZED` é um estado no qual o dispositivo **não atende produção** e no qual só se entra por ação física deliberada.

O raciocínio: em operação normal, a superfície de ataque deve ser mínima. Em `OPERATIONAL`, o dispositivo faz o trabalho e nada mais — não carrega chave, não exporta LMK, não muda política. Um invasor que comprometa **totalmente** o host de produção ainda não carrega componente de LMK, porque o dispositivo não aceita aquele opcode naquele estado, e mudar de estado exige mão humana no equipamento.

`TAMPERED` é absorvente, e **mais absorvente do que este texto dizia até a Fase 3 ser implementada**: não se sai dele por software, ponto. Nem por zeroize.

A distinção é fina e importa. O `ZEROIZE` funciona em `TAMPERED` — é exatamente ali que ele mais precisa funcionar — e apaga toda chave. Mas o estado **fica**. Apagar o que o dispositivo protegia não desfaz o veredito que ele deu sobre si mesmo; um módulo que se cura de tamper não detectou tamper nenhum, apenas registrou um incômodo.

Sair de `TAMPERED` exige energia removida e um boot em que o autoteste passe. Se a causa era física, ele não passa — que é o comportamento certo.

E um dispositivo em `TAMPERED` deve responder o mínimo possível: o estado existe para proteger o que restou, não para continuar servindo. Só quatro comandos respondem ali: `SELFTEST`, para dizer **o que** reprovou, `ZEROIZE`, para apagar, e, desde a Fase 6, `ENV_READ` e `AUDIT_READ`, para saber por quê.

10. Aleatoriedade: onde HSM vira ciência

Um HSM é tão bom quanto seu gerador. Chave previsível é chave pública, e o histórico de falhas nessa área é longo e caro.

A arquitetura padrão tem duas camadas:

```
fonte física  ──►  testes de saúde  ──►  DRBG  ──►  chaves
(ruído)            RCT + APT             AES-CTR
entropia bruta     SP 800-90B            SP 800-90A
```

Por que não usar a fonte física diretamente

Ruído físico tem viés e correlação. Osciladores em anel num FPGA — a fonte usada neste projeto — são sensíveis a temperatura e a tensão de alimentação. E um atacante que controle a fonte de alimentação pode **reduzir a entropia** sem que nada pareça errado do lado de fora.

O DRBG existe para transformar entropia imperfeita mas *suficiente* num fluxo computacionalmente indistinguível de aleatório. Ele não cria entropia — ele a estica e a uniformiza.

Testes de saúde: o coração da coisa

SP 800-90B exige que a fonte seja monitorada **continuamente**, não validada uma vez na fábrica. Dois testes obrigatórios:

RCT (*Repetition Count Test*) — dispara se a mesma amostra se repete acima de um limiar. Pega a falha catastrófica: o oscilador parou, a fonte travou num valor, o pino soltou.

APT (*Adaptive Proportion Test*) — janela de 512 ou 1024 amostras, dispara se um valor domina a janela. Pega a degradação: a fonte ainda oscila, mas ficou enviesada.

Falha em qualquer um → `TAMPERED`, DRBG parado, indicador vermelho. Não é paranoia: é a diferença entre “gerei uma chave fraca e não sei” e “me recusei a gerar”.

Escrever esses testes ensina mais sobre certificação de módulo criptográfico que qualquer texto sobre o assunto — eles são boa parte do motivo de uma avaliação levar meses.

11. Self-test: por que o dispositivo se recusa a funcionar

FIPS 140-3 exige que o módulo rode testes de resposta conhecida (**KAT**, *Known Answer Test*) na inicialização, **antes** de aceitar qualquer comando, e que entre em estado de erro se qualquer um falhar.

Parece burocracia. Não é, e o raciocínio é bonito:

> *Um AES que cifra errado não produz erro. Produz criptograma.*

Você não descobre nada até tentar decifrar — possivelmente meses depois, possivelmente em outro dispositivo, possivelmente sobre o único backup que existia. Um bit preso numa memória, uma violação de temporização marginal que só aparece a 60 °C, uma corrupção na configuração do FPGA: todos se manifestam como criptografia silenciosamente errada.

Os KAT são o único momento em que o dispositivo consegue dizer **"eu não estou funcionando"** antes de causar dano.

Neste projeto os mesmos vetores rodam nos testbenches de simulação **e** no POST em hardware. Isso é deliberado e diagnóstico: se um KAT passa na simulação e falha no POST, o problema está no barramento ou na integração, não no núcleo criptográfico.

E há uma regra que decorre disso, adotada como inviolável neste projeto:

> ***Se um KAT falha, o bug está no código, não no vetor.*** *Não ajustar vetor, não relaxar assert, não marcar teste como ignorado.*

Um teste enfraquecido para passar é pior que teste nenhum, porque agora existe uma afirmação falsa de correção, e alguém vai confiar nela.

12. Zeroização: apagar é mais difícil do que parece

```
    memset(chave, 0, 32);   /* ...e a função termina aqui */
```

Para o compilador, isso é um *dead store*: escrita num buffer que ninguém lê depois. O padrão C **permite** eliminá-lo, e com otimização ativada o compilador elimina. A chave permanece na memória, intacta, e o código *parece* correto na revisão — inclusive para um revisor experiente.

A solução é forçar o compilador a tratar cada escrita como efeito colateral observável:

```
void wipe(void *p, size_t n)
{
    volatile uint8_t *q = (volatile uint8_t *)p;
    while (n--) *q++ = 0u;
    __asm__ __volatile__("" ::: "memory");
}
```

`volatile` impede a eliminação; a barreira de memória impede reordenamento.

Zeroização é **requisito explícito** do FIPS 140-3, e a exigência vai além de chamar a função: um módulo certificado precisa demonstrar que o material foi efetivamente destruído. Por isso, neste projeto, o comando `ZEROIZE` sobrescreve a região com padrão e depois com zeros, e o **teste faz dump da região e verifica**. "Chamei a função de apagar" não é evidência.

13. Log de auditoria: por que antes, não depois

O registro é gravado **antes** de executar o comando. Isso é contraintuitivo — parece natural registrar o resultado.

O motivo: um log escrito depois só registra sucesso. Tentativa que falhou, comando que travou o dispositivo, sequência que causou reset — nada disso aparece. E é exatamente essa a informação que importa numa investigação.

Um atacante sondando a API produz **muitos erros e poucos sucessos**. Um log de sucessos mostra um dispositivo perfeitamente saudável enquanto ele está sendo mapeado.

Requisitos que decorrem:

- **contador monotônico**, para detectar remoção de entradas;
- **gravação antes da execução**, para capturar o que travou;
- **sem material de chave**, obviamente;
- **armazenamento que sobrevive à queda de energia**, senão o ataque termina com um puxão no cabo.

14. Ciclo de vida de uma chave

Amarrando as seções anteriores: uma chave, do nascimento à morte, e onde cada mecanismo entra.

Fase	O que acontece	Mecanismo
Geração	Dentro do módulo, a partir do DRBG	Testes de saúde da fonte, POST
Atribuição	Recebe atributos: uso, modo, exportabilidade	Estrutura do slot
Distribuição	Sai embrulhada em key block, ou nunca sai	TR-31, KBEK/KBAK, `exportability`
Armazenamento	BRAM interna; backup como key block	Fronteira, hierarquia sob LMK
Uso	Só operações permitidas pelo uso e pelo modo de uso	Mediação, máquina de estados
Rotação	Nova chave gerada, antiga marcada	`use_count` , política
Destruição	Zeroize, com prova	`wipe()` , verificação por dump

A parte que quase sempre é subestimada é a **destruição**. Uma chave que “não é mais usada” mas continua num backup de fita, num slot que ninguém limpou, ou na memória de um módulo que foi para o lixo, continua sendo uma chave viva do ponto de vista do atacante.

Parte III — Como HSMs são atacados

Esta parte é o contrapeso da anterior. Cada mecanismo descrito até aqui existe porque alguém, em algum momento, quebrou um dispositivo que não o tinha.

Os ataques abaixo são **públicos e históricos**, documentados em literatura acadêmica e usados há décadas no ensino de segurança. Estão aqui pelo motivo de sempre: não se projeta defesa sem entender o ataque.

15. Ataques de API — o mais importante

A observação central, e a que mais custa a ser internalizada:

> *Contra um HSM bem construído, você não ataca a criptografia. Você ataca a **API** — procura uma sequência de comandos, todos legítimos, que combinados revelem algo que nenhum comando isolado revelaria.*

O módulo funciona perfeitamente. Cada chamada é válida. Nenhum algoritmo é quebrado. E ainda assim a chave sai.

Este é o tipo de ataque que mais aparece contra HSMs reais, e o menos intuitivo para quem vem de criptografia teórica — porque a falha não está na matemática, está na **composição** de operações.

15.1 Confusão de tipo de chave

Nos formatos antigos da IBM CCA, o tipo de uma chave era codificado fazendo-se XOR de um *vetor de controle* com a chave mestra antes de cifrar a chave de trabalho. O tipo, portanto, não era autenticado — era apenas um valor combinado.

Mike Bond (2001) mostrou famílias de ataques que consistiam em manipular esses vetores para **converter uma chave de um tipo em outro**. Uma chave destinada a derivar PINs podia ser reapresentada ao módulo como chave de dados; a partir daí, bastava pedir ao HSM que cifrasse com ela para obter o que deveria ser secreto.

A defesa é o TR-31, e agora a seção 8 faz sentido: o cabeçalho declara o tipo *e é coberto pelo MAC*. Alterar o tipo invalida o bloco inteiro. Toda a feiura do formato existe por causa desta classe de ataque.

15.2 Ataque da tabela de decimalização

O exemplo mais didático que existe, porque a matemática é trivial e o efeito é devastador.

O método de verificação de PIN IBM 3624 funciona assim: cifra-se o número da conta com a chave de derivação, obtêm-se dígitos hexadecimais, e uma **tabela de decimalização** os mapeia para decimais:

```
    hex:    0 1 2 3 4 5 6 7 8 9 A B C D E F
    tabela: 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5     (padrão)
```

O problema: algumas APIs permitiam que **o chamador fornecesse a tabela**.

Bond e Zieliński (2003) observaram que, ao fornecer uma tabela degenerada — por exemplo, toda de zeros exceto uma posição — o atacante aprende, a cada consulta, se um determinado dígito aparece no PIN. Poucas consultas revelam o conjunto de dígitos; poucas mais revelam a ordem.

O resultado: recuperação de um PIN de quatro dígitos em cerca de **quinze chamadas de API**, contra as cinco mil de uma busca exaustiva. E cada chamada é perfeitamente legítima.

A lição de projeto: todo parâmetro controlado pelo chamador é uma entrada de ataque, inclusive os que parecem configuração inofensiva. É a razão do item mais importante do checklist deste projeto:

> *O que este comando pode fazer vazar se for chamado em laço com entradas escolhidas?*

Note que a pergunta não é "este comando vaza a chave?". É "**dez mil chamadas** deste comando vazam a chave?".

15.3 Ataques a PIN blocks

Um PIN não trafega sozinho: ele vai dentro de um *PIN block*, que combina o PIN com o número da conta. O formato mais comum, ISO-0 (ANSI X9.8), faz XOR entre o PIN preenchido e parte do número da conta.

HSMs de pagamento precisam **traduzir** PIN blocks — de uma chave para outra, de um formato para outro — porque a rede de pagamentos tem domínios de chave distintos entre adquirente, bandeira e emissor.

Clulow (2003) e outros mostraram que a combinação de tradução de formato com controle sobre o número da conta fornecido permite extrair informação sobre o PIN, um pedaço de cada vez. O módulo nunca revela o PIN; ele revela se uma operação teve sucesso, e isso basta.

A lição: funções de conversão são pontos de vazamento. Sempre que um módulo traduz entre representações, ele pode virar oráculo — e o mesmo padrão aparece fora de pagamentos, no *padding oracle*, em que um servidor que distingue "preenchimento inválido" de "MAC inválido" entrega decifração completa a quem perguntar o suficiente.

É por isso que, neste projeto, o arquivo de códigos de status carrega esta regra:

```
/* Um status nunca pode revelar mais do que o host tem direito de saber.
   "Chave errada" e "slot vazio" devem ser o mesmo codigo sempre que a
   distincao ajudar um atacante a enumerar o key store. */
```

15.4 O padrão comum

Repare no que os três têm em comum:

1. Todas as chamadas são **legítimas** — nenhuma validação foi burlada.
2. O vazamento vem da **composição**, não de um comando isolado.
3. O canal é estreito (um bit de sucesso/falha), e mesmo assim suficiente.
4. A defesa está no **projeto da API**, não em criptografia mais forte.

E é por isso que “todo opcode novo precisa de entrada na tabela, verificação de estado e teste, nessa ordem” é regra de projeto e não burocracia. A hora barata de perceber que um comando vaza é antes de escrevê-lo.

16. Canais laterais

Aqui o atacante não fala com a API. Ele **observa o dispositivo trabalhando** e infere o segredo a partir de algo que não era para ser saída.

16.1 Tempo

Kocher (1996) mostrou que o tempo de execução de exponenciação modular depende dos bits do expoente. Medindo tempos de resposta, recupera-se a chave privada.

A versão moderna mais comum não envolve matemática de chave pública: é **comparação de MAC que retorna cedo**.

```
    /* ERRADO: vaza o comprimento do prefixo correto */
    if (memcmp(mac_calculado, mac_recebido, 16) != 0) return ERRO;
```

`memcmp` retorna assim que encontra diferença. Um atacante mede o tempo, descobre quantos bytes iniciais acertou, e reconstrói o MAC byte a byte — 2*16*256 tentativas em vez de 2^128.

A defesa é **comparação em tempo constante**: acumular diferenças em vez de retornar cedo.

```
    uint8_t diff = 0;
    for (int i = 0; i < 16; i++) diff |= a[i] ^ b[i];
    return diff == 0;
```

16.2 Cache

Uma variação que merece destaque porque explica uma decisão deste projeto. Implementações de AES por tabela têm tempo de acesso dependente do dado: se a entrada da tabela está em cache, é rápida; se não, lenta. E o índice da tabela depende da chave.

Bernstein (2005) e Osvik–Shamir–Tromer (2006) demonstraram recuperação de chave AES a partir apenas de temporização de cache, inclusive remotamente.

É por isso que neste projeto **as caches do processador estão desligadas**. Elas seriam inúteis (a memória interna já responde em um ciclo) e nocivas (introduzem tempo dependente do dado). Duas razões independentes, ambas suficientes.

16.3 Consumo de energia

Kocher, Jaffe e Jun (1999) introduziram a análise de potência:

SPA (*Simple Power Analysis*) — observar um único traço de consumo e ler a estrutura da operação. Numa exponenciação ingênua, quadrado e multiplicação têm assinaturas diferentes, e o traço *desenha* o expoente.

DPA (*Differential Power Analysis*) — coletar milhares de traços com entradas variadas e correlacionar estatisticamente com uma hipótese sobre alguns bits da chave. A hipótese certa produz correlação; as erradas, ruído. Funciona mesmo quando o sinal é muito menor que o ruído, porque a estatística acumula.

DPA é devastadora contra implementações desprotegidas e é a razão de existir uma indústria inteira de contramedidas: **mascaramento** (dividir valores intermediários em partilhas aleatórias), **blinding** (randomizar a entrada de forma reversível), **embaralhamento** da ordem de operações, lógica de trilha dupla, e geradores de ruído.

16.4 Emissão eletromagnética

Mesmo princípio da análise de potência, mas captado por sonda próxima em vez de resistor em série. Vantagem para o atacante: é **local** — dá para posicionar a sonda sobre um bloco específico do chip e isolar o sinal que interessa, o que é especialmente eficaz contra circuitos grandes.

16.5 Onde este projeto está

Sem defesa de canal lateral **físico**, e vale ser explícito: o AES no fabric não é mascarado, não há gerador de ruído, e nada no projeto tenta igualar consumo de corrente.

O que existe são as defesas que custam pouco e valem sempre:

- **caches desligadas**, para que o padrão de acesso à memória não vire sinal;
- **uma única comparação de segredo em todo o firmware, e ela é de tempo constante.** Não é zelo espalhado — é o contrário: ter duas comparações, uma segura e uma comum, é exatamente como se escolhe a errada no caminho raro. Vale para MAC, KCV e vetores de teste igualmente, inclusive onde não há segredo nenhum a proteger;
- **um único código de erro para toda recusa de importação de key block**, pelo mesmo motivo pelo qual o oráculo de padding existe: o atacante não precisa da chave, precisa que a vítima diga em qual etapa parou.

As duas últimas são canal lateral **lógico** — vazamento por tempo e por mensagem de erro. São as que dá para fechar em firmware, e estão fechadas. O que fica aberto é o físico, e a Fase 6 do plano ataca o próprio dispositivo — é lá que o assunto vira prático.

17. Injeção de falha

Se observar não basta, **estrague o cálculo e observe o erro**. É uma das famílias de ataque mais eficientes contra hardware criptográfico.

17.1 O ataque Bellcore

O exemplo mais elegante da área. Boneh, DeMillo e Lipton (1997) mostraram que **um único erro** numa assinatura RSA feita com o Teorema Chinês do Resto revela a fatoração da chave.

O RSA-CRT calcula duas metades, uma módulo `p` e outra módulo `q`, e as combina. Se uma das metades sair errada por causa de uma falha induzida, a assinatura resultante `s` satisfaz uma congruência e não a outra. Então:

```
    gcd(s^e - m  mod N,  N)  =  p
```

Uma assinatura defeituosa, um cálculo de máximo divisor comum, e a chave privada de 2048 bits está fatorada.

Isso é imensamente relevante para a Fase 7 deste projeto, que implementa RSA-2048 por Montgomery nos DSPs. **A contramedida padrão é verificar a assinatura antes de emiti-la** — gastar uma exponenciação pública (barata, expoente pequeno) para garantir que a privada saiu certa.

17.2 Análise diferencial de falhas em cifras de bloco

Para AES, Piret e Quisquater (2003) mostraram que uma falha injetada nas últimas rodadas restringe drasticamente o espaço de chaves: poucos pares correto/defeituoso bastam para recuperar a chave de rodada, e dela a chave mestra.

17.3 Como se injeta a falha

Método	Custo	O que faz
Glitch de clock	muito baixo	Pulso curto demais faz a CPU pular ou corromper uma instrução
Glitch de tensão	baixo	Queda breve na alimentação, mesmo efeito
Pulso eletromagnético	médio	Sem contato, localizado
Laser	alto	Precisão de célula individual, exige decapsulação
Temperatura	baixo	Fora de faixa, memórias e lógica ficam instáveis

O glitch de clock é o mais didático e o mais barato. Se o pulso cair no instante certo, a CPU pula uma instrução — e se a instrução pulada for o `if` que compara o PIN, ou o que verifica a exportabilidade da chave, você contornou a verificação sem tocar em criptografia alguma.

Por que um clock rápido produz erro. Entre dois flip-flops existe lógica, e o sinal leva tempo para atravessá-la. Para o segundo flip-flop capturar o valor certo, vale:

```
T_clock  >=  t_saida + t_logica + t_setup
```

Se o período encolhe abaixo disso, a borda chega antes do resultado. O flip-flop guarda o valor antigo, ou uma mistura de bits novos e antigos. Nada quebra e nada avisa: o circuito segue funcionando, com um valor errado dentro.

Quatro consequências explicam o comportamento de um ataque real:

- **Só os caminhos longos falham.** Um período um pouco curto atinge os poucos caminhos mais lentos e poupa o resto. Por isso a falha é seletiva.
- **O que falha depende do que está ativo.** Um caminho só erra se seu valor estiver mudando naquele ciclo, e por isso o instante do pulso escolhe o alvo.

- **A ferramenta de timing não diz onde falha.** Ela calcula o pior caso de processo, tensão e temperatura. Um chip típico, frio e bem alimentado aguenta bem mais do que a frequência garantida.
- **Existe uma janela útil.** Pouco abaixo do limite nada acontece, e muito abaixo a CPU busca instruções erradas e trava. Só no meio a CPU continua viva com um resultado errado, que é o que o atacante quer.

Injeção de falha não é canal lateral. Canal lateral observa um dispositivo que funciona certo. Injeção de falha perturba o dispositivo para que ele funcione errado.

17.4 Contramedidas

- **Dupla execução e comparação** — calcular duas vezes e confrontar. Dobra o custo e derrota falhas isoladas. Não derrota duas falhas iguais: se o pulso atingir as duas execuções do mesmo jeito, elas concordam no erro.
- **Verificação do resultado** — refazer a conta pelo caminho inverso, como decifrar o que acabou de cifrar. No RSA, checar a assinatura antes de emitir. É mais forte que repetir, porque o caminho inverso não erra do mesmo jeito.
- **Sensores** — detectores de frequência fora de faixa, de tensão, de temperatura, de luz. Ao disparar: zeroize.
- **Redundância na codificação** — representar estados críticos com valores distantes em distância de Hamming, para que um bit trocado não converta "recusado" em "aceito".

Dupla execução só vê o que difere entre as execuções. Essa é a regra que decide se a contramedida funciona. Compare os dois arranjos:

```
errado:   carrega chave -> cifra -> cifra -> compara
certo:    carrega chave -> cifra -> carrega chave -> cifra -> compara
```

No primeiro, a carga da chave é um **ponto comum**: acontece uma vez, e as duas execuções herdam o resultado dela. Uma falha ali produz uma chave expandida errada, as duas execuções calculam certo sobre essa chave errada, e o comparador vê dois resultados iguais. O nome disso é **falha de modo comum**, e é o mesmo problema de dois motores de avião alimentados pelo mesmo tanque.

Por isso a pergunta útil não é "calculei duas vezes?", e sim "o que as duas execuções compartilham?". Tudo o que for compartilhado fica sem proteção:

Ponto comum	Como cobrir
Carga e expansão da chave	Repetir a carga dentro de cada execução
Entrada já lida da memória	Verificar antes, por CRC ou MAC do pedido
Saída depois da comparação	Comparar o mais tarde possível, e proteger o transporte
A própria comparação	Comparar de forma redundante, com valores distantes

O que não dá é fechar tudo. A última linha mostra por quê: o comparador é mais código, e pode ser atacado também. Contramedida de falha aumenta o custo do ataque, sem eliminá-lo. Repetir exige também que as execuções fiquem separadas no tempo além da largura do pulso, ou um pulso só atinge

as duas.

Isso também mostra por que contramedida se valida atacando. Em simulação, a falha é posta onde quem escreveu o teste imaginou, e por isso o teste passa. O ataque real cai onde a física deixa, inclusive nos pontos comuns.

O último ponto da lista é subestimado. Se `AUTORIZADO = 1` e `NEGADO = 0`, um único bit trocado inverte a decisão. Se `AUTORIZADO = 0x5A` e `NEGADO = 0xA5`, é preciso trocar muitos bits coordenadamente.

17.5 Onde este projeto está

O gerador de clock deste projeto trata a perda de lock do MMCM como violação: reafirma o reset, retém o evento, apaga as chaves e vai para `TAMPERED`. Mas o detector só vê o lock. Um pulso que encurta o clock sem derrubar o `LOCKED` passa por ele.

A Fase 6 faz esse ataque contra o próprio dispositivo. A reconfiguração dinâmica do MMCM, que o plano previa, exige reset e para o clock. O injetor usa então um segundo clock, mais rápido, e um multiplexador de clock global. Fica num bitstream separado, nunca no de produção, e faz o papel do equipamento externo de quem ataca: escuta a linha serial, conta bytes, espera um atraso e dá o pulso.

Isso é um modelo, com uma diferença importante. Um pulso aplicado de fora, no pino do oscilador, passaria pelo MMCM, que o filtraria. Quem atacasse esta placa de verdade usaria mais provavelmente a tensão. O injetor interno reproduz o efeito, a violação de timing, e não o caminho físico.

A medição contou a história da seção 17.4 na prática. Sem contramedida, o dispositivo devolveu resultados errados com CRC válido. Com dupla execução e a chave carregada uma vez só, continuou devolvendo, e nenhuma discordância apareceu: era a falha de modo comum. Com a chave carregada a cada execução, os resultados errados sumiram das medições e as recusas apareceram exatamente nos momentos das duas cargas. A falha não deixou de acontecer. Passou a ser vista.

Quando as execuções discordam, o comando é recusado, nada sai, e o equipamento marca **erro grave**. As chaves não são apagadas. Numa violação detectada por sensor, apagar é a resposta certa. Aqui o aparelho só sabe que uma conta deu errado uma vez, e recomissionar nem sempre é viável ou barato. Por isso a decisão fica com quem opera: o flag fica visível até o próximo boot, e o log guarda o comando.

18. Ataques físicos e antitamper

O degrau mais alto da escada: o atacante tem o aparelho na bancada.

18.1 A escada da resistência

Nível	Nome	O que significa
1	**Tamper-evident**	A violação deixa marca visível: lacre, resina que estilhaça, tinta que mancha
2	**Tamper-resistant**	A violação é difícil: resina dura, blindagem, parafusos especiais
3	**Tamper-responsive**	O dispositivo **detecta e reage**: malha ativa que zeroiza ao ser rompida
4	**Environmental failure protection**	Detecta condições anormais — temperatura, tensão — e zeroiza antes de falhar

O salto conceitual está entre 2 e 3. Resistência apenas ganha tempo; **resposta** destrói o alvo. Um HSM de alto nível tem uma malha de trilhas finíssimas envolvendo a eletrônica, monitorada continuamente. Furar, esmerilhar ou dissolver a resina rompe a malha, e a chave desaparece antes que se chegue nela.

18.2 Por que a bateria importa

A malha precisa estar viva mesmo com o equipamento desligado — senão o ataque é trivial: corte a energia, abra com calma. Por isso HSMs de alto nível têm bateria interna que mantém detectores e memória de chaves.

E daí decorre um comportamento que parece defeito e é projeto: **um HSM cuja bateria descarregou acorda zeroizado e inútil.** É a falha segura correta.

Este projeto reproduz isso por acidente de hardware, e o acidente é instrutivo: no core board QMTECH usado, o pino `VCCBATT` está ligado ao rail de 1,8 V, sem bateria. Uma chave de bitstream em BBRAM se perde a cada desligamento e teria de ser recarregada por JTAG (seção 18.5).

Aqui isso é desejável — nada persiste no hardware, tudo é recuperável — e é exatamente o comportamento de um HSM comercial com bateria descarregada.

18.3 Ataques invasivos

- **Decapsulação** — remover o encapsulamento com ácido ou plasma, expondo o die.
- **Microssondagem** — colocar sondas em trilhas internas e ler barramentos diretamente.
- **FIB** (*Focused Ion Beam*) — cortar e refazer ligações no próprio die; permite, por exemplo, desconectar um sensor.
- **Imageamento** — ler ROM ou fusíveis opticamente, ou por microscopia eletrônica.
- **Cold boot** — resfriar a memória para retardar a perda de conteúdo e lê-la em outro equipamento. Relevante para DRAM, e mais um motivo para chaves não morarem em memória externa.

18.4 O que este projeto não tem

Absolutamente nada disso. Não há resina, malha, sensor, bateria ou blindagem. O FPGA está numa placa de desenvolvimento com conectores de expansão expostos.

Vale registrar sem rodeios: **o modelo de ameaça deste projeto vai até o adversário que fala com a API.** Contra qualquer coisa acima disso, ele não oferece defesa — e não pretende.

O valor está em entender por que os degraus existem. Um leitor que termine esta parte sabendo *o que* uma malha ativa faz, *por que* ela precisa de bateria e *por que* um HSM sem bateria acorda inútil aprendeu algo que nenhum diagrama de blocos ensina.

18.5 Proteger o bitstream, e por que a produção não usa

O bitstream é o arquivo que configura o FPGA, e aqui ele carrega também o firmware. Quem tira a flash da placa lê tudo isso: o circuito, o código e os vetores de teste. Cifrar o bitstream fecha essa leitura.

O mecanismo tem duas partes:

- **Uma chave dentro do FPGA.** Ela nunca sai dali. O bitstream chega cifrado, um bloco de hardware dedicado decifra e confere a autenticidade, e só então o FPGA se configura. Quem lê a flash só vê bytes sem sentido.
- **Um lugar para guardar essa chave.** Existem dois, e a diferença entre eles é o centro da questão:

	BBRAM	eFUSE
O que é	memória mantida por uma bateria	fusíveis queimados uma única vez
Sem energia	a chave some	a chave fica para sempre
Erro	apaga e grava de novo	placa perdida
Neste projeto	permitida	proibida pela regra nº 1

A chave entra na BBRAM só pelo JTAG. Num produto real, isso acontece uma vez, na fábrica, e a bateria mantém a chave daí em diante. O equipamento liga sozinho: lê a flash, decifra e funciona. O JTAG não faz parte do uso.

O que a placa mostrou. O mecanismo foi provado com o experimento de RO-PUF. A placa só aceitou o bitstream com a chave certa na BBRAM. Com outra chave, ou com a BBRAM vazia, ela recusou a configuração.

Por que a produção segue sem cifra. Este é um dispositivo didático, e a decisão do autor foi manter o bitstream de produção sem cifra, por dois motivos:

1. **Sem bateria, cifrar transformaria cada desligamento em manutenção.** Nesta placa a chave some ao desligar (seção 18.2). Com a flash cifrada, o equipamento não subiria sozinho, e cada religamento exigiria gravar a chave de novo pelo JTAG. Sem cifra, ele liga e funciona sem cabo nenhum.
2. **Cifrar não protegeria o que importa aqui.** A cifra cobre o bitstream, e a LMK está em outra região da mesma flash (Fase 4). Quem lê a flash continuaria lendo a chave mestra. O código deste projeto, por sua vez, é aberto.

A lição vale para qualquer equipamento: **um controle só protege o que está dentro do seu alcance, e todo controle cobra um custo de operação.** Antes de ligar um, pergunte o que ele cobre e quem vai pagar por ele todos os dias.

Parte IV — Certificação

Um HSM comercial não é vendido pelo que faz, e sim pelo que **um terceiro verificou** que ele faz. Esta parte explica o que os selos significam — e, igualmente importante, o que não significam.

19. FIPS 140-3

A norma dos Estados Unidos para módulos criptográficos, hoje alinhada à ISO/IEC 19790. Substituiu a FIPS 140-2, e a validação é feita pelo CMVP (programa conjunto NIST/CCCS) por meio de laboratórios credenciados.

A norma define **quatro níveis**, e a diferença entre eles é quase toda física.

Nível 1

Requisitos mínimos. Algoritmos aprovados, implementados corretamente. **Sem exigência de segurança física** — uma biblioteca de software rodando num PC comum pode ser validada em nível 1.

Utilidade real: garante que o AES é AES de verdade e que os testes de resposta conhecida existem. Não diz nada sobre proteger a chave de quem tem acesso à máquina.

Nível 2

Acrescenta **evidência de violação**: lacres, revestimentos ou invólucros que mostrem que alguém abriu. E autenticação **por papel** — o módulo distingue "operador" de "administrador".

A palavra-chave é *evidência*: o nível 2 não impede a violação, garante que ela deixe marca.

Nível 3

Aqui a coisa fica séria. Acrescenta:

- **Resistência e resposta à violação** — o módulo detecta a intrusão e **zeroiza** os parâmetros críticos de segurança;
- **autenticação por identidade** — não basta o papel, é preciso saber quem;
- **separação física ou lógica** das interfaces por onde entram e saem parâmetros críticos.

A maior parte dos HSMs comerciais de propósito geral é validada em nível 3.

Nível 4

O topo. Um **envelope completo de proteção**: qualquer tentativa de acesso físico, de qualquer direção, é detectada e responde com zeroização. E acrescenta **proteção contra falhas ambientais** — o módulo detecta tensão ou temperatura fora da faixa e zeroiza antes que a condição possa ser explorada.

O nível 4 é caro, e existem poucos produtos. É a resposta direta aos ataques descritos na seção 17: se você pode congelar o chip ou baixar a tensão para induzir falhas, o módulo precisa perceber e se destruir primeiro.

O vocabulário que a norma impõe

Vale conhecer, porque aparece em toda documentação da área:

- **CSP** (*Critical Security Parameter*) — chave em claro, dado de autenticação, qualquer coisa cuja divulgação comprometa a segurança.
- **Estado de erro** — condição em que o módulo entra ao falhar um autoteste e na qual **não realiza operações criptográficas**.
- **Modo aprovado** — o conjunto de operações e algoritmos cobertos pela validação. Um módulo pode ter funções fora do modo aprovado; usá-las tira você da conformidade.
- **Autotestes** — os KAT da seção 11, exigidos na inicialização e sob condições específicas.

20. PCI: as regras de pagamentos

Enquanto FIPS avalia o **módulo**, o mundo de pagamentos regula também **como ele é operado**.

PCI PTS HSM define requisitos de segurança para o equipamento usado em processamento de PIN e chaves de pagamento.

PCI PIN Security Requirements cobre a operação: como chaves são geradas, transportadas, carregadas, trocadas e destruídas. É daqui que vêm, como exigência contratual e não como boa prática:

- **dual control** — nenhuma operação sensível de chave por uma pessoa só;
- **split knowledge** — nenhuma pessoa conhece uma chave inteira;
- **cerimônia documentada**, com testemunhas e registro;
- **key blocks** para transporte de chave — e há prazos de migração obrigatória para formatos como o TR-31.

É por isso que este projeto modela LMK, componentes e dual control com botões físicos: no mundo de pagamentos esses mecanismos são **requisito auditado**, não escolha de arquitetura.

21. Common Criteria

Norma internacional (ISO/IEC 15408) com abordagem diferente: em vez de níveis fixos, define **perfis de proteção** — descrições do que um tipo de produto deve fazer — e avalia produtos contra eles, com um nível de garantia (**EAL 1 a 7**) que indica o rigor da avaliação, não a força da segurança.

Confusão comum, e vale desfazer: **EAL alto não significa "mais seguro"**. Significa "avaliado com mais rigor e mais documentação formal". Um produto simples avaliado em EAL 6 pode proteger menos que um produto rico em EAL 4 — o que o EAL mede é a confiança na avaliação, não a altura do muro.

Na Europa, perfis como o EN 419221-5 cobrem módulos criptográficos para serviços de confiança, e são o caminho para dispositivos de assinatura qualificada sob o eIDAS.

22. O que “certificado” significa — e o que não significa

Esta seção é a mais importante da Parte IV.

Significa

- Um laboratório independente verificou as afirmações do fabricante.
- Os algoritmos foram testados contra vetores oficiais.
- Existe documentação de fronteira, de estados, de papéis e de zeroização.
- Há um processo de reavaliação quando o produto muda.

Não significa

Não significa “inviolável”. Certificação estabelece um piso verificado, não um teto. Módulos certificados já foram quebrados — e certificados são revogados ou emendados quando isso acontece.

Não significa que a sua configuração está segura. A validação cobre o módulo operando em **modo aprovado**, numa configuração específica descrita na política de segurança. Usá-lo fora dali é usar um produto não validado com um adesivo de validado.

Não significa que o sistema em volta está seguro. O HSM protege a chave. Se a aplicação assina qualquer coisa que peçam, o atacante não precisa da chave — ele pede a assinatura. Essa é, na prática, a falha mais comum em implantações reais: um HSM impecável atrás de uma API de aplicação que autoriza demais.

Não significa nada sobre canal lateral, salvo se explicitado. Resistência a DPA é avaliada separadamente e nem sempre está incluída.

A moral para este projeto

Não há nenhuma pretensão de certificação aqui, e nem poderia haver. Mas conhecer a estrutura das normas explica **por que** as fases do plano estão na ordem em que estão:

- os autotestes da Fase 2 são o requisito de autoteste do FIPS;
- a máquina de estados da Fase 3 é a separação de papéis;
- o zeroize com prova é o requisito de zeroização;
- o log de auditoria é a evidência;
- os sensores da Fase 6 são o nível 3/4 de resposta física.

Implementar cada mecanismo e depois ler o parágrafo da norma que o exige é uma forma muito eficiente de entender a norma — e de perceber que ela não é burocracia arbitrária, mas cicatriz acumulada.

Parte V — O equipamento hsm-a7

23. Objetivo, plataforma e restrições

O objetivo

Construir, do zero, um módulo criptográfico com hierarquia de chaves, cerimônia de carga, key blocks e API de host — reproduzindo a arquitetura de um HSM real em escala reduzida, para **entendê-la de dentro para fora**.

A escolha de fazer em FPGA, e não em software, é deliberada: obriga a enfrentar a fronteira física, a distinguir memória interna de externa, e a lidar com clock e reset como questões de segurança e não de infraestrutura.

A plataforma

Item	Valor
FPGA	Xilinx Artix-7 **XC7A35T**, encapsulamento FTG256, speed grade -1
Placa	QMTECH core board + daughterboard
Recursos	20.800 LUTs, 41.600 flip-flops, 50 BRAMs, 90 DSP48E1
Clock	50 MHz no pino N11 → 100 MHz por MMCM
Processador	NEORV32 (RISC-V RV32IMC), submódulo fixado em v1.13.3
Canal	UART 115200 8N1 pelo CP2102 do core board
Flash	MT25QL128, 16 MB, a mesma que guarda o bitstream

O orçamento de fabric é apertado de propósito: AES, SHA, key store e processador precisam caber em 20.800 LUTs, e isso força decisões explícitas em vez de acumulação.

As restrições invioláveis

Estas não são preferências. Estão no topo do plano e do arquivo de instruções do projeto.

1. Nada de eFUSE. Não gerar, sugerir ou executar `program_efuse`, `CFG_AES_ONLY`, `W_DIS` / `R_DIS` ou qualquer variante. São memórias de programação única: o erro é um *brick* permanente. Chave de bitstream apenas em BBRAM, que é recuperável.

2. Chaves em claro só em BRAM e na região segura da flash. Nunca DDR3, nunca um registrador exposto no toplevel. A região da flash a partir de 4 MB faz o papel da memória com bateria de um HSM comercial: a LMK fica nela em claro, e as chaves de trabalho, embrulhadas sob a LMK. A seção 27 explica por que, e o que isso não protege.

3. Nada atravessa a fronteira em claro. Saem apenas key blocks embrulhados, KCVs, criptogramas, *handles* e log.

4. Simulação antes de hardware. Nenhum módulo vai para a placa sem testbench passando. Depurar criptografia por UART, sem visibilidade interna, é a forma mais lenta possível de descobrir que faltou um byte no preenchimento.

5. Não enfraquecer testes para fazer algo passar. Se um KAT falha, o bug está no código, não no vetor.

A regra do eFUSE merece um comentário, porque ela ilustra um princípio geral: **preferir sempre o mecanismo recuperável ao irreversível, num projeto de estudo**. O JTAG também nunca é desabilitado — é a única via de recuperação, e um projeto de aprendizado precisa poder voltar atrás. Um produto real faria o oposto em ambos os casos, e entender por que os dois caminhos são corretos em contextos diferentes é parte do conteúdo.

24. A plataforma, peça por peça

Esta seção pega cada peça da plataforma e responde: **o que essa peça corresponde num HSM real?**

Várias terminam com *o que um HSM real tem a mais*. A distância entre este equipamento e um produto também é conteúdo.

Mapa

O que construímos	O que é num HSM real
`clk_rst_gen.v`	Monitor de integridade do clock (sensor antitamper)
`debounce.v`	Integridade da autorização física (dual control)
`neorv32_wrapper.vhd`	Endurecimento da plataforma: o que a máquina **não** faz
`hsm_top.v`	Definição física da fronteira criptográfica
`cmd.c`	A API — a superfície de ataque principal
CRC32	Integridade de transporte (num HSM real, viraria MAC)
`state.c`	A chave de switch de três posições
`wipe.c`	Requisito de zeroização do FIPS 140-3
LEDs D1–D5	Indicadores de estado e de violação
`BOOT_MODE_SELECT=2`	Autenticidade de firmware / secure boot
IMEM como ROM	Integridade do código em execução
`tb_soc_silent`	Higiene de canal lateral
`hsmtool.py`	O cliente da API — futuro PKCS#11
Teste de 10.000 pings	Requisito de confiabilidade operacional

24.1 `clk_rst_gen.v` — o clock é um sensor

MMCM que transforma 50 MHz em 100 MHz (VCO em 1000 MHz, meio da faixa do speed grade -1), mais geração de reset: afirma assíncrono, libera síncrono, segura 64 ciclos após o lock.

Num HSM real: monitoramento de clock é sensor antitamper. Ver seção 17.3 — glitch de clock é o ataque de injeção de falha mais barato que existe, e faz a CPU pular exatamente a instrução que verifica alguma coisa.

O ponto de projeto está numa linha:

```
    wire rst_src_n = locked_w & rst_n_sync[1];
```

Perda de `LOCKED` **reafirma o reset**. Não sabemos por que o lock caiu, mas sabemos que não dá para confiar no que vier depois.

O gerador é também a única exceção à convenção de reset síncrono do projeto, e por necessidade: com o MMCM em reset, o clock de saída não está correndo e não existe borda para registrar nada. Usa o padrão clássico *afirma assíncrono / libera síncrono*, justamente para que todo o resto do projeto possa usar reset puramente síncrono.

A perda de lock também fica retida num flip-flop sem reset. O módulo pulsa o `RST` do MMCM sozinho, porque o MMCM não volta a travar sem isso. O boot seguinte encontra a perda, apaga as chaves, vai para `TAMPERED` e registra o evento (seção 29).

Um HSM real tem a mais: monitoração ativa da frequência, em vez de só reagir à perda de lock.

24.2 `debounce.v` — quando um ressalto vira autorização

Exige 10 ms de estabilidade contínua antes de aceitar mudança; qualquer transição no meio zera a contagem.

Num HSM real: integridade da autorização física (seção 7). O mecanismo precisa refletir intenção humana, não estado elétrico de contato.

Os dois botões escolhidos são o **2º e o 5º** de uma fileira de cinco, cujo primeiro é o reset — o par mais afastado disponível, o que dificulta pressionar ambos com uma mão só.

A camada de política acima deste filtro está na seção 26: os dois botões autorizam a cerimônia de carga da chave mestra, e não basta que estejam pressionados — cada autorização exige um aperto *novo*. O `debounce` garante que o contato parou quieto; a política garante que o operador soltou.

Um HSM real tem a mais: autenticação de custodiante (smartcard + PIN) e registro de *quem* autorizou, não só de que houve autorização. Dois botões na mesma placa cabem nas duas mãos de uma pessoa — isto prova presença física, não consentimento de duas pessoas.

24.3 `neorv32_wrapper.vhd` — segurança é o que se desliga

Fixa os ~130 generics do processador. É o único lugar onde a configuração do SoC está escrita — e num relatório de avaliação, é exatamente essa lista que o laboratório examina.

Desligado	Por quê
`OCD_EN`	Debugger dá leitura e escrita irrestritas de memória por JTAG. Com a LMK em BRAM, **é** extração de chave sem passar pela API. O generic mais importante do arquivo.
`ICACHE` / `DCACHE`	Inúteis (BRAM já responde em 1 ciclo) e nocivas: tempo dependente do dado é canal lateral (seção 16.2).
`XBUS` / `SMC`	Sem barramento externo, não há caminho físico até a DDR3. A regra vira estrutura.
`IO_TRACER`	Buffer de rastreamento de execução é observabilidade indesejada.
`Zkne` / `Zknd` / `Zknh`	AES e SHA vão para o fabric via CFS: construí-los é o objetivo do projeto.

Sobre desligar o debugger, note a assimetria com a regra de nunca desabilitar JTAG no FPGA: lá é o caminho de recuperação do **bitstream**, e precisa existir num projeto de estudo. Aqui é o caminho de leitura da **memória do processador**, e não precisa. Duas coisas diferentes com o mesmo nome.

Ligados: RV32IMC (`RISCV_ISA_C` + `_M`), contadores base, barrel shifter (código criptográfico é pesado em deslocamento), IMEM 32 KB, DMEM 16 KB, UART0 com FIFO de 64, temporizador, GPIO de 8 bits e o SPI da flash.

Um HSM real tem a mais: PMP (proteção de memória física) para separar privilégio dentro do firmware, de modo que um bug no parser não alcance a região da LMK. Este equipamento não usa PMP.

24.4 `hsm_top.v` — onde a fronteira fica física

Único arquivo do projeto que conhece nome de pino. Corresponde à **definição da fronteira criptográfica** — o desenho que num documento FIPS marca dentro e fora, com a lista de todas as portas.

Aqui a lista é curta e verificável:

Porta	Direção	O que atravessa
`uart_rxd_i` / `uart_txd_o`	ambas	comandos e respostas
`btn_a_i` / `btn_b_i`	entra	autorização física
`led_o` , `seg_o` , `seg_an_o`	sai	estado

E só. Não existe porta de dados além da UART, e o que passa por ela é definido pela tabela de comandos.

O toplevel também absorve, num ponto só, as duas inversões da placa — LEDs e botões são ativos em nível baixo. Parece cosmético e não é: **um LED de `TAMPERED` invertido é defeito de segurança**, porque o dispositivo mostraria “tudo bem” exatamente quando não está.

24.5 `cmd.c` — o parser é a superfície de ataque

Máquina de estados de recepção, verificação de CRC, tabela de comandos com verificação de estado, montagem da resposta.

Num HSM real: esta é a API, e é onde mora a maior parte dos ataques reais (seção 15).

Por isso o checklist obrigatório para todo opcode novo:

- Em quais estados é permitido?
- Exige dual control?
- **O que pode vazar se for chamado em laço com entradas escolhidas?**
- Respeita a exportabilidade do slot?
- Entra no log **antes** da execução?

O parser é deliberadamente burro: máquina de estados explícita, todos os limites verificados, sem alocação dinâmica, sem recursão. É o único código que processa entrada arbitrária antes de qualquer verificação — não é lugar para esperteza.

O timeout entre bytes é o que satisfaz o critério "frame malformado nunca trava a máquina de estados". Sem ele, um host que morre no meio de um frame deixa o parser esperando para sempre e o dispositivo mudo: negação de serviço com um único byte enviado. É rearmado a cada byte, e não por frame, porque um key block TR-31 leva cerca de 45 ms a 115200 baud e um timeout de frame teria de ser afrouxado até deixar de proteger.

Disponibilidade é requisito num HSM: um módulo que para de responder derruba a autorização de transações.

O log de auditoria registra cada comando antes da execução (seção 27).

Um HSM real tem a mais: tempo de resposta constante. Aqui um comando recusado responde mais rápido que um aceito, e isso é informação.

24.6 O CRC32, e por que num HSM real seria um MAC

Detecta corrupção de enquadramento. Polinômio IEEE 802.3 refletido, idêntico ao `zlib.crc32`.

Distinção que vale internalizar: CRC é detecção de erro, **não** autenticação. Quem altera o payload recalcula o CRC trivialmente.

Num HSM de pagamentos, o canal com o host costuma ser autenticado de verdade — MAC sobre a mensagem com chave de sessão, ou TLS mútuo.

Aqui o CRC basta, e o motivo é conceitualmente importante: **a fronteira de confiança está no dispositivo, não no canal**. Assumimos que o host pode ser hostil, e um host hostil não precisa falsificar CRC — ele manda comandos válidos. A defesa não é autenticar o canal; é a máquina de estados, o dual control físico e os atributos de chave.

É por isso que dual control é feito com botões na própria placa e não com um comando "eu autorizo" vindo pelo cabo.

Duas decisões de formato que o plano deixava em aberto e precisavam ser fixadas antes de o host existir:

- **o CRC cobre `LEN` + corpo** — incluir o `LEN` importa, porque é o campo que um atacante usaria para induzir leitura fora do buffer;
- **polinômio idêntico ao `zlib.crc32`** — mantém o host trivial e elimina uma fonte clássica de divergência.

O CRC existe em três implementações independentes: C no firmware, Verilog no testbench, Python no host. Divergência apareceria como erro de CRC intermitente — dos sintomas mais enganosos que existem, porque parece problema de cabo.

24.7 `state.c` — a chave de switch

Cada linha da tabela de comandos carrega a **máscara de estados** permitidos para aquele comando. O despachante consulta a máscara antes de chamar o handler. Nenhum handler verifica estado por conta própria.

```
#define ST_NORMAL  (ST_UNINIT | ST_AUTH | ST_OPER)
```

`TAMPERED` fica fora de propósito: dispositivo comprometido responde o mínimo possível. A única exceção é o `SELFTEST`, e ela é deliberada — sem ela o operador teria um LED vermelho e nenhuma informação sobre o quê reprovou.

A cerimônia de carga da chave mestra (seção 26) move o dispositivo pela escada `UNINITIALIZED` → `AUTHORIZED` → `OPERATIONAL`. Cada passo muda o conjunto de comandos aceitos sem nenhum código que ligue ou desligue comandos: a tabela decide.

É a diferença entre política declarada e política executada. Um `if` dentro de cada handler seria a mesma regra escrita uma vez por comando — e cada cópia é um lugar onde alguém a esquece.

24.8 `wipe.c` — zeroização antes de haver chave

Ponteiro `volatile` mais barreira de memória, pelo motivo explicado na seção 12.

Os buffers de frame são limpos após cada comando, porque são eles que carregam componente de LMK e key block.

O `ZEROIZE` usa `wipe_padrao()`, que escreve um padrão antes dos zeros. A razão não é a que o folclore de disco magnético sugere: para memória estática a passada de zeros é a que conta. A primeira existe porque uma zeroização **interrompida** tem de deixar padrão, não meia chave.

A prova de que apagou vem de duas verificações independentes, e a segunda é a interessante — o KCV. Seção 26.

24.9 Os LEDs, e por que D1 é em hardware

D1 pisca a 1 Hz por lógica no fabric; D2 a D5 são GPIO controlado pelo firmware.

A decisão interessante: **D1 é independente da CPU**. Se o firmware travar, D1 continua piscando. Isso distingue duas falhas que de fora parecem iguais:

Sintoma	Diagnóstico
D1 parado	clock morto — MMCM sem lock, alimentação, bitstream
D1 piscando, D2 apagado	clock bom, firmware pendurado
D1 piscando, D2 aceso	plataforma viva; o problema é protocolo ou host

Num dispositivo sem console, essa é a diferença entre depurar e adivinhar. É o mesmo princípio do watchdog independente: o indicador de vida não pode depender da coisa cuja vida ele indica.

Bônus de verificação: D1 a 1 Hz cronometrado a olho **confirma a divisão do MMCM**, porque o contador é `CLK_HZ/2` e só bate 1 Hz se o clock for mesmo 100 MHz.

24.10 `BOOT_MODE_SELECT = 2` — autenticidade de firmware

O SoC sobe a partir de uma imagem embutida na IMEM, não do bootloader do NEORV32.

Num HSM real: isto é *secure boot*, um dos requisitos mais duros de certificação.

O bootloader do NEORV32 aceita upload de firmware pela UART. Com ele ligado, qualquer um com acesso ao serial substitui o firmware — e firmware substituído contorna a máquina de estados, o dual control, a exportabilidade e a fronteira inteira. **Todas as defesas discutidas neste documento passam a valer zero**, porque quem escreve o código escreve as regras.

Num HSM comercial isso se resolve com boot ROM imutável que verifica assinatura da imagem antes de executá-la, com a chave pública em memória de programação única.

A versão deste projeto é mais simples e mais rude: **não há caminho de atualização**. O firmware está no bitstream; trocá-lo exige regerar o bitstream e gravar por JTAG, o que é acesso físico.

Efeito colateral que veio de graça: com esse modo, o NEORV32 liga `MEM_INIT` e a IMEM vira memória **somente de leitura**. A memória de código não é gravável em tempo de execução — não há injeção de código nem código automodificável, independentemente de qualquer bug no parser.

Isso é integridade de código em execução. Num sistema com MMU seria o equivalente a marcar as páginas de texto como não-graváveis; aqui é mais forte, porque não é uma permissão que alguém pode mudar — é a ausência de circuito de escrita.

24.11 `tb_soc_silent` — não falar sem ser perguntado

Verifica que o dispositivo não transmite nada pela UART até receber um comando — **e** que está vivo.

Num HSM real: higiene de canal lateral e de vazamento de identidade. Um banner de boot entrega versão e identidade a quem só escuta o barramento, o que é reconhecimento gratuito. E um `printf` de depuração esquecido no firmware é um canal lateral permanente, historicamente uma das formas mais comuns de vazamento em dispositivos embarcados.

O detalhe que torna o teste honesto: silêncio sozinho não prova nada, porque firmware travado também fica quieto. Por isso o teste exige silêncio **e** vida — `main()` precisa ter chegado ao laço e aceso o LED de atividade. Um teste que passa quando o dispositivo está morto é pior que teste nenhum.

24.12 `hsmtool.py` — o cliente da API

Codec do protocolo binário, transporte serial, CLI. É o **console**: cerimônia, estado, diagnóstico e log. O cliente das aplicações usa outra porta lógica, o command set ASCII da seção 28.

O comentário no topo do arquivo é a regra que o mantém honesto:

```
# FRONTEIRA: tudo aqui esta FORA. Este programa nunca ve chave em claro --
# so key blocks wrapped, KCVs, criptogramas, handles e log. Se algum comando
# futuro fizer material de chave aparecer neste arquivo, o comando esta
# errado, nao o script.
```

Aqui se entende por que a API de um HSM é do jeito que é: **é você que precisa impedi-la de vazar.** Quando se escreve os dois lados, a tentação de adicionar um "só para depurar, me devolva a chave" fica concreta — e recusá-la ensina mais do que ler que não se deve.

24.13 O teste de 10.000 pings

Num HSM real: disponibilidade e determinismo são requisitos operacionais. Um módulo autorizando transações tem acordo de nível de serviço; um que perde uma mensagem em dez mil produz falhas intermitentes que ninguém reproduz.

E há um aspecto de segurança: **erro intermitente esconde ataque**. Se o canal normalmente perde alguns frames, ninguém investiga o dia em que alguém está injetando tráfego. Uma taxa de erro de exatamente zero é o que torna qualquer erro digno de investigação.

Parte VI — As funções, fase a fase

Esta parte descreve o que cada fase acrescentou ao equipamento, e o conceito por trás de cada função. As medidas, os testes e o histórico da construção ficam nas notas de fase, em `doc/fase1-notas.md` a `doc/fase7-notas.md`.

25. Fase 2 — engines e self-test

Objetivo: primitivas corretas e verificáveis. Correção antes de desempenho.

Bloco	Fonte	Estado
AES-256	secworks/aes	**no CFS, verificado**
SHA-256	secworks/sha256	**no CFS, verificado**
`DNA_PORT`	primitiva Xilinx	**no CFS, verificado**
TRNG	neoTRNG (do NEORV32)	**no CFS, com health tests em RTL**
CTR_DRBG	firmware	**implementado, no POST**

Vetores oficiais

Os vetores KAT foram baixados das fontes oficiais — NIST CAVP para AES e SHA, RFC 4231 para HMAC, SP 800-90A para o DRBG — com URL e hash SHA-256 de cada arquivo registrados num manifesto. Nada foi escrito de memória.

Isso não é formalidade. A regra inviolável do projeto diz que, se um KAT falha, o bug está no código e não no vetor — e essa regra só tem força se a procedência do vetor for verificável por um terceiro. Um script rebaixa, confere hash e reextrai; sobre um repositório limpo, o resultado é byte a byte idêntico.

O AESAVS traz quatro tipos de vetor e todos foram usados, porque cada um pega uma classe diferente de defeito: `VarKey` varre cada bit da chave isoladamente e acha indexação errada na expansão; `VarTxt` varre cada bit do bloco e acha ordem de byte e permutação trocadas. Um core que passa apenas no `GFSbox` pode estar inteiro errado.

Uma divisão de responsabilidade ficou registrada e vai importar no firmware: o core de AES faz ECB, e o encadeamento de CBC é de quem chama; o core de SHA recebe blocos de 512 bits já preenchidos, e o *padding* também é de quem chama. Testar o core com essas coisas embutidas misturaria falhas de naturezas diferentes.

O coprocessador: o CFS

Os cores entraram pelo **CFS** (*Custom Functions Subsystem*) do NEORV32 — um slot de periférico projetado para receber coprocessadores. O arquivo do upstream é um template feito para ser jogado fora; a substituição é feita trocando qual arquivo o build compila, sem patch e sem tocar no submódulo.

O CFS, e não um barramento externo, por um motivo de arquitetura: o coprocessador precisa estar **dentro da fronteira criptográfica** (seção 5), não pendurado num barramento que sai do die. Coerente com `XBUS_EN => false`, que é a mesma decisão vista do outro lado.

Três escolhas do mapa de registradores merecem nota, porque cada uma é um conceito da Parte II virando linha de RTL:

- **O único fio do CFS que chega ao toplevel está amarrado em zero.** Não existe caminho físico do bloco que guarda a chave até um pino. A fronteira deixa de ser promessa de firmware e vira topologia.
- **Chave e blocos de entrada são escrita-somente**; ler devolve zero. Não protege contra a CPU, que acabou de escrever a chave — evita criar um caminho de leitura que um bug use por acidente.
- **Não há AES-128.** O tamanho de chave é fixo no hardware. Um modo mais fraco alcançável por escrita em registrador seria um *downgrade* de graça, e a seção 15 mostra o que APIs fazem com modos mais fracos disponíveis.

E há um `WIPE` que não é decorativo: ele sobrescreve a **chave expandida** dentro do core, não só o registrador visível. O teste correspondente é o que separa zeroização de teatro de zeroização — depois do wipe, cifrar sem carregar chave nenhuma tem de produzir o resultado sob a chave zero.

Health tests: o hardware vê, o firmware decide

Os testes RCT e APT (seção 10) se dividem entre hardware e firmware. A SP 800-90B exige que os testes *contínuos* vejam **toda** amostra, e a fonte produz uma por ciclo de 100 MHz. A CPU veria uma em mil. Por isso:

- **hardware** faz os testes contínuos, amostra a amostra, e congela um retrato de 1024 amostras brutas consecutivas;
- **firmware** faz os testes de partida sobre esse retrato e é dono da política — reprovou, o dispositivo vai para `TAMPERED`.

Os dois testes acabam implementados **duas vezes, de forma independente**, e uma discordância entre eles denuncia erro de interpretação da norma.

Cuidado térmico: meia dúzia de anéis osciladores, não centenas. Array grande gera calor e ruído de alimentação localizados sem ganho de entropia.

O **POST** (seção 11) roda antes de aceitar qualquer comando. Ele cobre AES-256, SHA-256, HMAC-SHA-256, CTR_DRBG, os testes de partida da fonte, CMAC-AES-256, o key store, o key block X9.143 e a verificação RSA-2048 — todos contra vetores oficiais ou de procedência registrada.

Dois autotestes ficam fora do boot, porque são lentos: o da assinatura RSA e o do ECDSA. Cada um roda **antes do primeiro uso** do algoritmo depois do boot. A FIPS 140-3 chama isso de autoteste condicional de algoritmo, e o permite. O que ela não permite é usar o algoritmo antes de testá-lo.

Falha leva a `TAMPERED`, e o dispositivo passa a atender **só** ao `SELFTEST`, que é o único jeito de o operador saber *o que* reprovou em vez de olhar um LED vermelho.

Cada propriedade que o dispositivo garante sobre si mesmo cobra milissegundos de todo boot, para sempre. O POST deste equipamento leva cerca de 28 ms.

26. Fase 3 — hierarquia de chaves

O coração do projeto, e onde os conceitos das Partes II e III viram código.

- **Key store** com os campos do cabeçalho TR-31 modelados desde o início — o MAC cobre o cabeçalho inteiro, então os campos não podem mudar depois.
- **Cerimônia de LMK** com três componentes XOR, KCV a cada passo, e dual control exigindo os dois botões.
- **Máquina de estados** com o display de 7 segmentos soletrando `Uni` / `Aut` / `OPE` / `tPr`, mais o ponto decimal confirmando o dual control.
- **Key blocks ANSI X9.143 (TR-31 versão D)**: KBEK e KBAK derivadas por CMAC, corpo em AES-CBC, autenticação por CMAC sobre cabeçalho e corpo — escritos **duas vezes**, em C e em Python.
- **Zeroize** com prova — e a prova não vem de dump.
- **Comandos de chave**: gerar dentro da fronteira, exportar embrulhado, reimportar, consultar metadados, apagar um slot, cifrar e gerar MAC com a chave referida por handle, e formar chave de trabalho por componentes. Só o último exige dual control, e a razão está mais abaixo.

A cerimônia, e o que o dual control prova

A LMK entra em **três componentes** que se acumulam por XOR: cada custodiante carrega o seu e nenhum vê os demais, e nenhum componente isolado revela nada sobre a chave. A cada passo volta o KCV **do componente** — não do acumulado — para que quem digitou errado descubra na hora, e não no fim, quando já não dá para saber qual dos três estava trocado.

Os dois botões físicos autorizam, e há um detalhe que só aparece quando se tenta atacar o próprio mecanismo: **não basta que os dois estejam pressionados**. Entre uma autorização e a seguinte, os dois têm de ser vistos **soltos**. Sem essa exigência, fita adesiva sobre os dois botões carregaria a LMK inteira sozinha, e o dual control seria teatro — e, pior, um botão travado em "pressionado" passaria a autorizar tudo em vez de nada.

A escada de estados é de **uma via só**. Carregou os três componentes, o dispositivo vai a `AUTHORIZED` e o comando de carregar deixa de existir — não há caminho para trocar a chave mestra por cima da existente. Ativou, vai a `OPERATIONAL` e o comando de ativar some. Descer exige apagar.

O painel

Durante a cerimônia, o operador precisa saber se o dispositivo vê os dois botões. O display de 7 segmentos responde. Ele soletra o estado, e o **ponto decimal acende enquanto o dual control está satisfeito**. O operador aperta e vê o dispositivo concordar, antes de gastar o comando. É a diferença entre uma autorização que se descobre pelo erro e uma que se confirma pelo gesto.

Duas notas de projeto:

- O display **não lê a máquina de estados**; ele recebe dois bits pelo GPIO. Um caminho de hardware até a estrutura de estado seria caminho de hardware até o que está ao lado dela na memória.

- O `default` do decodificador é `tPr`, não `Uni` nem apagado. Estado corrompido tem de aparecer como comprometido — falhar para o lado seguro vale também para o painel.

O key block, e a pergunta que ele responde

Uma chave que nunca sai não serve para nada além de si mesma. O key block é a resposta para "como deixar uma chave sair **sem que ela vaze**" — que é um problema diferente de guardá-la, e mais difícil.

O formato é uma string de texto:

```
cabeçalho(16 caracteres) || corpo cifrado em hex || MAC em hex

D 0144 D0 A B 00 E 00 00
| |    |  | | |  | |  +-- reservado
| |    |  | | |  | +----- blocos opcionais
| |    |  | | |  +------- exportabilidade
| |    |  | | +---------- versão da chave
| |    |  | +------------ modo de uso
| |    |  +-------------- algoritmo
| |    +----------------- uso: dados, KEK, MAC, BDK…
| +---------------------- comprimento total
+------------------------ versão do formato
```

Três decisões dele valem mais que o código inteiro.

Duas chaves derivadas, não uma. A LMK não cifra o corpo nem autentica o bloco: dela saem duas filhas por CMAC, uma para cada trabalho, distinguidas por um campo de *propósito* na entrada da derivação. É a separação de chaves da seção 15 aplicada onde ela é mais fácil de esquecer — se as duas fossem a mesma, um oráculo de MAC seria um oráculo de cifragem de graça.

O cabeçalho vai dentro do MAC. São dezesseis bytes de texto legível, à vista de qualquer um — e autenticados. Sem isso, alguém que não consegue decifrar o corpo simplesmente **edita o caractere** da exportabilidade de `N` para `E` e devolve o bloco. O criptograma não mudou; a política mudou. É o ponto que resume a Parte II inteira: chave protegida sob metadado desprotegido não está protegida.

O MAC é o vetor de inicialização. Não há IV para transmitir, e o MAC é calculado sobre o texto **claro** — o que permite recusar um bloco adulterado *antes* de acreditar em qualquer campo do que se decifrou.

E o enchimento aleatório no fim do corpo não é sobra: sem ele, o tamanho do bloco denunciaria o tamanho da chave, e "esta é uma chave de 128 bits" já é informação para quem escolhe onde gastar esforço.

Por que o mesmo formato foi escrito duas vezes

Uma implementação em C, dentro do dispositivo; outra em Python, no host, sobre uma biblioteca diferente. Não é redundância — é o **método de verificação**.

Um formato binário se aprende errado em silêncio: a implementação lê a norma, se convence, e testa contra si mesma. Duas implementações escritas para discordar não têm essa saída, e a menor divergência aparece como MAC inválido, que é barulhento e imediato. Se as duas compartilhassem o AES, concordariam sobre um erro no AES sem nunca discordar — daí a exigência de bibliotecas diferentes.

E aqui aparece um limite que vale enunciar, porque ele explica de onde vem a confiança em cada camada. **Não existe vetor oficial para este formato.** O programa de validação do NIST valida *algoritmo*; X9.143 é *formato*, e a norma que traz o exemplo é paga. O que se usa é um valor conhecido de terceiros, e o resto da confiança vem de propriedades: ida e volta, e um bit trocado em **cada posição do bloco** tendo de invalidar tudo.

Esse último é o teste que vale mais que os outros. Uma implementação que esqueceu de incluir o cabeçalho no MAC passa no vetor e passa na ida e volta. Falha só ali.

Apagar, e provar que apagou

"Chamei a função de apagar" não é a mesma coisa que "apagou", e a diferença é invisível de fora. Um `wipe` que o compilador eliminou por ser escrita morta, um campo novo que ninguém acrescentou ao laço, um índice fora do intervalo: os três falham em **silêncio** e deixam chave viva num dispositivo que se anuncia limpo.

O problema é que a verificação óbvia — o firmware varrer a própria memória — é **auto-atestação**: é o mesmo código dizendo que o mesmo código funcionou. Ela vale (e roda a cada boot), mas não fecha a pergunta.

O que fecha é usar **criptografia para provar memória**. A chave mestra se acumula por XOR dos três componentes. Se a zeroização deixasse um único bit para trás, a cerimônia seguinte acumularia sobre o resíduo — e o KCV resultante não seria o dos componentes. Então o teste é: cerimônia, KCV esperado; apagar; **mesma cerimônia, e o KCV tem de voltar a bater**. E o valor esperado não é "o que o dispositivo devolveu da outra vez": é um vetor oficial do NIST.

Nenhum firmware passa nesse teste sem que a memória estivesse mesmo zerada.

Duas notas de projeto, e a segunda é a mais instrutiva:

- **A sobrescrita é em duas passadas**, um padrão e depois zeros. Para memória estática a segunda é a que conta — dizer o contrário seria repetir folclore de disco magnético. A primeira existe porque uma zeroização **interrompida** tem de deixar padrão, não meia chave.
- **O comando exige dois operadores; o gatilho automático não exige ninguém.** Apagar é destrutivo, então uma pessoa não basta. Mas quando é o próprio dispositivo que se descobre comprometido, exigir autorização seria absurdo: bastaria não haver ninguém na sala para a chave sobreviver ao comprometimento. A assimetria não é inconsistência — é o reconhecimento de que as duas situações têm ameaças opostas.

E há um detalhe que a máquina de estados resolve de graça: apagar em `TAMPERED` funciona, e **não tira** o dispositivo de `TAMPERED`. A chave vai embora, que era o pedido; o veredito fica, porque um HSM que se cura de tamper não detectou tamper nenhum.

Os comandos de chave, e por que quase nenhum pede dois dedos

Depois da cerimônia vem o trabalho: gerar uma chave, exportá-la embrulhada, reimportá-la, consultar o que ela é, usá-la. **Nenhum desses comandos exige dual control** — o que contraria a intuição o suficiente para valer o parágrafo.

Dual control é para cerimônia, não para operação. Carregar a chave mestra, ativar o dispositivo e apagar tudo são eventos raros, com gente na frente do equipamento. Gerar e exportar chave é o que um HSM faz o dia inteiro. Exigir dois dedos ali não aumentaria segurança nenhuma — garantiria apenas que ninguém usa o equipamento. E um controle que impede o uso legítimo é desligado no primeiro dia ruim, junto com os que serviam para alguma coisa.

O que protege esses comandos é estrutural, não humano:

- a chave **nunca sai em claro** — sai embrulhada num key block, e quem recebe não sabe abrir;
- o key block é **autenticado sobre cabeçalho mais corpo**, então a política viaja junto com a chave e não pode ser reescrita a caminho;
- `exportabilidade` decide quem pode sair, e existe **um único caminho** no código para os bytes deixarem um slot — é ele que consulta o campo.

A última linha é a diferença entre uma regra e uma promessa. Se houvesse duas formas de obter os bytes, a verificação viraria convenção — e convenção é o que se esquece no caminho raro.

A exceção é **formar uma chave de trabalho por componentes**. É a cerimônia de LMK um nível abaixo: cada custodiante carrega a sua parte, e por isso ela exige os dois botões, como a cerimônia.

O contraste que fecha a Parte II. Um comando que recebe a chave *no pedido* faz material de chave atravessar a fronteira na entrada. Aqui o host escolhe os **metadados** e nunca vê o material. Nenhum comando deste equipamento aceita chave em claro.

E uma decisão que só parece detalhe: **toda recusa de importação devolve o mesmo código**. Bloco malformado, hexadecimal inválido, MAC errado e comprimento impossível são indistinguíveis de fora. É o comando mais exposto do dispositivo — um atacante manda blocos forjados em laço e cada resposta é informação. Dizer em qual etapa a validação parou é o oráculo de padding da seção 15, com outra roupa: o atacante não precisa da chave, precisa que a vítima seja prestativa.

Console e host na mesma porta

Num HSM de verdade o componente entra por teclado local ou cartão do custodiante, **nunca pela mesma porta por onde o host fala**. Aqui o console e o host são protocolos diferentes na mesma UART (seção 28), e o componente passa pela máquina do host. O dual control impede *carregar*; não impede *observar*. É a maior distância entre este projeto e o modelo — e está escrita, não escondida.

27. Fase 4 — armazenamento não-volátil

Um HSM comercial desliga e volta *online* com a chave mestra, as chaves de trabalho e o estado. Este equipamento faz o mesmo: a cerimônia só é necessária na primeira vez e depois de um `ZEROIZE`.

A memória segura, e por que ela é fraca de propósito

Num HSM comercial, a LMK sobrevive ao desligamento numa **memória mantida por bateria, dentro de um invólucro com detecção de violação** (§18.2). Abrir a caixa corta a bateria, e a chave some.

Esta placa não tem nada equivalente. O eFUSE é proibido pela regra nº 1, a BBRAM está sem bateria, o DNA do chip é público e não há invólucro. Embrulhar a LMK antes de gravá-la só mudaria o problema de lugar: a chave que a embrulha teria de ficar guardada em algum lugar, e esse lugar seria a mesma flash.

Então, por ser um projeto **educacional** e diante dessa **limitação de hardware**, a decisão do autor foi:

> *A região da flash SPI a partir de 4 MB é **tratada como memória segura**. A LMK é guardada nela em claro; as chaves de trabalho, embrulhadas sob a LMK como key blocks X9.143.*

O comportamento operacional fica igual ao de um equipamento comercial. A proteção física não fica: quem tiver um gravador SPI lê a LMK. A diferença está escrita em `doc/fase4-notas.md`, em vez de disfarçada por um esquema que pareceria proteger e não protegeria.

O que existe

- **Hardware.** O SPI do NEORV32 fala com a mesma flash que guarda o bitstream. O clock passa pela primitiva `STARTUPE2`, a única forma de a lógica do usuário alcançar o pino de clock de configuração.
- **A guarda do bitstream.** O driver recusa apagar ou gravar abaixo de 4 MB, num lugar só, por onde toda escrita passa.
- **O registro persistente**, com a LMK, o estado e os slots, em **duas cópias**. Cada gravação vai para a mais velha, e uma queda de energia no meio deixa a outra intacta. No boot vale a mais nova cujo CRC confere.
- **Chave anunciada é chave guardada.** Se a flash falhar, `GEN_KEY` e parentes desfazem o slot e devolvem `STORE_FAIL`.
- **O log de auditoria.** Cada comando é registrado **antes** de executar, com o resultado “pendente”; depois só o byte de resultado é programado por cima. Um registro que ficou pendente é um comando que começou e não terminou. E se o log não puder ser gravado, o comando não executa. A exceção é o `ZEROIZE`.

Anti-rollback: fora, de propósito

Um HSM real tem um **contador anti-rollback**. O problema é real: quem restaurar uma imagem antiga da flash ressuscita uma chave apagada. Mas a defesa exige um contador monotônico **fora** da flash, que a placa não tem, e um contador dentro dela seria restaurado junto com o resto. A limitação está documentada, não mitigada. É um dos pontos onde HSMs reais gastam esforço desproporcional, e aqui fica como lição, não como código.

28. Fase 5 — API de host

A porta host usa um **command set ASCII** no estilo dos HSM de pagamento — código de comando de dois caracteres, código de resposta igual ao comando mais um, erros numéricos, campos posicionais — e não de um subconjunto de PKCS#11 como biblioteca compartilhada.

Não é troca de sintaxe. PKCS#11 é uma API de *biblioteca*: objetos, sessões, atributos, estado do lado do cliente. Um command set de HSM de pagamento é um protocolo de *serviço*, sem sessão e sem estado — **cada comando chega sozinho e precisa se bastar**. Isso muda o que o dispositivo tem de verificar, e muda para pior: não há contexto acumulado em que confiar.

É aqui que se entende por que a API é o que é: **é você que precisa impedi-la de vazar material de chave**. Depois de ter lido a seção 15, escrever a API com a pergunta "o que isto vaza em laço?" na cabeça é uma experiência diferente.

Três decisões de arquitetura

Quem interpreta o ASCII é o dispositivo. No computador fica só uma ponte que liga um socket TCP à UART e repassa os bytes sem entendê-los. A outra opção — um programa no PC traduzindo o ASCII para o protocolo binário — seria mais rápida de escrever e ensinaria a coisa errada: a defesa da API ficaria fora da fronteira criptográfica, que é justamente o lugar onde ela não protege nada.

Console e host passam a ter papéis separados. O protocolo binário das fases anteriores vira o *console*: cerimônia, estado, diagnóstico, log. O ASCII é o *host*, e não tem nenhum comando de cerimônia — quem opera não administra. Os dois dividem a mesma UART (o primeiro byte de cada mensagem diz qual é qual), então a separação é de papéis, não física; a distância em relação a um equipamento comercial, que tem portas separadas, continua registrada.

A chave de trabalho viaja no comando. Esta é a mudança de modelo mental da fase. Até aqui o dispositivo guardava as chaves em 16 slots e o host as citava por número. Um HSM de pagamento não funciona assim: ele não guarda as chaves de trabalho de ninguém. O sistema do cliente as mantém no banco de dados dele, cada uma como um key block embrulhado sob a LMK, e manda o bloco junto com cada pedido. O dispositivo abre o bloco por dentro, usa a chave e a esquece **antes de responder**.

É por isso que um equipamento com memória para poucas chaves atende milhões delas, e é por isso que a LMK é a única chave que ele precisa guardar. A frase "sem estado" do parágrafo anterior quer dizer exatamente isto: o estado existe, mas mora do lado do cliente, embrulhado.

Os comandos

Nove comandos simétricos, com códigos deste projeto (da categoria vem só a forma geral): diagnóstico, gerar chave, conferir um key block, exportar e importar sob uma chave de troca, cifrar e decifrar em AES-CBC, gerar e verificar CMAC. A chave de cada comando é aberta num slot **efêmero** do key store — o mesmo código de sempre faz a checagem de modo de uso — e apagada ao fim do comando, em qualquer desfecho. Os comandos de RSA estão na seção 30. O apêndice B tem o formato.

Trocar chave com outra instituição

Uma chave guardada sob a LMK só abre dentro deste dispositivo. Para mandá-la a outra instituição, as duas combinam antes uma **KEK** — uma chave cujo único serviço é proteger outras chaves em trânsito — formada por componentes em cerimônia, com um custodiante de cada lado. Cada instituição guarda a KEK sob a sua própria LMK.

Exportar é re-embrulhar: o dispositivo abre o bloco da chave (sob a LMK), abre o da KEK (também sob a LMK) e devolve a chave sob a KEK. Do outro lado, importar é o caminho inverso. A chave nunca existe em claro fora de um dispositivo, e nenhuma LMK sai de casa.

É aqui que o campo de exportabilidade passa a significar alguma coisa de verdade: antes, "exportar" queria dizer sempre "sob a LMK", e o campo era quase um sim-ou-não. Agora a pergunta é "esta chave pode ir **para lá**?" — e uma chave marcada `N` recebe a resposta não, enquanto continua podendo ser usada aqui à vontade. E o modo de uso da KEK também conta: uma KEK que só embrulha serve para mandar chaves e não para recebê-las.

Na prática, o uso tem cinco passos:

1. **Formar a KEK com os mesmos componentes nos dois dispositivos.** Cada custodiante leva o seu componente aos dois lados. O mesmo valor entra nos dois dispositivos, e ninguém conhece a KEK inteira.
2. **Conferir pelo KCV.** As duas instituições comparam o KCV da KEK, por telefone ou em papel. KCVs iguais provam que as KEKs são iguais, sem que a chave viaje.
3. **Guardar a KEK fora do dispositivo.** Cada host fica com a KEK como key block sob a sua LMK, como qualquer outra chave, e o slot é apagado.
4. **Exportar.** O lado que manda apresenta o bloco da KEK e o da chave e recebe a chave sob a KEK, com o KCV. É isso que vai pela rede.
5. **Importar.** O lado que recebe apresenta o seu bloco da KEK e o que chegou, e recebe a chave sob a sua própria LMK. O KCV tem de bater com o informado.

O cabeçalho atravessa intacto: uso, modo e exportabilidade continuam os que a chave tinha na origem. O roteiro com os comandos está em `doc/fase5-notas.md`.

29. Fase 6 — tamper e ataques

Duas metades.

Defesa: XADC monitorando temperatura e tensão, disparando zeroize fora de faixa. É o mecanismo de proteção contra falhas ambientais do FIPS nível 4 (seção 19), em versão mínima. Implementada, junto com a perda de lock do MMCM. Ver `doc/fase6-notas.md`, que diz também o que ela não protege.

Ataque: atacar o próprio HSM. Glitch de clock num bitstream separado, observar o resultado sair errado, e então implementar contramedidas — dupla execução, verificação de resultado (seção 17.4). O alvo é o AES e o MAC. Tudo o que sai ao host é calculado duas vezes, e discordância vira erro grave.

Também: bitstream encriptado com chave em BBRAM, como confidencialidade de firmware. O mecanismo existe, e a produção segue sem cifra, por decisão explicada na seção 18.5. E o experimento de RO-PUF, documentado à parte, que mede na própria placa **por que** um PUF de FPGA é ruim e por que o corretor de erros é obrigatório.

30. Fase 7 — assimétrico

RSA-2048 e ECDSA sobre a curva P-256. O aprendizado de arquitetura de HSM está nas fases 3 a 5. Mas há um gancho direto com a seção 17.1 — implementar RSA-CRT sem verificar a assinatura antes de emitir é reproduzir o ataque Bellcore no próprio equipamento.

O equipamento tem o RSA-2048 completo: um multiplicador modular no fabric, a exponenciação no firmware, a geração de chave, a assinatura por CRT em tempo constante, a verificação, o KAT no POST e três comandos na porta host — `R0`, `R2` e `R4`. Tem também o ECDSA P-256, sobre o mesmo multiplicador, com os comandos `E0`, `E2` e `E4`. As medidas e o estado de cada peça estão em `doc/fase7-notas.md`.

Por que um HSM precisa de criptografia assimétrica

As funções das seções 25 a 29 são **simétricas**: quem cifra e quem decifra têm a mesma chave. Isso funciona dentro de uma instituição, e entre duas só depois de elas já terem uma chave em comum — foi o que a KEK da seção 28 resolveu, por componentes levados em mãos.

A criptografia assimétrica separa a chave em duas metades. A **privada** fica dentro da fronteira e nunca sai; a **pública** pode ser publicada. O que uma faz, só a outra confere. Disso saem dois serviços que a simétrica não presta:

- **assinatura:** só o dono da privada produz, qualquer um confere. Prova origem sem segredo compartilhado;
- **troca de chave:** duas partes que nunca se encontraram combinam uma chave simétrica por um canal aberto.

Num HSM de pagamento, os dois aparecem na carga remota de chaves: o terminal e o centro de processamento trocam a chave de transporte por certificados e assinaturas, sem custodiantes viajando com envelopes.

RSA em uma página

Escolhem-se dois primos grandes, `p` e `q`, e calcula-se `n = p·q`. O expoente público `e` é um número pequeno e ímpar; o privado `d` é o inverso de `e` módulo `(p−1)(q−1)`. Chave pública: `(n, e)`. Chave privada: `d` — ou, equivalentemente, `p` e `q`, porque com eles se calcula `d`.

```
assinar:    s = m^d mod n          (só quem tem d)
verificar:  s^e mod n == m ?       (qualquer um com n e e)
```

Com números de brinquedo: `p = 61`, `q = 53`, `n = 3233`, `e = 17`, `d = 2753`. A assinatura de `m = 65` é `65^2753 mod 3233 = 588`, e quem verifica calcula `588^17 mod 3233 = 65`. Confere.

A segurança vem de que, conhecendo só `n`, achar `p` e `q` é fatorar — e para `n` de 2048 bits (617 dígitos decimais) não se conhece método viável. Toda a operação, porém, é **uma única conta**: elevar um número de 2048 bits a outro e reduzir módulo um terceiro. O resto desta seção é sobre como fazer essa conta num FPGA pequeno, e fazê-la sem vazar `d`.

Exponenciação: elevar sem calcular a potência

`m^d` com `d` de 2048 bits não se calcula multiplicando `m` por ele mesmo `d` vezes — seriam `2^2048` multiplicações. Lê-se o expoente bit a bit, do mais alto para o mais baixo:

```
acc = 1
para cada bit de d:
    acc = acc · acc mod n          (eleva ao quadrado: sempre)
    se o bit é 1:
        acc = acc · m mod n        (multiplica: só nos bits 1)
```

É o *square-and-multiply*. Para um expoente de 2048 bits são 2048 quadrados e, em média, 1024 multiplicações — cerca de 3000 produtos modulares de 2048 bits. Para verificar, com `e` pequeno, bastam algumas dezenas. É por isso que verificar é barato e assinar é caro.

⚠ **O `se o bit é 1` é o ataque de temporização da seção 16.1.** Um bit 1 custa um produto a mais, e quem mede o tempo lê o expoente. Com `e`, que é público, o desvio é inofensivo e o firmware o usa. Com `d`, a exponenciação tem de fazer **o mesmo trabalho para todo bit** — é o que a escada de Montgomery, que vem com a assinatura, garante.

O problema da divisão, e a solução de Montgomery

Cada produto modular é uma multiplicação seguida de `mod n`, e `mod n` é o resto de uma **divisão** de um número de 4096 bits por um de 2048. Divisão é a operação mais cara que existe em hardware: não se faz em paralelo, e cada dígito do quociente depende do anterior.

Montgomery (1985) trocou a divisão por `n` por uma divisão por **potência de dois**, que em binário é só descartar bits. A ideia é trabalhar com os números numa representação deslocada: em vez de `a`, guarda-se `a·R mod n`, com `R = 2^2048`. O produto de Montgomery de duas representações calcula

```
MontProd(x, y) = x · y · R⁻¹ mod n
```

e, se `x = a·R` e `y = b·R`, o resultado é `a·b·R` — a representação de `a·b`. O fator `R` extra se cancela a cada produto, e o resultado já está pronto para o próximo.

O truque está em como se divide por `R` sem errar. O produto `T = x·y` não é múltiplo de `R`, mas pode-se somar a ele um múltiplo de `n` que torna os bits baixos zero — e somar múltiplo de `n` não muda o valor módulo `n`. O múltiplo certo é `m = T · n' mod R`, com `n' = −n⁻¹ mod R`, calculado uma vez por chave:

```
T = x · y
m = (T mod R) · n'  mod R        (só os bits baixos interessam)
u = (T + m · n) / R              (divisão exata: descarta bits)
se u ≥ n:  u = u − n             (a "subtração final")
```

Com números pequenos: `n = 97`, `R = 128`, `a = 20`, `b = 30`. As representações são `20·128 mod 97 = 38` e `30·128 mod 97 = 57`; `T = 38·57 = 2166`; `n' = 95`; `m = 2166·95 mod 128 = 74`; `u = (2166 + 74·97) / 128 = 9344 / 128 = 73`. E de fato `20·30·128 mod 97 = 73`: a representação de `a·b`, sem nenhuma divisão por 97.

Entrar na representação custa um produto por `R² mod n`, e sair, um produto por `1`. Numa exponenciação de milhares de produtos, as duas conversões são irrelevantes.

Limbs, DSPs e o FIOS

Um número de 2048 bits não cabe em multiplicador nenhum. Ele é cortado em **limbs** — 128 pedaços de 16 bits — e o produto vira uma multiplicação de escola primária, limb contra limb, como se faz à mão com algarismos. Cada multiplicação de 16 × 16 bits cabe num **DSP48E1**, o bloco multiplicador dedicado do Artix-7.

A forma ingênua calcula `x·y` inteiro (4096 bits) e só depois reduz. O FIOS (*Finely Integrated Operand Scanning*, Koç, Acar e Kaliski, 1996) intercala as duas coisas: a cada limb de `x`, multiplica por `y`, soma o múltiplo de `n` que zera o limb mais baixo e descarta esse limb. O acumulador nunca passa de 2048 bits mais um pouco, e a redução não custa uma segunda passada. Dois multiplicadores trabalham juntos — um para `x·y`, outro para `m·n` —, e um produto de 2048 bits leva cerca de 128^2 ciclos: 18 433, ou 184 µs a 100 MHz.

O core usa **4** dos 90 DSP48E1. Paralelizar mais encurtaria o produto, mas o gargalo de um HSM didático não é a vazão de assinaturas — e cada DSP a mais é área e rota no fabric apertado do XC7A35T.

Hardware faz o produto, firmware faz a política

O coprocessador faz **uma** coisa: $\texttt{dst} = a \cdot b \cdot R^{-1} \bmod n$, sobre oito registradores internos de 2048 bits numa Block RAM. Tudo o mais é firmware: calcular `n'` e $R^2 \bmod n$, percorrer o expoente, montar e conferir o enchimento. É a mesma divisão do TRNG na seção 25: o que custa ciclo fica no hardware, e o que é decisão fica em C, onde se lê e se testa.

Três propriedades do core são de segurança, não de desempenho:

- **Tempo constante.** O algoritmo de livro faz a subtração final só quando $u \geq n$ — e esse desvio vaza informação sobre os operandos, o que já bastou para recuperar chaves RSA (Walter e Thompson, 2001; Schindler, 2000). Aqui a subtração **sempre** acontece, e um multiplexador escolhe entre `u` e `u − n` na hora de gravar. Todo produto do mesmo tamanho leva o mesmo número de ciclos, e o testbench cobra isso.
- **Módulo só de escrita.** Na verificação, `n` é público. Na assinatura com CRT, o módulo de cada metade é `p` ou `q` — a chave privada. O registrador do módulo não tem caminho de leitura pelo barramento.
- **Zeroização.** O `WIPE` global do coprocessador apaga as três memórias do core, interrompendo qualquer produto em curso.

O tamanho do produto é configurável, de 16 a 2048 bits, para que o mesmo core sirva às metades de 1024 bits do CRT.

PKCS#1 v1.5: o que se assina não é o hash

RSA puro é perigosamente maleável: se s_1 assina m_1 e s_2 assina m_2, então $s_1 \cdot s_2$ assina $m_1 \cdot m_2$. E um hash de 256 bits, elevado a `e`, é um número pequeno demais perto de `n`. Por isso o que se assina é o hash **embrulhado** num bloco do tamanho de `n` (RFC 8017, seção 9.2):

```
    EM = 00 01 FF FF ... FF 00 || DigestInfo(SHA-256) || hash
```

O `DigestInfo` diz qual hash foi usado — sem ele, uma assinatura sobre um hash fraco poderia passar por assinatura sobre um forte.

⚠ **A verificação reconstrói o `EM` inteiro e compara, em vez de interpretá-lo.** Bleichenbacher (2006) mostrou que verificadores que *liam* o bloco — achavam o `00`, pulavam os `FF`, extraíam o hash — aceitavam lixo depois do hash. Com `e = 3`, isso deixava forjar assinaturas com uma raiz cúbica, sem chave nenhuma. Quem monta o bloco esperado e compara byte a byte não tem parser para enganar. A comparação acumula diferenças, como a de MAC da seção 16.1.

O `R4`, o primeiro comando sem chave nenhuma

`R4` recebe `n`, `e`, o hash e a assinatura, e devolve **um bit**: `00` confere, `01` não confere. Só dados públicos entram, e nada do dispositivo sai. Fica em `OPERATIONAL` e entra no log, como os demais.

Parece estranho um HSM verificar algo que qualquer computador verifica. A razão é a mesma da fronteira inteira: quando a decisão de aceitar uma chave importada depende de uma assinatura conferir — que é o que acontece na carga remota —, a verificação tem de ser feita **dentro** da fronteira, ou um host comprometido diria "conferiu" sobre qualquer coisa.

Como todo resultado que sai ao host, a verificação roda duas vezes, e discordância responde `91` (seção 29). Um pedido `R4` tem mais de 1100 caracteres, e o limite de mensagem é de 2048 bytes.

O preço no boot

O POST tem o KAT de RSA: verificar uma assinatura oficial do CAVP e **recusar** a mesma assinatura com um bit trocado. O vetor tem `e = 0x260445`, de 22 bits, e cada verificação custa cerca de 9 ms. É a regra da seção 11: cada propriedade que o dispositivo garante sobre si mesmo cobra milissegundos de todo boot, para sempre.

Assinar: o Teorema Chinês do Resto

Assinar é `s = m^d mod n`, com `d` de 2048 bits: cerca de 3000 produtos de 2048 bits. Quem tem a chave privada, porém, conhece `p` e `q` — e pode fazer a conta **duas vezes pela metade**:

```
sp = m^dp mod p        dp = d mod (p−1)
sq = m^dq mod q        dq = d mod (q−1)
h  = qinv · (sp − sq) mod p      qinv = q⁻¹ mod p
s  = sq + h · q
```

O Teorema Chinês do Resto garante que só existe um número menor que `n` com esses dois restos, e a última linha (a recombinação de Garner) o reconstrói. Cada metade tem expoente e módulo de 1024 bits, e um produto de 1024 bits custa um quarto do de 2048. Duas exponenciações de meio tamanho custam cerca de **um quarto** de uma inteira.

Com os números de brinquedo da seção anterior (`p = 61`, `q = 53`, `d = 2753`, `m = 65`): `dp = 53`, `dq = 49`, `qinv = 38`; `sp = 39`, `sq = 5`, `h = 11` e `s = 5 + 11·53 = 588` — a mesma assinatura de antes.

Por isso a chave privada deste dispositivo **não guarda `d`**: guarda `p, q, dp, dq, qinv` e o `e`. E por isso o ataque Bellcore (seção 17.1) é assunto desta fase: se uma das metades sair errada, `s` fica certo módulo um primo e errado módulo o outro, e `mdc(s^e − m, n)` entrega a fatoração. A contramedida: **toda assinatura é verificada com `e` antes de sair**. Se não conferir, nada sai, o comando responde `91` e o erro grave fica marcado até o próximo boot, como a dupla execução da Fase 6.

A escada de Montgomery

O *square-and-multiply* da seção anterior faz um produto a mais em cada bit 1, e com `dp` e `dq` isso é o ataque de temporização. A **escada de Montgomery** (o nome é do mesmo Montgomery, mas é outra ideia) mantém dois valores, `X` e `Y`, sempre com `Y = X · m`, e faz **exatamente um produto e um quadrado por bit**:

```
bit 0:   Y = X · Y     X = X²
bit 1:   X = X · Y     Y = Y²
```

O bit decide só **quais registradores** entram em cada operação — e o coprocessador leva o mesmo número de ciclos qualquer que seja o registrador. No firmware, a escolha é feita por máscara, sem `if`. O expoente é percorrido inteiro, zeros à esquerda inclusive, então o tempo depende só do tamanho dele.

Tudo no coprocessador, inclusive o que não é modular

As contas da assinatura que não são exponenciação também vão para o hardware:

- **reduzir `m` módulo `p`**: `m` tem 2048 bits e `p`, 1024. Partindo `m = hi·R + lo`, a representação de Montgomery de `m mod p` é `Mont(hi, R³) + Mont(lo, R²)` — dois produtos e uma soma, sem divisão;
- **`h · q` e `n = p · q`**, que são produtos comuns de inteiros, usam o módulo `M = 2²⁰⁴⁸ − 1`. Como `R = 2²⁰⁴⁸` é congruente a 1 módulo `M`, o produto de Montgomery `a·b·R⁻¹ mod M` é o próprio `a·b` — e, sendo `a·b < M`, é o inteiro. O mesmo core que faz a exponenciação faz a multiplicação comum.

Em C sobram somas, subtrações, comparações e o enchimento PKCS#1.

Gerar a chave: procurar primos

Gerar um par RSA é achar dois primos de 1024 bits ao acaso. Não há fórmula para isso: sorteia-se um número e testa-se. Perto de `2¹⁰²⁴`, um número ímpar ao acaso é primo com probabilidade de cerca de 1 em 355, então a busca leva centenas de sorteios.

O dispositivo segue a FIPS 186-5, Apêndice A.1.3 (*primos prováveis*):

1. sorteia 1024 bits do DRBG, liga os dois bits altos (o que garante `p ≥ √2 · 2¹⁰²³`, e portanto `n` com 2048 bits exatos) e o bit baixo;
2. descarta o candidato se ele for divisível por algum ímpar até 1023 — conta barata, em C, que elimina mais de 80% dos candidatos antes de qualquer exponenciação;
3. exige `mdc(p − 1, e) = 1`, senão `e` não teria inverso;
4. aplica o **teste de Miller-Rabin** com 5 bases sorteadas;
5. para `q`, exige ainda `|p − q| > 2⁹²⁴`: primos próximos demais tornam `n` fatorável pelo método de Fermat.

O **Miller-Rabin** parte do pequeno teorema de Fermat: se `w` é primo, então `b^(w−1) ≡ 1 (mod w)`. Escrevendo `w − 1 = 2^a · m`, calcula-se `b^m` e eleva-se ao quadrado até `a` vezes. Num primo, a sequência termina em 1, e o valor antes do primeiro 1 é sempre `w − 1`. Um composto quase sempre

quebra essa regra. Exemplo clássico: `561 = 3·11·17` engana o teste de Fermat com qualquer base, mas não o de Miller-Rabin. Com `b = 2` e `560 = 2⁴ · 35`, a sequência é `263, 166, 67, 1` — chega a 1 sem passar por 560, e 561 é declarado composto.

Cada rodada deixa passar um composto com probabilidade de no máximo 1/4 (na prática, muito menos). A Tabela B.1 da FIPS 186-5 fixa **5 rodadas** para primos de 1024 bits, para um erro de 2^{-112}, que é a força de segurança do RSA-2048. Cada rodada é uma exponenciação de 1024 bits na escada — no coprocessador.

Achados `p` e `q`, o resto vem deles:

- `dp = e⁻¹ mod (p − 1)`, calculado sem dividir número grande por número grande: com `k = −(p − 1)⁻¹ mod e`, o número `1 + k(p − 1)` é múltiplo de `e`, e o quociente é `dp`. Como `e = 65537` é pequeno, tudo isso é conta de 32 bits;
- `qinv = q^(p−2) mod p`, pelo pequeno teorema de Fermat — uma exponenciação, na escada, porque `p` é segredo;
- `n = p · q`, no coprocessador.

Antes de sair, o par novo passa por um **teste de consistência**: o dispositivo assina com ele e verifica a assinatura. É exigência da FIPS 140-3 para geração de chave assimétrica, e pega na hora um primo errado ou um `qinv` mal calculado.

A chave privada no key block

A chave sai como **key block X9.143 sob a LMK** e fica com o host, como as chaves AES da porta host (seção 28). O cabeçalho usa os códigos da norma: uso `S0` (par assimétrico de assinatura), algoritmo `R` (RSA), modo `S` (só assinar). As duas implementações do formato — em C e em Python — recusam qualquer outra combinação para `R`.

⚠ **O conteúdo do bloco é um layout deste projeto**, e não da norma: `e`, `p`, `q`, `dp`, `dq` e `qinv` em campos fixos, 644 bytes. A X9.143 define os códigos de chave assimétrica, mas a forma de codificar a chave dentro do bloco não estava acessível — a norma é paga. Se um dia estiver, só esse campo muda. O módulo `n` não vai no bloco: o dispositivo o recalcula como `p · q`.

O bloco tem 1360 caracteres, e a chave aberta vive numa área própria do key store **só durante o comando** — nunca num slot, nunca na flash. O despachante a apaga no fim de todo comando, como faz com o slot efêmero.

Os comandos

Pedido	Resposta	Entrada	Saída
`R0` gerar par	`R1`	exportabilidade(1)	key block · n(512) · e(8)
`R2` assinar	`R3`	key block · digest(64)	assinatura(512)
`R4` verificar	`R5`	n(512) · e(8) · digest(64) · assinatura(512)	—

`R2` com um key block AES responde `11`: é a confusão de tipo de chave (seção 15.1), recusada como nas chaves simétricas. Um bloco adulterado responde `10`, o mesmo código de qualquer outro bloco inválido.

Curvas elípticas: outro problema difícil

O RSA apoia-se na fatoração. As curvas elípticas apoiam-se em outro problema, e ele permite chaves muito menores: uma chave de 256 bits numa curva tem a força de um RSA de 3072 bits.

Uma curva elíptica, para este uso, é o conjunto dos pares `(x, y)` que satisfazem uma equação como

```
    y² = x³ + a·x + b        (contas módulo um primo p)
```

mais um ponto especial, o **ponto no infinito**, que faz o papel do zero. Os números `x` e `y` não são reais: são inteiros de `0` a `p − 1`, e toda conta termina com `mod p`. O desenho da curva como uma linha suave serve para entender a regra. O que o equipamento calcula é uma nuvem de pontos isolados.

O nome engana. Uma curva elíptica não é uma elipse: o nome vem das integrais que medem o comprimento de arco da elipse, onde essas equações apareceram primeiro.

Somar pontos

Os pontos da curva podem ser **somados**, e a soma de dois pontos da curva é outro ponto da curva. A regra é geométrica:

1. Trace a reta que passa por `P` e por `Q`.
2. A reta corta a curva em um terceiro ponto.
3. Espelhe esse ponto no eixo `x`. O resultado é `P + Q`.

Para somar um ponto a ele mesmo (**dobrar**), a reta é a tangente à curva naquele ponto. Escrita em fórmulas, a regra só usa somas, produtos e **uma divisão** módulo `p`:

```
    somar P + Q:      λ = (yQ − yP) / (xQ − xP)
    dobrar P:         λ = (3·xP² + a) / (2·yP)

    x = λ² − xP − xQ                (na dobra, xQ = xP)
    y = λ·(xP − x) − yP
```

O `λ` é a inclinação da reta. Dividir módulo `p` é multiplicar pelo inverso: `1/14 mod 17 = 11`, porque `14 · 11 = 154 = 9·17 + 1`.

Dois casos ficam fora das fórmulas:

- O **oposto** de `P = (x, y)` é `−P = (x, −y)`. A reta por `P` e `−P` é vertical e não corta a curva de novo. Por definição, `P + (−P)` é o ponto no infinito.
- O ponto no infinito é o zero: `P + 0 = P`.

Com essas regras, os pontos formam um **grupo**: a soma é associativa, tem zero e todo ponto tem oposto. A ordem das parcelas não importa.

Uma curva de brinquedo

A curva $y^2 = x^3 - 3x + 3$ módulo 17 tem 22 pontos, mais o ponto no infinito: 23 elementos. Parta de $G = (1, 1)$ e some G repetidas vezes:

```
 1G = ( 1,  1)     7G = ( 4, 15)    13G = ( 8,  7)    19G = ( 3,  2)
 2G = (15, 16)     8G = (13, 11)    14G = ( 7,  6)    20G = ( 9, 12)
 3G = ( 9,  5)     9G = ( 7, 11)    15G = (13,  6)    21G = (15,  1)
 4G = ( 3, 15)    10G = ( 8, 10)    16G = ( 4,  2)    22G = ( 1, 16)
 5G = (11, 14)    11G = (10,  2)    17G = (14,  6)    23G = infinito
 6G = (14, 11)    12G = (10, 15)    18G = (11,  3)
```

Uma soma, passo a passo: $3G = 2G + G$, com $2G = (15, 16)$ e $G = (1, 1)$.

```
λ = (1 − 16) / (1 − 15) = 15 / 14 = 15 · 11 mod 17 = 12
x = 12² − 15 − 1 = 128 mod 17 = 9
y = 12 · (15 − 9) − 16 = 56 mod 17 = 5            3G = (9, 5)
```

E uma dobra: $4G = 2 \cdot (2G)$.

```
λ = (3 · 15² − 3) / (2 · 16) = 9 / 15 = 9 · 8 mod 17 = 4
x = 4² − 15 − 15 = −14 mod 17 = 3
y = 4 · (15 − 3) − 16 = 32 mod 17 = 15            4G = (3, 15)
```

Três coisas aparecem na tabela:

- **A sequência fecha.** $23G$ é o ponto no infinito, e $24G$ volta a G. O número 23 é a **ordem** de G. Os múltiplos de G só dependem do multiplicador módulo 23. É por isso que as contas de ECDSA com escalares são feitas módulo a ordem, e não módulo p.
- **A tabela é simétrica.** $22G = (1, 16)$ é o oposto de $G = (1, 1)$, porque $16 = -1 \bmod 17$. Em geral, $(23 - k)\cdot G = -(k\cdot G)$.
- **Não há padrão.** $5G$ não fica perto de $4G$. Dado o ponto $(8, 7)$, a única forma de achar o 13 é procurar.

O problema difícil: o logaritmo discreto

Calcular $k \cdot P$ é fácil. Lê-se k bit a bit, dobra-se sempre e soma-se conforme o bit, como na exponenciação. É a **multiplicação de ponto**, a operação cara da curva. Para um k de 256 bits são 256 dobras e cerca de 128 somas.

O inverso é o problema difícil: dados P e $Q = k \cdot P$, achar k. É o **logaritmo discreto** na curva. Na curva de brinquedo, 22 tentativas resolvem. Numa curva com cerca de 2^{256} pontos, o melhor método conhecido (o rô de Pollard) custa cerca de 2^{128} operações. É a raiz quadrada do número de pontos, e não há atalho conhecido.

A correspondência com o RSA é direta. Só muda a notação:

No RSA	Na curva
multiplicar dois números módulo `n`	somar dois pontos
elevar ao quadrado	dobrar
`m^d mod n` (exponenciação)	`k · P` (multiplicação de ponto)
square-and-multiply	dobrar e somar
achar o expoente: logaritmo discreto	achar o escalar: logaritmo discreto

Por que a chave da curva é menor

A diferença está no melhor ataque conhecido contra cada problema.

Contra a fatoração existe o **crivo do corpo de números**, um método subexponencial. Ele explora a estrutura dos inteiros: números pequenos, fatores pequenos, relações entre eles. Cada bit a mais no módulo dá pouco trabalho a mais ao atacante. Para compensar, o módulo tem de crescer muito.

Os pontos de uma curva bem escolhida não têm essa estrutura. Não existe "ponto pequeno" nem "ponto primo". Só sobram os ataques genéricos, que servem para qualquer grupo e custam a raiz quadrada da ordem. Cada dois bits a mais na chave dobram o trabalho do atacante.

A SP 800-57 do NIST dá a equivalência:

Força de segurança	Simétrica	RSA (módulo)	Curva (ordem)
112 bits	3DES de três chaves	2048	224
128 bits	AES-128	3072	256
192 bits	AES-192	7680	384
256 bits	AES-256	15360	512

A P-256 é mais forte que o RSA-2048 deste equipamento, com uma chave oito vezes menor. E a distância cresce: dobrar a força da curva custa dobrar a chave, e dobrar a força do RSA custa multiplicar o módulo por cinco.

⚠ **Nenhum dos dois resiste a um computador quântico grande.** O algoritmo de Shor resolve a fatoração e o logaritmo discreto. A chave menor da curva não é defesa contra isso.

A curva P-256

Uma curva é definida por um conjunto de números públicos, os **parâmetros de domínio**. Todos os usuários da curva usam os mesmos. A **P-256** é uma das curvas do NIST (FIPS 186, SP 800-186). Outras normas dão a ela os nomes `secp256r1` e `prime256v1`. Os parâmetros:

- **O primo `p`**, de 256 bits, que define o corpo das coordenadas: $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$. A forma não é acaso. Um primo feito de poucas potências de dois permite reduzir módulo `p` só com somas e subtrações de palavras de 32 bits, o que acelera o software. Este equipamento não usa esse atalho: o produto de Montgomery do coprocessador serve para qualquer módulo ímpar, e já estava pronto.

- **Os coeficientes `a` e `b`.** `a = −3`, um valor que economiza produtos na fórmula da dobra. `b` é um número de 256 bits que saiu de um hash (SHA-1) de uma semente publicada. A ideia é mostrar que ninguém escolheu `b` para criar uma curva fraca. A norma não explica de onde veio a semente, e essa é a crítica conhecida às curvas do NIST. Não se conhece fraqueza na P-256.
- **O ponto base `G`**, de onde saem todas as chaves públicas.
- **A ordem `n`** de `G`: o menor número com `n · G` igual ao ponto no infinito. É o `23` da curva de brinquedo. Na P-256, `n` é um primo de 256 bits, um pouco menor que `p`.
- **O cofator `h = 1`.** O número de pontos da curva é `h · n`. Com `h = 1`, todo ponto da curva, menos o infinito, é múltiplo de `G` e tem ordem `n`. Curvas com cofator maior têm pontos de ordem pequena, e quem os aceita sem conferir vaza bits da chave. Na P-256, conferir que o ponto está na curva basta.

O número de pontos fica sempre perto de `p`: o teorema de Hasse garante a distância máxima de `2·√p`. Por isso `n` e `p` têm o mesmo tamanho, e por isso são dois números diferentes, com dois papéis diferentes:

- **módulo `p`** fazem-se as contas com **coordenadas** (somar e dobrar pontos);
- **módulo `n`** fazem-se as contas com **escalares** (a chave privada, o `k`, o `s` da assinatura).

Trocar um pelo outro é o erro clássico de quem implementa. O firmware carrega um ou outro no coprocessador conforme a conta.

A chave:

```
chave privada:  d, um número de 1 a n − 1, sorteado
chave pública:  Q = d · G, um ponto da curva
```

Gerar a chave custa **uma** multiplicação de ponto. Não há busca de primos: qualquer número de `1` a `n − 1` é uma chave privada válida. O `d` sai do DRBG por rejeição: sorteiam-se 256 bits, e um valor fora da faixa é descartado e sorteado de novo. Reduzir o valor com `mod n` seria mais simples, mas favoreceria os números baixos.

ECDSA: a assinatura

O ECDSA (*Elliptic Curve Digital Signature Algorithm*, FIPS 186-5) é a versão em curva do DSA. Para assinar um hash `e`, com a chave privada `d`:

```
sorteia k, de 1 a n − 1              (o número secreto DESTA assinatura)
R = k · G
r = (coordenada x de R) mod n
s = k⁻¹ · (e + r · d) mod n
a assinatura é o par (r, s)
```

Para verificar, com a chave pública `Q`:

```
u1 = e · s⁻¹ mod n
u2 = r · s⁻¹ mod n
R' = u1 · G + u2 · Q
confere se (coordenada x de R') mod n == r
```

Funciona porque `Q = d · G`, e então `u1 · G + u2 · Q = (e + r · d) · s⁻¹ · G = k · G`. Quem verifica refaz o ponto `R` sem conhecer `k` nem `d`.

O papel de cada peça:

- `r` compromete o assinante com o ponto `R`, sem revelar `k`. Tirar `k` de `R` é o logaritmo discreto.
- `s` amarra o hash, o `r` e a chave privada numa equação só. Sem `d`, ninguém acha um `s` que feche a conta.
- `k⁻¹` é um inverso módulo `n`. Como `n` é primo, o pequeno teorema de Fermat dá o inverso com uma exponenciação: `k⁻¹ = k^(n−2) mod n`. Ela roda na mesma escada do RSA, porque `k` é segredo.
- **O hash** tem 256 bits, o mesmo tamanho de `n`, e entra inteiro. Com um hash mais longo que `n`, a norma manda usar só os bits da esquerda.

Com a curva de brinquedo (`n = 23`): chave privada `d = 7`, chave pública `Q = 7G = (4, 15)`, hash `e = 5` e `k = 10`.

```
assinar:    R  = 10G = (8, 10)           r = 8
            k⁻¹ = 7                      (10 · 7 = 70 = 3·23 + 1)
            s  = 7 · (5 + 8·7) mod 23 = 13
            assinatura: (8, 13)

verificar:  s⁻¹ = 16                     (13 · 16 = 208 = 9·23 + 1)
            u1 = 5 · 16 mod 23 = 11      u2 = 8 · 16 mod 23 = 13
            11G = (10, 2)                13Q = (1, 16)
            R' = (10, 2) + (1, 16) = (8, 10)
            x de R' = 8 = r              confere
```

⚠ **O `k` é tão secreto quanto a chave, e não pode se repetir.** Duas assinaturas com o mesmo `k` têm o mesmo `r`, e delas sai `d` com uma subtração e uma divisão:

```
k = (e1 − e2) / (s1 − s2)        d = (s1 · k − e1) / r
```

Na curva de brinquedo: assine o hash `12` com o mesmo `k = 10`. A assinatura é `(8, 16)`, com o mesmo `r = 8`. Quem vê as duas calcula `k = (5 − 12) / (13 − 16) = 10` e `d = (13 · 10 − 5) / 8 = 7`, tudo módulo 23. A chave privada saiu de duas assinaturas e quatro contas.

Foi assim que a chave de assinatura de um console de videogame foi recuperada em 2010: o `k` era constante. E não precisa repetir: conhecer alguns bits de `k` em várias assinaturas já basta (ataques de reticulado). Por isso o `k` deste equipamento sai do DRBG a cada assinatura, por rejeição, e a multiplicação `k · G` tem tempo constante.

Existe uma variante que calcula `k` a partir da chave e do hash, sem sorteio (ECDSA determinístico, RFC 6979). A FIPS 186-5 aceita as duas. Este equipamento usa o sorteio: ele tem um gerador aprovado dentro da fronteira.

⚠ **A assinatura não é determinística.** Assinar o mesmo hash duas vezes dá duas assinaturas diferentes, as duas válidas. Um teste não pode comparar a assinatura com um valor esperado, como no RSA. Ele a confere pela verificação. O autoteste do equipamento usa um `k` fixo do vetor oficial, por uma entrada que nenhum comando alcança.

⚠ **Para cada assinatura válida existe outra.** Se `(r, s)` confere, `(r, n − s)` também confere, sobre o mesmo hash. Quem usa a assinatura como identificador único de uma mensagem tem de saber disso. O equipamento aceita as duas formas, como a norma manda.

RSA e ECDSA, lado a lado

	RSA-2048	ECDSA P-256
Problema difícil	fatorar `n = p · q`	logaritmo discreto na curva
Força de segurança	112 bits	128 bits
Chave privada	dois primos de 1024 bits	um número de 256 bits
Chave pública	`n` e `e`: 256 bytes e mais um pouco	um ponto: 64 bytes
Assinatura	256 bytes	64 bytes
Gerar a chave	procurar dois primos: lento, tempo variável	uma multiplicação de ponto
Assinar	caro (duas exponenciações de 1024 bits)	uma multiplicação de ponto
Verificar	barato (`e` pequeno)	duas multiplicações de ponto: mais caro que assinar
Sorteio na assinatura	nenhum (PKCS#1 v1.5)	um `k` novo por assinatura
Mesma mensagem, duas vezes	mesma assinatura	assinaturas diferentes
Parâmetros	cada chave tem o seu módulo	todos usam a mesma curva
Erro que entrega a chave	falha numa metade do CRT (Bellcore)	`k` repetido, previsível ou vazado
Enchimento	obrigatório, e fonte de ataques	não existe

Duas linhas merecem atenção num HSM:

- **A qualidade do sorteio.** No RSA, o gerador aleatório só importa quando a chave nasce. No ECDSA, ele importa em **toda** assinatura. Um gerador que falha depois de a chave existir não compromete a chave RSA, e compromete a chave ECDSA na primeira assinatura. Os testes contínuos da fonte de entropia (seção 25) pesam mais aqui.
- **O custo de verificar.** Um terminal que só verifica assinaturas trabalha menos com RSA. Um equipamento que assina muito trabalha menos com ECDSA.

O que os dois têm em comum, neste equipamento: a operação privada roda em tempo constante, a assinatura é verificada antes de sair, e a chave privada só existe fora da fronteira como key block sob a LMK.

A multiplicação de ponto, sem vazar o escalar

O método de dobrar e somar tem o defeito do *square-and-multiply*: um bit 1 custa uma soma a mais. Com `k` ou `d` no escalar, o tempo entrega o segredo.

O equipamento usa a **escada de Montgomery** também aqui. Ela mantém dois pontos, `R0` e `R1`, sempre com `R1 = R0 + P`, e a cada bit faz uma soma e uma dobra, quaisquer que sejam os bits. O bit decide só qual dos dois pontos recebe cada resultado.

Há um custo escondido na soma de pontos: a **divisão** módulo `p`. Dividir é calcular um inverso, que custa centenas de produtos. A saída é a de sempre: trocar a representação. Em vez de `(x, y)`, guarda-se `(X, Y, Z)`, com $x = X/Z^2$ e $y = Y/Z^3$. Nessa forma, somar e dobrar não dividem, e um único inverso no fim devolve `(x, y)`. A escada deste equipamento usa uma variante em que os dois pontos têm sempre o mesmo `Z`, e ele nem precisa ser guardado.

Cada bit do escalar custa 14 produtos modulares de 256 bits, e uma multiplicação de ponto, cerca de 4 000. Todos vão para o mesmo coprocessador do RSA, com o tamanho ajustado para 256 bits. As somas e subtrações ficam em C, escritas sem desvio pelo valor.

⚠ **As fórmulas têm casos de exceção, e quem ataca os procura.** A soma de dois pontos iguais é uma dobra, com outra fórmula. A soma de um ponto com o seu oposto é o ponto no infinito, que `(x, y)` não representa. Na assinatura, esses casos têm chance desprezível. Na verificação, quem monta a assinatura escolhe `u1` e `u2`, e consegue produzi-los. O firmware trata cada um deles de forma explícita.

⚠ **A chave pública que chega é conferida.** O `E4` recebe `Q` do host, e um `Q` fora da curva leva as contas para outra curva, mais fraca. Antes de qualquer conta, o firmware confere que as coordenadas são menores que `p` e que o ponto satisfaz a equação.

A chave de curva elíptica no key block, e os comandos

A chave privada sai como key block X9.143 sob a LMK, como a do RSA. O cabeçalho usa uso `S0`, algoritmo `E` (curva elíptica) e modo `S`. O conteúdo é um layout deste projeto: `d`, `Qx` e `Qy`, 32 bytes cada. A chave pública vai junto, para o dispositivo verificar a própria assinatura antes de entregá-la.

Pedido	Resposta	Entrada	Saída
`E0` gerar par	`E1`	exportabilidade(1)	key block · Qx(64) · Qy(64)
`E2` assinar	`E3`	key block · digest(64)	r(64) · s(64)
`E4` verificar	`E5`	Qx(64) · Qy(64) · digest(64) · r(64) · s(64)	—

As regras são as do RSA. A assinatura é verificada antes de sair. O `E4` verifica duas vezes e devolve um bit. Um key block RSA no `E2`, ou um de ECDSA no `R2`, responde `11`: é confusão de tipo de chave.

O que o equipamento não faz

- **Acordo de chave (ECDH).** Sobre a curva, o equipamento só assina e verifica.
- **Cifrar com RSA.** A chave RSA só assina.

31. Fase 8 — criptografia de pagamento

O modelo de referência do projeto é a categoria dos HSMs de pagamento. Esta fase é a última porque depende de tudo o que vem antes: sem hierarquia de chaves (Fase 3) e sem API de serviço (Fase 5), não há onde encaixar um comando de tradução de PIN.

O conteúdo dela é a **Parte VII** deste manual. O que decide se um item é implementável aqui não é a dificuldade — é o **hardware** e a **procedência dos vetores**:

- o que é AES cabe (PIN block formato 4, DUKPT AES);
- o que é inerentemente 3DES não cabe, porque o coprocessador é AES-256 sem caminho de downgrade, e isso é decisão de arquitetura e não tarefa pendente;
- o que não tem vetor público não entra, ou entra marcado como não verificado — a regra nº 5 exige procedência, e "implementei e parece certo" é exatamente o que ela proíbe.

O melhor item da lista é o **ataque de tabela de decimalização** (seção 35.3), e por um motivo que vale registrar: ele não precisa de 3DES nem de vetor de bandeira nenhuma. O ataque é sobre a **API**, não sobre a cifra, e reproduz inteiro num esquema nosso com AES.

32. Ordem de trabalho, e a regra que a sustenta

Uma regra vale para todas as fases:

> ***Nada vai para a placa sem passar antes no testbench.***

Depurar criptografia por UART, no hardware, sem visibilidade interna, é a forma mais lenta possível de descobrir que faltou um byte no preenchimento. Um erro de enquadramento em simulação custa segundos; o mesmo erro na bancada custa uma tarde e várias hipóteses erradas.

Parte VII — Criptografia de pagamento

Esta parte existe porque o modelo de referência deste projeto passou a ser um **HSM de pagamento comercial**. Entender o que esses equipamentos fazem o dia inteiro é o que separa "sei o que é uma fronteira criptográfica" de "sei por que aquele comando existe".

Aviso que vale para o capítulo inteiro: **este projeto não implementa nada do que está aqui.** A seção 37 explica o que dele é implementável e o que não é, e por quê. O material abaixo é para entender o domínio, não para copiar em código.

> ***Independência e fontes.*** *Este capítulo* ***não nomeia fabricante nem produto****, de propósito. O modelo de estudo é a categoria — HSM de pagamento — e nada aqui alega vínculo, endosso ou compatibilidade com equipamento comercial algum.*
>
> *Todo o conteúdo vem de* ***normas públicas*** *(ISO 9564, ANSI X9.24 e X9.143, NIST SP 800-38B) e de* ***literatura acadêmica publicada****. Nada é derivado de documentação proprietária: tabela de comandos, código de erro e esquema de chave de manual de fabricante são material licenciado e* ***não entram aqui****.*

> ***Sobre os layouts de bytes.*** *Descrevo a estrutura e o porquê de cada construção, não o mapa exato de nibbles. Isso é deliberado: os layouts normativos estão na ISO 9564, na ANSI X9.24 e nas especificações das bandeiras, e a regra deste projeto proíbe escrever valor normativo de memória. Quem for implementar tem de ler a norma — e essa é a lição, não um contratempo.*

---

33. PIN blocks — por que um PIN não viaja sozinho

Um PIN de quatro dígitos tem 10.000 valores possíveis. Cifrar isso diretamente com uma chave, sozinho, é um desastre: o mesmo PIN produz sempre o mesmo criptograma, e um dicionário de 10.000 entradas quebra tudo em tempo nenhum.

A solução é o **PIN block**: um bloco de tamanho fixo que combina o PIN com o **número da conta** (PAN) antes de cifrar. Duas consequências, e as duas importam:

1. O criptograma de um mesmo PIN é **diferente para cada conta**. O dicionário morre.
2. O PIN block fica **amarrado à conta**. Um PIN block capturado numa transação não serve para outra conta — pelo menos é essa a intenção.

A ISO 9564-1 padroniza vários formatos. Interessam três.

33.1 Formato 0 (ANSI X9.8) — o que existe em todo lugar

O mais antigo e o mais difundido. A ideia:

```
campo do PIN   comprimento + digitos do PIN + preenchimento fixo
campo do PAN   parte do numero da conta, alinhada
PIN block      cifra( campo_PIN  XOR  campo_PAN )
```

Bloco de 64 bits, porque nasceu para DES.

Onde ele é fraco, e não é na cifra:

- **O preenchimento é fixo.** Sabendo o formato, o atacante conhece boa parte do bloco em claro antes de começar.
- **O bloco não é autenticado.** Nada prova que aquele PIN block é o que o emissor mandou. Autenticidade vem de fora, se vier.
- **64 bits é pouco.** Com DES, o tamanho de bloco vira problema estatístico antes de o tamanho de chave virar.
- **O PAN é fornecido por quem chama.** Esta é a pior. Se o atacante controla o PAN que entra no XOR, ele controla metade do bloco — e é daí que saem os ataques de tradução da seção 15.3.

33.2 Formato 1 — quando não há conta

Usa preenchimento **aleatório** e não inclui o PAN. Existe para o caso em que o número da conta não está disponível no momento de formar o bloco.

O preço é exatamente o que o formato 0 comprava: **o bloco não está amarrado a conta nenhuma**. Um PIN block formato 1 capturado vale para qualquer conta que aceite formato 1. Por isso ele é restrito a trechos onde outra coisa garante o vínculo.

33.3 Formato 3 — o remendo intermediário

Como o formato 0, mas com preenchimento **aleatório** em vez de fixo. Tapa o buraco do "atacante já conhece metade do bloco" sem mexer no resto. Continua com bloco de 64 bits e continua não autenticado.

33.4 Formato 4 — o de AES, e o único que este projeto poderia fazer

Reprojetado para AES, bloco de 128 bits. A estrutura tem **duas passagens de cifra**, e é isso que importa:

```
bloco A   controle + comprimento + digitos do PIN + preenchimento ALEATORIO
bloco B   campo do PAN, alinhado em 128 bits

intermediario = AES( K, bloco A )
bloco C       = intermediario XOR bloco B
PIN block     = AES( K, bloco C )
```

Compare com o formato 0, que é **uma** cifra sobre um XOR. A diferença não é cosmética:

- No formato 0, controlar o campo do PAN é controlar diretamente a entrada da cifra. No formato 4, o XOR com o PAN acontece **depois** de uma passagem de AES — o atacante não controla mais a entrada, controla a entrada de uma função que ele não sabe inverter.
- O preenchimento aleatório faz o mesmo PIN, na mesma conta, produzir blocos diferentes a cada vez.
- 128 bits de bloco elimina o problema estatístico do DES.

Este é o único formato que este projeto poderia implementar de verdade, porque é o único que usa AES — e AES é o que existe no fabric.

---

34. Tradução de PIN — o comando mais perigoso que existe

Uma transação com cartão atravessa domínios de chave diferentes: terminal, adquirente, bandeira, emissor. Cada domínio tem a sua chave. O PIN precisa chegar do primeiro ao último **sem nunca aparecer em claro fora de um HSM**.

Quem faz isso é a **tradução de PIN block** — o comando `CA` de um de pagamento. Ele:

1. recebe o PIN block cifrado sob a chave de entrada,
2. decifra **dentro** da fronteira,
3. recifra sob a chave de saída (possivelmente noutro formato),
4. devolve o novo PIN block.

O PIN existe em claro por microssegundos, dentro do módulo, e nunca sai.

Por que é o comando mais perigoso: ele é, por construção, um oráculo. O chamador escolhe as chaves, escolhe o PAN, escolhe os formatos, e observa se a operação teve sucesso. Todos os ataques da seção 15.3 nascem daí — o módulo nunca revela o PIN, revela apenas *se deu certo*, e isso basta quando se pode perguntar milhares de vezes.

Se você entender um único comando de HSM de pagamento a fundo, que seja este.

---

35. Verificação de PIN — PVV e IBM 3624

O emissor precisa decidir se o PIN digitado está certo. Guardar os PINs numa base seria insano, então guarda-se um **valor derivado** de quatro dígitos, e a verificação recalcula e compara.

Duas famílias dominam.

35.1 PVV (Visa)

Combinam-se dígitos do PAN, um índice de chave e o PIN; cifra-se o conjunto com um par de chaves PVK; **decimaliza-se** o resultado; guardam-se quatro dígitos. Para verificar, o emissor refaz a conta com o PIN apresentado e compara.

Repare no que ele **não** faz: não guarda o PIN e não permite recuperá-lo. Quatro dígitos de PVV não determinam o PIN — várias entradas colidem, e é proposital.

35.2 IBM 3624 e o *offset*

Cifra-se um dado de validação derivado do PAN, decimaliza-se, e obtêm-se alguns dígitos: o **PIN natural**. Como o cliente quer escolher o próprio PIN, guarda-se o **offset** — a diferença dígito a dígito, módulo 10, entre o PIN escolhido e o natural.

O offset **não é secreto**: ele pode ser impresso, transportado, guardado em claro. Sem a chave, ele não diz nada sobre o PIN.

35.3 A decimalização, e por que ela é o ponto fraco

Cifrar produz **hexadecimal**. PIN é **decimal**. A ponte entre os dois é uma tabela:

```
hex:    0 1 2 3 4 5 6 7 8 9 A B C D E F
tabela: 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
```

Essa tabela, que parece detalhe de formatação, é a origem do ataque mais instrutivo da literatura de HSM — **Bond e Zieliński, 2003**. Algumas APIs deixavam o *chamador* fornecê-la. Fornecendo tabelas degeneradas e observando apenas "verificou / não verificou", recupera-se um PIN de quatro dígitos em cerca de **quinze chamadas**, contra cinco mil de força bruta.

O ataque está detalhado na **seção 15.2**. Aqui só o alerta que ele deixa:

> *Um parâmetro que parece configuração inofensiva pode ser a superfície de ataque inteira.*

E é por isso que o checklist deste projeto pergunta, para todo comando novo, *o que ele vaza se for chamado em laço com entradas escolhidas*.

---

36. DUKPT — uma chave por transação

Imagine um milhão de terminais em campo. Se todos compartilham a mesma chave, um terminal aberto na bancada de alguém compromete a rede inteira. Se cada um tem chave própria, é preciso distribuir e gerenciar um milhão de chaves.

DUKPT (*Derived Unique Key Per Transaction*) resolve os dois de uma vez:

- Existe uma **BDK** (*Base Derivation Key*), que fica só no HSM do adquirente e **nunca vai para o terminal**.
- Cada terminal recebe, na personalização, uma chave inicial derivada da BDK com o identificador dele.
- A cada transação, o terminal deriva uma **chave nova** e **apaga a anterior**.
- O terminal envia o **KSN** (*Key Serial Number*) — identificador do dispositivo mais um contador de transação. Com o KSN e a BDK, o HSM do adquirente rederiva exatamente a mesma chave.

As três propriedades que isso compra:

1. **Compromisso isolado.** Abrir um terminal não dá nada sobre os outros.
2. **Sigilo do passado.** As chaves usadas já foram apagadas no terminal; quem o abrir agora não decifra o que passou por ele ontem.
3. **Nada para distribuir.** O HSM rederiva; não existe base de um milhão de chaves para sincronizar.

36.1 X9.24-1 (TDES) e X9.24-3 (AES)

	TDES DUKPT (X9.24-1)	AES DUKPT (X9.24-3)
Cifra	3DES de duas chaves	AES-128/192/256
Registradores de chave futura	21	32
Derivação	esquema próprio, histórico	função de derivação uniforme
Situação	legado, em desativação	o caminho adiante

A variante AES não é só "a mesma coisa com outra cifra": a derivação foi reescrita para ser uniforme e para suportar chaves de propósitos diferentes a partir da mesma raiz — separação de chaves construída no esquema, em vez de convencionada por fora.

A variante AES é implementável neste projeto. A TDES não, e o motivo é o assunto da próxima seção.

---

37. O que este projeto pode e o que não pode ensinar

A separação não é por dificuldade. É por **hardware** e por **procedência de vetor de teste**.

Assunto	Aqui?	O que decide
PIN block formato 4	**sim**	é AES, e AES é o que existe no fabric
PIN block formato 0 e 3	**estrutura sim**	a construção e as fraquezas se ensinam; a cifra real é DES
Tradução de PIN	**sim**	o valor está no formato do comando e no oráculo, não na cifra
Ataque de decimalização	**sim, e é o melhor item**	é ataque de **API**. Reproduz-se num esquema com AES, sem 3DES e sem vetor de bandeira
DUKPT AES (X9.24-3)	**sim**	cabe no hardware AES-only
CVV / CVC	**não**	inerentemente 3DES; não há variante AES em uso
PVV	**não**	idem — mas o *ataque* sobre ele se ensina sem ele
DUKPT TDES (X9.24-1)	**não**	idem

37.1 Um HSM precisa de 3DES?

A resposta depende de qual HSM, e a diferença ensina mais que a resposta.

HSM de propósito geral, hoje: não. O NIST desautorizou o TDEA para *cifrar* a partir de 2024 (SP 800-131A Rev. 2). Ele sobrevive apenas como algoritmo de **decifração legada** — para abrir o que foi cifrado antes. Projeto novo usa AES, e um módulo que ofereça 3DES para cifrar dados novos está oferecendo um pé no passado.

HSM de pagamento em produção: sim, na prática. Um módulo sem 3DES não conversa com a rede. PIN blocks sob ZPK de 3DES, PVV e CVV sob chaves 3DES, terminais DUKPT TDES aos milhões em campo — tudo isso existe agora, em operação, movendo dinheiro. Um módulo que só faça AES é inútil como substituto de um que está instalado.

E aqui está o ponto que vale o capítulo: a migração para key blocks e para AES foi mandatada em fases pelo PCI e levou **mais de uma década**. Não porque AES seja difícil, mas porque trocar criptografia numa rede com milhões de pontas não é decisão técnica — é logística, contrato e prazo regulatório. Terminal em campo não se atualiza por vontade; norma de mensagem não muda sozinha; e cada elo da cadeia precisa aceitar o novo formato *antes* que o anterior possa ser desligado.

É por isso que criptografia obsoleta sobrevive tanto em pagamentos. Não é descuido: é o custo real de mover um ecossistema.

Este projeto: não, e a ausência é conteúdo. Não há requisito de interoperar com ninguém, então não há a pressão que mantém o 3DES vivo lá fora. A restrição vira lição em vez de limitação.

37.2 Por que não há 3DES aqui

Não é omissão: é decisão registrada. O coprocessador deste projeto amarra o tamanho de chave em **AES-256, sem caminho para AES-128** — porque um modo mais fraco alcançável por escrita em registrador é um *downgrade* de graça para quem tomar o barramento. Acrescentar um núcleo DES contradiz esse princípio no mesmo bloco.

É defensável ter um motor legado, separado e rotulado como tal — que é honestamente o que um HSM de pagamento real tem, porque o mundo ainda roda 3DES. Mas é escolha consciente, e fica para quando a fase chegar.

37.3 O obstáculo maior são os vetores

A regra inviolável deste projeto diz que, se um teste de resposta conhecida falha, **o erro está no código, não no vetor** — e isso só vale se a procedência do vetor for verificável por terceiros. AES, SHA, HMAC, CMAC e CTR_DRBG têm vetores públicos do NIST e do IETF, com hash registrado.

Criptografia de pagamento não tem esse luxo. Os vetores da ANSI X9.24-3 estão atrás de paywall; os de CVV e PVV vivem em documentação de bandeira e em manual de fabricante. Sem vetor autoritativo não há KAT, e "implementei e parece certo" é exatamente o que a regra existe para proibir.

Consequência prática: onde não houver vetor público, o item não entra — ou entra explicitamente marcado como não verificado. Preferir a lacuna honesta à cobertura fingida é o mesmo critério que orienta o resto do documento.

37.4 Uma regra que não se negocia

⚠ **Nenhum número de cartão real, nunca.** Pela mesma razão que nenhuma chave de produção. Gerar CVV ou PIN para um PAN que existe deixa de ser exercício, e o fato de o dispositivo ser de brinquedo não muda isso.

Apêndices

A. Glossário

APT — *Adaptive Proportion Test*. Teste de saúde de fonte de entropia (SP 800-90B) que dispara quando um valor domina uma janela de amostras.

BBRAM — memória volátil do FPGA onde pode ficar a chave de decifra do bitstream. Perde conteúdo sem alimentação, e por isso é recuperável — ao contrário do eFUSE.

BDK — *Base Derivation Key*. Chave de pagamentos da qual se derivam chaves de terminal, tipicamente por DUKPT.

CFS — *Custom Functions Subsystem*. Slot do NEORV32 projetado para receber coprocessadores do usuário. É onde AES e SHA entram na Fase 2.

CMAC — código de autenticação de mensagem baseado em cifra de bloco. Usado no TR-31 para autenticar cabeçalho e corpo, e para derivar KBEK/KBAK.

CMVP — programa NIST/CCCS que valida módulos contra a FIPS 140-3.

CSP — *Critical Security Parameter*. Termo da FIPS para qualquer segredo cuja divulgação compromete a segurança: chave em claro, dado de autenticação, estado do gerador.

Dual control — nenhuma operação sensível pode ser realizada por uma pessoa sozinha. Ver seção 7.

DRBG — *Deterministic Random Bit Generator*. Gerador determinístico semeado por entropia física, especificado na SP 800-90A.

CVV / CVC — três dígitos derivados de PAN, validade e código de serviço sob um par de chaves. Inerentemente 3DES, e por isso fora do alcance deste projeto (seção 37).

Decimalização — mapear os dígitos hexadecimais que uma cifra produz para dígitos decimais, para formar um PIN. A tabela que faz esse mapeamento foi a origem do ataque de Bond e Zieliński (seções 15.2 e 35.3).

DUKPT — *Derived Unique Key Per Transaction*. Esquema em que cada transação usa uma chave distinta derivada de uma BDK, identificada por um KSN. Duas normas: X9.24-1 (TDES, legado) e X9.24-3 (AES). Seção 36.

eFUSE — memória de programação única do FPGA. **Proibida neste projeto**: erro vira brick permanente.

EAL — *Evaluation Assurance Level*, do Common Criteria. Mede o rigor da avaliação, **não** a força da segurança.

Fronteira criptográfica — o limite físico e lógico dentro do qual material de chave em claro existe. Ver seção 5.

KAT — *Known Answer Test*. Vetor de teste com entrada e saída conhecidas, usado nos autotestes.

KBEK / KBAK — chaves de cifra e de autenticação de key block, derivadas da LMK por propósito.

KCV — *Key Check Value*. Três bytes do resultado de cifrar um bloco de zeros com a chave. Permite verificar sem revelar.

KSN — *Key Serial Number*. Identificador do dispositivo mais o contador de transação, enviado junto com cada transação DUKPT. Com ele e a BDK, o HSM rederiva a chave usada. Seção 36.

KEK — *Key Encryption Key*. Chave cuja função é proteger outras chaves.

LMK — *Local Master Key*. A chave mestra do módulo, no topo da hierarquia. Nunca sai, nem embrulhada.

MMCM — *Mixed-Mode Clock Manager*. Bloco do FPGA Xilinx que sintetiza clocks. Aqui, 50 → 100 MHz.

PAN — *Primary Account Number*. O número do cartão. Entra na formação do PIN block para amarrar o PIN à conta. **Nunca usar um PAN real neste projeto** (seção 37.4).

PIN block — formato que combina PIN e número da conta para transporte. Os formatos da ISO 9564 e suas fraquezas estão na seção 33; os ataques clássicos, na 15.3.

PVV — *PIN Verification Value*. Quatro dígitos derivados de PAN e PIN sob uma chave PVK, guardados pelo emissor no lugar do PIN. Seção 35.1.

PMP — *Physical Memory Protection*. Mecanismo RISC-V de restrição de acesso à memória por região.

POST — *Power-On Self-Test*. Bateria de KAT executada no boot, antes de aceitar comandos.

PUF — *Physically Unclonable Function*. Raiz de confiança derivada de variações de fabricação, em vez de armazenada.

RCT — *Repetition Count Test*. Teste de saúde que dispara na repetição excessiva de uma amostra.

Split knowledge — nenhuma pessoa conhece uma chave inteira; ela é montada de componentes.

Tamper-evident / resistant / responsive — a escada de proteção física. Ver seção 18.1.

Tradução de PIN — decifrar um PIN block sob uma chave e recifrá-lo sob outra, sem que o PIN saia da fronteira. É o comando mais perigoso de um HSM de pagamento, porque é um oráculo por construção. Seção 34.

TR-31 — formato de key block da indústria de pagamentos, hoje ANSI X9.143. É o formato adotado por este projeto (seção 8).

TRNG — gerador de números aleatórios verdadeiro, baseado em fenômeno físico.

Zeroização — destruição de material de chave de forma verificável. Ver seção 12.

B. Especificação do protocolo

Implementado em `fw/src/cmd.c` (firmware) e `host/hsmtool.py` (host).

Enquadramento

```
pedido    LEN(2) | CMD(1)    | PAYLOAD | CRC32(4)
resposta  LEN(2) | STATUS(1) | PAYLOAD | CRC32(4)
```

- Todos os campos multi-byte em **big-endian**.
- `LEN` cobre `CMD` / `STATUS` + `PAYLOAD`. Não se inclui nem inclui o CRC.
- `LEN` válido: 1 a 513 (payload máximo de 512 bytes).
- **CRC32** cobre `LEN` + `CMD` / `STATUS` + `PAYLOAD` — todos os bytes menos os quatro dele próprio. Polinômio IEEE 802.3 refletido (`0xEDB88320`), inicialização `0xFFFFFFFF`, xor final `0xFFFFFFFF`. Idêntico ao `zlib.crc32` do Python.
- Timeout entre bytes no dispositivo: 250 ms. Timeout de resposta no host: 2 s.

Comandos da Fase 1

Opcode	Nome	Payload do pedido	Payload da resposta
`0x01`	`PING`	vazio	`"PONG"`
`0x02`	`GET_VERSION`	vazio	major, minor, patch, estado
`0x03`	`GET_DNA`	vazio	8 bytes: identidade do die, 57 bits big-endian

O `GET_DNA` ficou respondendo `NOT_IMPLEMENTED` durante toda a Fase 1 e só fechou com o CFS da Fase 2 (seção 25) — o `DNA_PORT` é primitiva Xilinx e precisava de um caminho até a CPU. O valor **não é segredo**: qualquer um com um cabo JTAG lê o mesmo número, e ele nunca muda. Serve para identificar a placa em log e inventário. Derivar chave dele é um erro clássico, porque público e constante são exatamente as duas propriedades que uma chave não pode ter.

Comandos da Fase 2 — primitivas

Opcode	Nome	Payload do pedido	Payload da resposta
`0x10`	`AES_ENC`	chave(32) || bloco(16)	bloco(16)
`0x11`	`AES_DEC`	chave(32) || bloco(16)	bloco(16)
`0x12`	`SHA256`	mensagem	digest(32)
`0x13`	`HMAC`	klen(1) || chave || mensagem	mac(32)
`0x14`	`RANDOM`	n(2), máximo 256	n bytes do CTR_DRBG
`0x15`	`SELFTEST`	vazio	máscara de falhas(2)

⚠ **`AES_ENC`, `AES_DEC` e `HMAC` recebem a chave em claro no payload**, e por isso só respondem em `UNINITIALIZED`. Não é convenção: é a máscara de estados na tabela de comandos. Um comando que aceita chave em claro não pode coexistir com chave de verdade no mesmo dispositivo — e no instante em que a LMK é carregada, os três param de responder sozinhos. A Fase 3 os **substitui** por versões que falam por handle de slot.

O `SELFTEST` é o único comando que responde em `TAMPERED`, e a exceção é deliberada: sem ele, um dispositivo que reprovou no boot ficaria mudo sobre *o que* reprovou, e o operador teria apenas um LED vermelho.

Comandos da Fase 3 — hierarquia de chaves

Opcode	Nome	Payload do pedido	Payload da resposta
`0x20`	`LMK_LOAD_COMPONENT`	n(1) ‖ componente(32)	kcv(3) ‖ carregados(1) ‖ estado(1)
`0x21`	`LMK_STATUS`	vazio	carregados(1) ‖ completa(1) ‖ kcv(3)
`0x22`	`GEN_KEY`	uso(2) ‖ alg(1) ‖ modo(1) ‖ exp(1)	handle(1) ‖ kcv(3)
`0x23`	`EXPORT_KEY`	handle(1)	key block X9.143 em ASCII
`0x24`	`IMPORT_KEY`	key block X9.143 em ASCII	handle(1) ‖ kcv(3)
`0x25`	`KEY_INFO`	handle(1)	uso(2) ‖ alg(1) ‖ modo(1) ‖ exp(1) ‖ len(1) ‖ kcv(3) ‖ usos(4)
`0x26`	`SET_STATE`	estado alvo(1)	estado atual(1)
`0x27`	`ENCRYPT`	handle(1) ‖ iv(16) ‖ dados	dados cifrados
`0x28`	`DECRYPT`	handle(1) ‖ iv(16) ‖ dados	dados em claro
`0x29`	`MAC_GENERATE`	handle(1) ‖ mensagem	tag(16)
`0x2A`	`MAC_VERIFY`	handle(1) ‖ tag(16) ‖ mensagem	vazio — o veredito é o status
`0x2B`	`DELETE_KEY`	handle(1)	slots livres(1)
`0x2C`	`KEY_FROM_COMPONENTS`	n(1) ‖ total(1) ‖ uso(2) ‖ alg(1) ‖ modo(1) ‖ exp(1) ‖ componente(32)	kcv(3) ‖ carregados(1) ‖ total(1) ‖ handle(1)
`0x2F`	`ZEROIZE`	vazio	estado atual(1)

Estados em que cada um responde. Não é convenção — é uma máscara na tabela de comandos, e é ela que faz a cerimônia ser uma escada de uma via.

Comando	Estados
`LMK_LOAD_COMPONENT`	só `UNINITIALIZED`
`SET_STATE`	só `AUTHORIZED`
`GEN_KEY`, `EXPORT_KEY`, `IMPORT_KEY`, `KEY_INFO`, `ENCRYPT`, `DECRYPT`, `MAC_GENERATE`, `MAC_VERIFY`, `DELETE_KEY`, `KEY_FROM_COMPONENTS`	só `OPERATIONAL`
`LMK_STATUS`	os três normais
`ZEROIZE`	**todos**, `TAMPERED` inclusive

Carregar os três componentes leva a `AUTHORIZED` e o `0x20` desaparece sozinho; ativar leva a `OPERATIONAL` e o `0x26` desaparece do mesmo jeito. Nenhum degrau se repete, e não há como descer sem apagar.

Quais exigem dual control, e a divisão diz mais que a lista:

exigem	`LMK_LOAD_COMPONENT`, `SET_STATE`, `ZEROIZE`, `KEY_FROM_COMPONENTS`
não exigem	`GEN_KEY`, `EXPORT_KEY`, `IMPORT_KEY`, `KEY_INFO`, `ENCRYPT`, `DECRYPT`, `MAC_GENERATE`, `MAC_VERIFY`, `DELETE_KEY`

Dual control é para **cerimônia**, não para operação. Carregar a chave mestra, ativar o dispositivo e apagar tudo são eventos raros, com gente na frente da placa. Gerar, exportar e importar chave é o que um HSM faz o dia inteiro — exigir dois dedos ali não aumentaria segurança nenhuma, só garantiria que ninguém usa o equipamento. E um controle que impede o uso legítimo é desligado no primeiro dia ruim.

O que protege os quatro de baixo é outra coisa, e é estrutural: a chave nunca sai em claro, o key block é autenticado sobre cabeçalho **mais** corpo, e `exportabilidade` decide quem pode sair.

⚠ **O `ZEROIZE` tem uma assimetria deliberada.** O comando exige dois operadores; o gatilho **automático** — autoteste reprovado — não exige ninguém. Se ele exigisse, bastaria não haver operador na sala para a chave sobreviver ao comprometimento.

Notas por comando:

- **`GEN_KEY`** — a chave vem do CTR_DRBG do dispositivo e **nunca existe fora da fronteira**. O host escolhe os metadados e recebe handle e KCV. É o contraste que a fase inteira ensina: o `AES_ENC` da Fase 2 recebia a chave *no payload*.
- **`EXPORT_KEY`** — o único comando que faz material de chave atravessar a fronteira, e ele atravessa embrulhado. Respeita `exportabilidade`: um slot marcado `'N'` devolve `NOT_EXPORTABLE`. Exportar o mesmo slot duas vezes dá blocos **diferentes**, porque o enchimento é aleatório — dois blocos idênticos denunciariam que a mesma chave saiu duas vezes.

- `IMPORT_KEY` — os metadados vêm **do bloco**, autenticados pelo MAC. Toda recusa devolve o mesmo `BAD_PARAM`: bloco malformado, hexadecimal inválido, MAC errado e comprimento impossível são indistinguíveis de fora. Distinguir em qual etapa a validação parou é o oráculo de padding clássico.
- `KEY_INFO` — metadados, nunca chave. Handle inválido e slot vazio devolvem o mesmo código: separar permitiria mapear o key store sem instalar nada.
- `ENCRYPT` / `DECRYPT` — usar a chave guardada, referida por **handle**. É o contraste que fecha a fase: os comandos da Fase 2 recebiam a chave no pedido; aqui vai um número de gaveta.

  AES-CBC com IV explícito, e não ECB — ECB para dados é o erro que a seção 16 usa como exemplo. O IV vem de quem chama, porque uma função que puxa entropia sozinha não é testável de forma determinística.

  O `uso` e o `modo` do slot decidem: só uma chave de dados (`D0`) cifra, e uma marcada `'E'` (só cifrar) recusa `DECRYPT` com `BAD_KEY_USE`. E `exportabilidade='N'` **não** impede usar — não poder sair é diferente de não poder trabalhar.

  ⚠ **É um oráculo de cifragem, e vale dizer.** Quem tem o handle cifra e decifra o que quiser sob aquela chave, em laço. Não há como não ser — é o serviço que um HSM presta. O que ele não entrega é a chave, e é para isso que o handle existe. O controle real está no `modo` e no log de auditoria.
- `MAC_GENERATE` / `MAC_VERIFY` — CMAC-AES sob a chave de um slot. Os modos de uso se dividem em **dois grupos que não se cruzam**: `E` / `D` / `B` cifram, `G` / `V` / `C` autenticam. Uma chave de cifra não assina, e uma de MAC não cifra — é assim que a confusão de tipo de chave se fecha.

  ⚠ `MAC_VERIFY` **devolve um bit**, no status. O dispositivo compara em tempo constante, do lado de dentro. Devolver a tag calculada para quem perguntou comparar entregaria 128 bits em vez de 1, e reabriria o canal lateral de tempo que a comparação interna existe para fechar.
- `DELETE_KEY` — apaga **um** slot, e **não pede dual control**. A assimetria com o `ZEROIZE` é deliberada: apagar um é reversível, porque o key block daquela chave continua existindo fora do equipamento. Apagar tudo não é.
- `KEY_FROM_COMPONENTS` — a cerimônia da chave mestra aplicada a uma chave de trabalho: cada custodiante entra com a sua parte, ninguém vê a chave inteira, e o equipamento monta por XOR do lado de dentro.

  É o único comando de operação que exige dual control, e por isso: é cerimônia, não operação. Aperto novo a cada componente.

  O KCV que volta é o **do componente**, nunca o do acumulado — é o que permite ao custodiante conferir que digitou o dele. E a resposta tem comprimento fixo, com handle zero enquanto a montagem não termina.

O `0x20` e o `0x26` exigem **dual control**: os dois botões pressionados no instante em que o frame chega, e um aperto *novo* a cada autorização (seção 26). São os únicos comandos do projeto cuja autorização não está no link do host — o que é o ponto inteiro deles.

O KCV que o `0x20` devolve é o do **componente**, não o da LMK acumulada. O `0x21` devolve o da LMK, e só quando ela está completa; enquanto incompleta, os três bytes vêm zerados, e a resposta mantém o comprimento fixo para que o tamanho do frame não anuncie nada.

Uma LMK **zerada** é recusada no terceiro componente com `BAD_PARAM`, e a cerimônia recomeça do componente 0 — não há como saber qual dos três estava errado. Acontece se o último componente for o XOR dos dois primeiros: cada um parece aleatório, e a soma é zero. LMK *fraca* (de pouca entropia) não é checada; a qualidade do componente é responsabilidade do custodiante.

Comando da Fase 4 — log de auditoria

Opcode	Nome	Pedido	Resposta	Estados
`0x2D`	`AUDIT_READ`	seq inicial (4)	próximo (4) ‖ continuar (4) ‖ até 16 registros de 16 bytes	todos

Cada registro: `seq(4) | comando(1) | estado(1) | argumento(1) | resultado(1) | reservado(4) | crc32(4)`. Comando `0x00` é o boot; resultado `0xFF` é "pendente", um comando que começou e não terminou. Comando `0x0F` é uma violação de ambiente, com a causa no argumento.

Comando da Fase 6 — sensores

Opcode	Nome	Pedido	Resposta	Estados
`0x2E`	`ENV_READ`	vazio	retido (1) ‖ agora (1) ‖ válido (1) ‖ temperatura, VCCINT, VCCAUX, VCCBRAM (2 cada)	todos

Os quatro valores são códigos de 12 bits do XADC, e `hsmtool env` os converte.

Porta host — command set ASCII (Fase 5)

Tudo acima é o **console**. A porta **host** usa a mesma UART com mensagens ASCII, enquadradas pelo mesmo `LEN` de 2 bytes e **sem CRC**:

```
pedido     LEN(2) | cabeçalho(4) | comando(2) | campos
resposta   LEN(2) | cabeçalho(4) | resposta(2) | erro(2) | campos
```

O primeiro byte do corpo separa os protocolos: de `'0'` a `'~'` é ASCII, e qualquer outro é opcode binário. O cabeçalho é do host e volta igual; a resposta é o comando com o segundo caractere mais um.

A chave de trabalho vai **no comando**, como key block X9.143 sob a LMK. O comprimento do bloco sai do próprio cabeçalho dele. Dados vão como comprimento em 4 dígitos decimais seguido de hex.

Pedido	Resposta	Campos do pedido	Campos da resposta	Estados
`D0`	`D1`	—	estado(1) ‖ KCV da LMK(6) ‖ versão(6)	todos menos `TAMPERED`
`K0`	`K1`	uso(2) ‖ modo(1) ‖ exportabilidade(1)	key block ‖ KCV(6)	`OPERATIONAL`
`K2`	`K3`	key block	uso(2) ‖ alg(1) ‖ modo(1) ‖ exp(1) ‖ KCV(6)	`OPERATIONAL`
`K4`	`K5`	bloco da KEK ‖ bloco da chave (os dois sob a LMK)	bloco da chave sob a KEK ‖ KCV(6)	`OPERATIONAL`
`K6`	`K7`	bloco da KEK (sob a LMK) ‖ bloco da chave (sob a KEK)	bloco da chave sob a LMK ‖ KCV(6)	`OPERATIONAL`
`C0`	`C1`	key block ‖ IV(32) ‖ dados	dados	`OPERATIONAL`
`C2`	`C3`	key block ‖ IV(32) ‖ dados	dados	`OPERATIONAL`
`M0`	`M1`	key block ‖ dados	MAC(32)	`OPERATIONAL`
`M2`	`M3`	key block ‖ MAC(32) ‖ dados	— (o veredito é o erro)	`OPERATIONAL`
`R0`	`R1`	exportabilidade(1)	key block ‖ n(512) ‖ e(8)	`OPERATIONAL`
`R2`	`R3`	key block ‖ digest(64)	assinatura(512)	`OPERATIONAL`
`R4`	`R5`	n(512) ‖ e(8) ‖ digest(64) ‖ assinatura(512)	— (o veredito é o erro)	`OPERATIONAL`
`E0`	`E1`	exportabilidade(1)	key block ‖ Qx(64) ‖ Qy(64)	`OPERATIONAL`
`E2`	`E3`	key block ‖ digest(64)	r(64) ‖ s(64)	`OPERATIONAL`
`E4`	`E5`	Qx(64) ‖ Qy(64) ‖ digest(64) ‖ r(64) ‖ s(64)	— (o veredito é o erro)	`OPERATIONAL`

Erros: `00` sem erro · `01` verificação falhou · `10` key block inválido · `11` o uso ou o modo do bloco não permite · `12` chave não exportável · `13` o bloco de proteção não é KEK · `15` dados inválidos · `16` parâmetro inválido · `20` estado não permite · `21` comando desconhecido · `22` mensagem malformada · `27` falha no log · `90` erro interno · `91` erro grave. Os códigos de comando e de erro são deste projeto.

Os comandos `R` são de RSA-2048 e os `E`, de ECDSA sobre a curva P-256. O digest é SHA-256, calculado pelo host. A seção 30 explica os dois.

`K4` e `K6` trocam chave com outra instituição sob uma **KEK** — chave de uso `K0` que as duas têm, formada por componentes no console (`comp-load --uso K0`). `K4` é exportar e recusa `'N'`; `K6` guarda o que chegou sob a LMK. O `modo` da KEK diz se ela embrulha (`E`), desembrulha (`D`) ou os dois (`B`).

No log de auditoria, um comando da porta host aparece com os dois caracteres dele nos campos `comando` e `argumento` (`K0` = `0x4B`, `0x30`) e o código de erro em decimal no `resultado`.

Códigos de status

Código	Nome	Significado
`0x00`	`OK`	
`0x01`	`BAD_CRC`	CRC não confere
`0x02`	`BAD_LEN`	`LEN` fora da faixa
`0x03`	`TIMEOUT`	frame incompleto, resincronizado
`0x10`	`UNKNOWN_CMD`	opcode não está na tabela
`0x11`	`BAD_PARAM`	tamanho ou conteúdo do payload
`0x12`	`NOT_IMPLEMENTED`	na tabela, sem implementação ainda
`0x20`	`WRONG_STATE`	comando não permitido neste estado
`0x21`	`NOT_AUTHORIZED`	falta dual control
`0x22`	`NOT_EXPORTABLE`	exportabilidade do slot proíbe
`0x23`	`NO_SLOT`	key store cheio
`0x24`	`BAD_KEY_USE`	o `uso` ou o `modo` do slot proíbe a operação
`0x25`	`MAC_INVALID`	o MAC não confere
`0x26`	`STORE_FAIL`	fase 4: a flash não gravou; a operação foi desfeita
`0x27`	`LOG_FAIL`	fase 4: sem registro no log, o comando não executa
`0x30`	`SELFTEST_FAIL`	
`0x32`	`CRYPTO_FAIL`	A dupla execução discordou: nada sai, as chaves ficam e o dispositivo marca erro grave
`0x31`	`TAMPERED`	
`0xFF`	`INTERNAL_ERROR`	

Exemplo verificado em hardware

```
pedido   : 00 01 01 91 5D D8 C5
resposta : 00 05 00 50 4F 4E 47 FB 28 3D 2A
                ^^  P  O  N  G
```

`91 5D D8 C5` é `zlib.crc32(b'\x00\x01\x01')`.

C. Pinagem

Fonte: esquemáticos QMTECH e verificação em hardware. Detalhes e procedência em `doc/pinout.md`.

Função	Pino	Estado
Clock 50 MHz	N11	verificado em hardware
Reset (SW1)	B7	esquemático; ativo baixo
UART RX / TX	T15 / T14	verificado em hardware
Dual control A (SW2)	M6	verificado em hardware; ativo baixo
Dual control B (SW5)	P6	verificado em hardware; ativo baixo
LEDs D1–D5	R6, T5, R7, T7, R8	verificados; **ativos baixos**; todos vermelhos
7 segmentos	T10, K13, P11, R11, R10, N9, K12, P9	`a` … `g`, `dp`; **acende com 0**
Dígitos (varredura)	T9, P10, T8	3 dígitos; **habilita com 1**

Armadilha registrada: com o gravador ligado existem **duas** portas seriais. O adaptador JTAG usado é um FT232H que também cria `/dev/ttyUSB*`, e costuma enumerar **antes** da placa. Escolher a porta por ordem alfabética acerta o cabo de gravação, e o sintoma é um timeout sem explicação. A ferramenta de host escolhe por identificador USB por causa disso.

Nenhuma pendência de pinagem. As três que existiam — cores dos LEDs, polaridade do display e ordem física dos botões — foram fechadas por medida em hardware, e duas delas mudaram o projeto:

- **Os cinco LEDs são vermelhos.** O requisito "LED vermelho para `TAMPERED`" está atendido e ao mesmo tempo vazio: **cor não distingue nada nesta placa**. Um indicador de estado tem de se separar por *posição* e *padrão* — e é o display que carrega a informação de verdade.
- **A polaridade do display estava invertida no código**, por palpite. O segmento acende com `0` e o dígito habilita com `1`; o toplevel supunha o contrário. Não aparecia porque, com os segmentos desligados, habilitar ou não os dígitos dá no mesmo — apareceria no primeiro estado exibido.

O detalhe que fecha o assunto: **um testbench conferia o valor errado e passou meses verde**, porque afirmava a suposição em vez de uma medida.

D. Reproduzir o trabalho

Dependências

```
git clone --recurse-submodules <url>

sudo apt install -y gcc-riscv64-unknown-elf picolibc-riscv64-unknown-elf
sudo apt install -y python3-serial openfpgaloader
```

Vivado 2026.1 (o `xsim` também é usado na simulação, por causa das primitivas Xilinx e do VHDL do NEORV32).

O compilador sozinho não basta: ele fornece apenas os headers freestanding, e o header do NEORV32 precisa de uma biblioteca C. Não há `libnewlib` para RISC-V nos repositórios Debian; picolibc é o equivalente.

Construir e testar

```
source /opt/AMD/2026.1/Vivado/settings64.sh

./scripts/sim.sh                # todos os testbenches; recompila o
                                # firmware sozinho se o C estiver mais novo
make -C fw image                # o build da FPGA depende desta imagem
vivado -mode batch -source scripts/build.tcl

./scripts/program.sh flash      # grava na SPI flash

python3 host/hsmtool.py selftest # sem placa: o codec do protocolo
python3 host/test_hsmtool.py     # sem placa: o transporte
python3 host/test_tr31.py        # sem placa: o key block (precisa de
                                # `cryptography`)
python3 host/hsmtool.py ping
python3 host/hsmtool.py bench -n 10000
```

⚠ **`flash`, e não a RAM de configuração.** O firmware mora na Block RAM de instruções, cujo conteúdo inicial vem do bitstream. Configurar por JTAG pode deixar a placa com `Done = 1`, o clock travado e a CPU **sem executar uma instrução** — porque a inicialização das Block RAMs não foi aplicada. O sintoma é cruel: tudo indica sucesso e o dispositivo está mudo. Detalhes e discriminadores de falha em `doc/bancada.md`.

⚠ **`sim.sh` recompila o firmware quando o C está mais novo que a imagem.** Até isso existir, editar C e simular validava o **binário anterior** — e o modo como isso foi descoberto está em `doc/fase3-notas.md`: uma sabotagem deliberada do código passou na simulação, porque o código sabotado nunca chegou a ser compilado.

Política de código de terceiros

Nunca editar `third_party/` diretamente: a mudança não fica guardada no repositório, é descartada por `git submodule update`, e quebra a procedência — o pin continua dizendo `v1.13.3` enquanto o bitstream contém outra coisa.

Ordem para mudar código externo:

1. **Configuração** (generic no wrapper) — resolve quase sempre
2. **Ponto de extensão projetado** — é o caso do CFS na Fase 2
3. **Patch** em `patches/`, aplicado pelo build
4. **Fork** e repontar o submódulo — só com patch que precise persistir

Detalhes em `doc/submodulos.md`. O espelho offline reproduzível dos cores externos sai de `./scripts/mirror-deps.sh`.

E. Operar o dispositivo

O apêndice D ensina a **construir**. Este ensina a **usar** — uma sessão inteira, do power-on até o dispositivo operacional, com o que cada resposta significa.

Tudo daqui é feito com `host/hsmtool.py`, que fala o protocolo do apêndice B pela UART do CP2102 (115200 8N1, a mesma porta USB que alimenta o core board).

E.1 Ligar e conferir

O bitstream mora na **flash SPI**, então a placa sobe sozinha na energização. Não há nada a carregar.

Antes de digitar qualquer coisa, olhe o display. Ele soletra o estado:

```
    U n i      UNINITIALIZED   sem chave mestra
    A u t      AUTHORIZED      LMK carregada, ainda não em serviço
    O P E      OPERATIONAL     em serviço
    t P r      TAMPERED        reprovou, e não sai daí por software
```

Isso é o dispositivo dizendo em que estado está **sem que ninguém pergunte** — e é a única informação que ele oferece de graça. Confirme depois pela serial, que é o canal com autoridade:

```
python3 host/hsmtool.py version
```

```
firmware : v0.1.0
estado   : UNINITIALIZED (0)
```

`UNINITIALIZED` é o estado de fábrica: o dispositivo funciona, mas não guarda chave nenhuma.

Antes de qualquer outra coisa, o autoteste. Ele já rodou no boot — o `SELFTEST` apenas o repete, e é assim que se confere que o módulo continua calculando certo *agora*, não só quando ligou:

```
python3 host/hsmtool.py post
```

```
AES-256               ok
SHA-256               ok
HMAC-SHA-256          ok
CTR_DRBG              ok
TRNG / health tests   ok
CMAC-AES-256          ok
key store             ok
key block X9.143      ok
```

Oito linhas, oito verdades diferentes.

As cinco primeiras são respostas conhecidas contra vetores oficiais (seção 11) — exceto a quinta, que são os **testes de partida** da fonte de entropia sobre um retrato de 1024 amostras brutas (seção 10).

As três últimas são de natureza diferente e vale distinguir, porque a norma distingue: **testes de função crítica**, não de algoritmo. Não existe "resposta conhecida" para instalar uma chave — existem propriedades que, se falharem, tornam o dispositivo perigoso *sem parecer quebrado*:

- **key store** — uma chave marcada como não exportável não sai; um slot apagado não responde; uma chave de cifrar não é aceita para decifrar; e a zeroização, depois de rodar, não deixou um byte diferente de zero na região de chaves;
- **key block X9.143** — um key block de procedência externa é desembrulhado e devolve a chave certa; um cabeçalho adulterado é recusado; e uma ida-e-volta fecha.

⚠ Sobre esse último vale uma ressalva de procedência que não aparece em nenhum outro: **o vetor dele não é do CAVP, e não há como ser.** O programa de validação do NIST valida *algoritmo*, e X9.143 é *formato*; a norma que traz o exemplo é paga. O que se usa é um valor conhecido de terceiros, fixado por commit e por hash. Está registrado em `vectors/MANIFEST.txt` em vez de dissimulado entre os demais.

⚠ **O autoteste zera a memória de trabalho.** O teste do key store instala e apaga chaves de verdade e termina com o store vazio, chave mestra inclusive. Um autoteste que poupasse a chave exercitaria um caminho que não é o do boot. Até a Fase 3, rodá-lo num dispositivo carregado apagava a LMK; desde a Fase 4 o dispositivo restaura tudo da flash em seguida e volta ao estado em que estava.

Se alguma reprovar, o dispositivo vai para `TAMPERED` e passa a atender **só** ao `SELFTEST`. Isso é o comportamento correto, e a exceção é deliberada: sem ela o operador teria apenas um LED vermelho e nenhuma informação sobre *o que* falhou.

E.2 O que dá para fazer antes de haver chave

Em `UNINITIALIZED` o dispositivo é uma caixa de primitivas. Estes comandos recebem a chave **no payload**, o que um HSM de verdade jamais aceitaria:

```
python3 host/hsmtool.py sha256 616263          # SHA-256 de "abc"
python3 host/hsmtool.py aes <chave-hex> <bloco-hex>
python3 host/hsmtool.py random -n 32           # bytes do CTR_DRBG
```

Eles existem para exercitar o hardware antes de existir key store, e **param de responder no instante em que houver LMK** — não por uma linha de código que os desligue, mas porque a máscara de estados deles diz `UNINITIALIZED` e o dispositivo não estará mais lá. Guarde isso: é a demonstração mais limpa de por que estado importa num HSM.

O `random` é o único dos quatro que continua funcionando depois. Ele não recebe chave nenhuma.

E.3 A cerimônia de LMK

A **Local Master Key** é a raiz da hierarquia (seção 7): é ela que embrulha todas as demais. Ela não é digitada por uma pessoa. Ela nasce de **três componentes**, cada um sob a guarda de um custodiante diferente, que se combinam por XOR dentro do dispositivo.

Por que XOR e por que três: nenhum componente isolado revela **nada** sobre a chave — nem um bit — e conhecer dois de três não ajuda. Três é prática operacional, não criptografia: dois custodiantes não dão margem se um faltar no dia, e mais de três transforma a cerimônia em logística.

O gesto físico. Cada componente exige dois botões da placa pressionados no instante em que o comando chega. São o **2º e o 5º** de uma fileira de cinco, cujo primeiro é o reset:

```
    [reset]  [ * ]   [   ]   [   ]   [ * ]
      SW1     SW2     SW3     SW4     SW5
```

O par mais afastado disponível, para dificultar apertar os dois com uma mão só. O `hsmtool.py` desenha esse mapa a cada pedido, porque ler silkscreen minúsculo no meio de uma cerimônia é como se aperta o botão errado.

E o dispositivo confirma. Aperte os dois: o **ponto decimal** do dígito da direita acende. Solte: apaga.

```
    U n i.        os dois botões apertados, autorização disponível
    U n i         não
```

Esse ponto é a resposta de bancada para "que botões?" — em vez de confiar num rótulo, aperte e veja o dispositivo concordar. Se acendeu, o comando seguinte vai ser autorizado; se não acendeu, seria recusado, e você descobre isso **antes** de gastar o comando em vez de depois.

Ele não vaza nada: quem está apertando os botões já está na frente da placa, e não há observador remoto para quem a informação seja novidade.

E exige um aperto **novo**: entre um componente e o seguinte, os dois têm de ser vistos **soltos**. Segurar os dois durante a cerimônia inteira carrega **um** componente, não três (seção 26).

```
python3 host/hsmtool.py lmk-status
```

```
componentes : 0 de 3
KCV da LMK  : -- (incompleta)
```

Agora cada componente, um de cada vez:

```
python3 host/hsmtool.py lmk-load 0 --random
```

O `--random` gera o componente **no host** e o imprime na tela. Isso é uma concessão de brinquedo, e o próprio comando avisa: material de chave na tela é exatamente o que um HSM existe para evitar. Num equipamento real cada componente nasce com o seu custodiante, num cartão ou num dispositivo de entrada dedicado. Para fornecer um componente existente, passe-o em hex no lugar de `--random`.

A ferramenta então pede o gesto e espera:

```
Carregar o componente 0 da LMK.
  SOLTE os dois botoes, depois SEGURE SW2 e SW5 juntos
  e tecle Enter sem soltar.
```

Segure os dois, tecle Enter sem soltar, e a resposta volta:

```
KCV do componente : 539062
componentes       : 1 de 3
estado            : UNINITIALIZED (0)
```

O KCV que volta é o do componente, não o da LMK. É o que permite ao custodiante conferir, ali, que digitou o dele e não o do vizinho. Sem esse passo, um componente trocado só apareceria no KCV final — quando já não dá para saber qual dos três estava errado. Confira contra o valor que o custodiante trouxe anotado; se não bater, **pare a cerimônia**.

Repita para os componentes `1` e `2`. Ao terceiro:

```
componentes       : 3 de 3
estado            : AUTHORIZED (1)
```

O estado muda no mesmo comando que completa a chave, e não num comando separado: *"tenho LMK"* e *"estou em AUTHORIZED"* precisam ser a mesma afirmação, ou o estado vira opinião.

```
python3 host/hsmtool.py lmk-status
```

```
componentes : 3 de 3
KCV da LMK  : 46F2FB
```

Anote esse KCV. Ele é a única coisa derivada da LMK que atravessa a fronteira, e é como se confere, em qualquer sessão futura, que o dispositivo tem a chave certa — sem que ninguém veja a chave. Três bytes dão uma colisão em 16 milhões: basta para pegar erro de digitação, e não basta para atacar.

Se `STATUS_NOT_AUTHORIZED` voltar, os botões não estavam apertados — ou estavam apertados desde a autorização anterior. Solte, aperte e repita. Essa recusa é a única prova de que o dual control existe de verdade: é o único comando do dispositivo cuja autorização não está no link do host.

E.4 Ativar

`AUTHORIZED` significa "tenho a raiz". `OPERATIONAL` significa "estou em serviço". A passagem é um ato de cerimônia, também com dual control:

```
python3 host/hsmtool.py activate
```

```
estado : OPERATIONAL (2)
```

Repare no que desapareceu. Tente agora:

```
python3 host/hsmtool.py aes <chave> <bloco>
```

```
dispositivo recusou: STATUS_WRONG_STATE (0x20)
```

O comando que aceitava chave em claro não existe mais neste estado. E a própria cerimônia também não se repete — não há caminho para trocar a chave mestra por cima da existente, o que seria substituir a raiz sem apagar o que ela protege. A escada é de **uma via só**, e descer exige apagar.

E.5 Os indicadores da placa

O display de 7 segmentos é o indicador que importa, e há um motivo concreto: os cinco LEDs desta placa são **todos vermelhos**. A cor não carrega informação nenhuma, só a posição — o requisito "LED vermelho para `TAMPERED`" está atendido e vazio.

O que mostra	Significado
`Uni` `Aut` `OPE` `tPr`	o estado da máquina (seção 8)
ponto decimal aceso	dual control satisfeito **neste instante**

Um detalhe de projeto que vale registrar: se os dois bits de estado chegarem corrompidos, o display mostra `tPr`. O `default` do decodificador é `TAMPERED`, e não `Uni` nem apagado — falhar para o lado seguro vale também para o painel. Um display que mostrasse "sem chave" quando não sabe seria pior que um display apagado.

E o que ele **nunca** vai mostrar: KCV, handle, qualquer coisa derivada de chave. Ótico é o único canal do dispositivo que não aparece numa captura de UART, então é o canal em que um vazamento passaria despercebido por mais tempo. O módulo recebe dois bits de estado e um de autorização, e não tem caminho nenhum até material de chave.

Os LEDs, que sobraram como indicadores grosseiros:

LED	Significado
D1	pisca a 1 Hz — o clock de 100 MHz está vivo. É **hardware**, independente da CPU
D2	aceso — o POST passou e o dispositivo está no laço de comandos
D5	aceso — `TAMPERED`

D1 ser hardware é deliberado: se o firmware travar, D1 continua piscando, e isso distingue "clock morto" de "firmware pendurado". Num dispositivo sem console, essa distinção é a diferença entre depurar e adivinhar.

O display, ao contrário, é dirigido pelo firmware — ele mostra o que a CPU diz. As duas coisas se complementam: **D1 prova que o hardware vive, o display prova que o firmware sabe onde está.** Display congelado com D1 piscando é firmware pendurado; os dois parados é clock morto.

Fora isso, o dispositivo é **mudo até ser perguntado**. Um terminal aberto na porta serial não mostra absolutamente nada, e esse é o comportamento correto: um módulo criptográfico que conversa sozinho está contando alguma coisa a alguém.

E.6 Trabalhar com chaves

Depois de `OPERATIONAL`, **nenhum comando pede botão** — e isso é o ponto, não uma economia. A cerimônia acabou; o que vem agora é operação.

```
hsmtool.py gen-key --uso D0 --modo B --exp E
#   handle : 1
#   KCV    : 7ED838
```

A chave veio do gerador do dispositivo e **nunca existiu fora da fronteira**. O que voltou foi um handle e três bytes de KCV. Compare com o `aes` da Fase 2, onde a chave ia no pedido: é a mesma diferença entre um cofre e uma gaveta.

```
hsmtool.py export-key 1
#   D0144D0AB00E0000B82679114F470F54...
```

São 144 caracteres de texto: dezesseis de cabeçalho legível, o corpo cifrado, e o MAC. Exporte de novo e o bloco será **diferente** — o enchimento é aleatório, e dois blocos idênticos denunciariam que a mesma chave saiu duas vezes.

```
hsmtool.py import-key D0144D0AB00E0000...
#   handle : 2
#   KCV    : 7ED838      <- o mesmo
```

Mesmo KCV, outro handle: a chave voltou inteira. E o experimento que vale a pena fazer é o negativo — troque **um caractere** do bloco, em qualquer posição, e tente importar. O dispositivo recusa. O caractere na posição 11 é a exportabilidade; reescrevê-lo à mão é literalmente o ataque que o MAC sobre o cabeçalho existe para impedir.

```
hsmtool.py gen-key --exp N
hsmtool.py export-key 3
#   dispositivo recusou: STATUS_NOT_EXPORTABLE
```

Uma chave marcada `'N'` não sai. Não por convenção: existe **um único caminho** para os bytes deixarem um slot, e é ele que consulta o campo.

```
hsmtool.py key-info 1
#   uso D0 · algoritmo A · modo B · exportabilidade E · 32 bytes
#   KCV 7ED838 · usos 0
```

Metadados, nunca chave. Não existe “me devolva o slot inteiro”.

E finalmente **usar** a chave — que é o que separa um cofre de um depósito:

```
hsmtool.py encrypt 1 <iv 16 bytes> <dados múltiplos de 16>
hsmtool.py decrypt 1 <mesmo iv>    <o criptograma>
```

A chave é referida por **handle**. Compare com o que a Fase 2 fazia: lá a chave ia dentro do pedido. Aqui vai um número de gaveta, e o material nunca atravessa a linha.

O experimento que fecha o assunto: exporte uma chave, reimporte, e cifre o **mesmo bloco com o mesmo IV** usando os dois handles. Os criptogramas têm de ser idênticos — é a prova de que a chave voltou bit a bit, e é bem mais forte que comparar os três bytes do KCV.

E o negativo, que ensina mais:

```
hsmtool.py gen-key --modo E        # 'E' = só cifrar
hsmtool.py decrypt <handle> ...    # -> BAD_KEY_USE
```

Confusão de tipo de chave é a origem de uma família inteira de ataques de API (seção 15). A defesa é o campo `modo`, e a verificação vive num lugar só — dentro do key store, não espalhada pelos comandos.

⚠ Uma distinção que confunde: `exportabilidade='N'` **não** impede usar. Impede **sair**. Uma chave que nunca sai e trabalha o dia inteiro é o caso mais comum de uma chave bem configurada.

E autenticar, que é o outro serviço:

```
hsmtool.py gen-key --uso M0 --modo C --exp E   # 'C' = gerar e verificar MAC
hsmtool.py mac 5 <mensagem hex>
hsmtool.py mac-verify 5 <tag> <mensagem>       # "MAC confere"
```

O veredito vem do **status**, não do payload: o equipamento compara e diz sim ou não. Devolver a tag calculada para você comparar entregaria 128 bits em vez de um, e é o tipo de conveniência que abre canal lateral.

E repare no `--modo C`. Os modos se dividem em **dois grupos que não se cruzam**:

grupo	valores	o que faz
cifra	`E` `D` `B`	`encrypt` / `decrypt`
MAC	`G` `V` `C`	`mac` / `mac-verify`

Tente `mac` numa chave `--modo B` e veja `BAD_KEY_USE`. Tente `encrypt` numa `--modo C` e veja o mesmo.

O `uso` manda junto: `encrypt` exige `D0`, e `mac` exige um uso da família `M`. Por isso a chave acima é `--uso M0`, e não só `--modo C`. Gere uma `--uso K0 --modo B` e tente `encrypt`: o modo deixaria, o uso não deixa.

Essa separação não é burocracia de campo — é a defesa contra **confusão de tipo de chave**, que é a origem de uma família inteira de ataques de API (seção 15). Uma KEK que decifrasse "dados" entregaria as chaves que ela protege.

E.7 Liberar um slot, e montar uma chave a várias mãos

O key store tem dezesseis slots. Para devolver um:

```
hsmtool.py delete-key 5
#   apagado. slots livres: 12
```

Sem dual control, ao contrário do `zeroize`. A razão é que apagar um slot é **reversível**: se você guardou o key block daquela chave, reimportar devolve tudo. Apagar o equipamento inteiro não é reversível, e por isso pede duas pessoas.

E a operação que fecha o capítulo — montar uma chave que **ninguém conhece inteira**:

```
hsmtool.py comp-load 0 2 --random --uso K0 --modo B --exp E
hsmtool.py comp-load 1 2 --random --uso K0 --modo B --exp E
#   handle : 6  <- chave montada e instalada
```

É a cerimônia da chave mestra aplicada a uma chave de trabalho. Cada custodiante entra com a sua parte, sob dual control, e o equipamento combina por XOR **do lado de dentro**. É assim que uma chave combinada entre duas instituições entra num HSM sem que nenhuma das duas a conheça.

Três detalhes que valem reparar:

- **É o único comando de operação que pede os dois botões.** Os outros são trabalho; este é cerimônia.
- **O KCV que volta é o do componente**, não o da chave em construção. Serve para o custodiante conferir a parte dele. O do acumulado seria um oráculo sobre a chave que ainda está sendo montada.
- **Começar pelo componente o reinicia a montagem.** É como se abandona uma cerimônia que saiu errada, sem precisar de um comando para isso.

E o comando que desfaz tudo:

```
hsmtool.py zeroize      # com os dois botões
```

Apaga os 16 slots e a LMK, **e confere que apagou** antes de dizer que apagou. Funciona em qualquer estado — inclusive em `TAMPERED`, e ali ele apaga a chave sem tirar o dispositivo de `TAMPERED`: um HSM que se cura de tamper não detectou tamper nenhum.

E.8 Desligar, religar e ler o log *(fase 4)*

```
# depois da cerimônia e de gerar chaves: desligue e ligue a placa
python3 host/hsmtool.py version
#   estado   : OPERATIONAL (2)      <- sem cerimônia
python3 host/hsmtool.py lmk-status
#   KCV da LMK  : (o mesmo de antes)
python3 host/hsmtool.py key-info 1  # mesmo handle, mesmo KCV

python3 host/hsmtool.py audit
#      0  BOOT                Uni  sem registro persistente   OK
#      1  LMK_LOAD_COMPONENT  Uni  arg=0x00                   OK
#    ...
#     14  BOOT                OPE  restaurado                 OK
```

Cada linha do `audit` é um comando, gravado **antes** de executar. A coluna de estado é o estado em que o dispositivo estava quando o comando chegou. Um `GEN_KEY` tentado em `Uni` aparece com o resultado `WRONG_STATE`: tentativas recusadas também entram.

O `post` continua zerando a memória de trabalho, mas agora restaura tudo da flash em seguida, e o dispositivo volta ao estado em que estava. Só o `zeroize` apaga de vez: ele limpa a LMK e as chaves da flash e mantém o log, que é o registro de quem zeroizou.

E.9 O que ainda não existe

Honestidade sobre o estado do projeto, para que ninguém procure um comando que não foi escrito:

- **A LMK sobrevive ao desligamento, mas não está protegida.** Desde a Fase 4 ela fica na flash, que o projeto *trata* como memória segura. Quem tiver um gravador SPI a lê. É uma decisão didática diante do hardware disponível, explicada no §27.
- **Não há anti-rollback.** Regravar a flash com uma imagem antiga faz uma chave apagada voltar, e o dispositivo não percebe.
- **Não há exportação em formato legado.** Só em key block X9.143. Por isso `exportabilidade='S'` se comporta como `'E'` — e está certo: na norma, `'S'` é o valor *mais permissivo*, que libera também formatos legados (criptograma simétrico, embrulho RSA). Como eles não existem aqui, `'S'` e `'E'` liberam o mesmo caminho.
- **O log de auditoria detecta corrupção, não adulteração.** O CRC de cada registro pega um bit trocado por acidente; quem editar a flash de propósito recalcula o CRC.
- **`hsmtool keycycle` precisa da LMK em claro**, o que um HSM de verdade nunca permitiria. Serve para as duas implementações do formato de key block se conferirem; é bancada, não operação.

F. Leitura

Normas

- **FIPS 140-3** e ISO/IEC 19790 — requisitos de módulo criptográfico
- **NIST SP 800-90A/B/C** — DRBG, fontes de entropia, construções
- **ANSI X9.143** (sucessor do TR-31) — key blocks
- **RFC 8017** (PKCS #1 v2.2) — RSA, e o enchimento de assinatura v1.5
- **PCI PIN Security Requirements** e **PCI PTS HSM** — de onde vêm dual control e split knowledge como exigência

Ataques

- Bond, M. — *Attacks on Cryptoprocessor Transaction Sets* (2001)
- Bond, M. e Zieliński, P. — *Decimalisation Table Attacks for PIN Cracking*

2003.

- Clulow, J. — *On the Security of Real-World PIN Processing* (2003)
- Boneh, DeMillo e Lipton — *On the Importance of Checking Cryptographic Protocols for Faults* (1997) — o ataque Bellcore
- Kocher, P. — *Timing Attacks* (1996) e, com Jaffe e Jun, *Differential Power Analysis* (1999)
- Piret e Quisquater — análise diferencial de falhas em AES (2003)

- Schindler, W. — *A Timing Attack against RSA with the Chinese Remainder Theorem* (2000), e Walter, C. e Thompson, S. — *Distinguishing Exponent Digits by Observing Modular Subtractions* (2001) — a subtração final do produto de Montgomery como canal lateral
- Bleichenbacher, D. — falsificação de assinaturas RSA com `e = 3` contra verificadores que interpretam o enchimento (2006)

Aritmética

- Montgomery, P. — *Modular Multiplication Without Trial Division* (1985)
- Koç, Ç., Acar, T. e Kaliski, B. — *Analyzing and Comparing Montgomery Multiplication Algorithms* (1996) — de onde vem o FIOS

Livros

- Anderson, R. — *Security Engineering*. Os capítulos sobre APIs de segurança e sobre segurança de hardware são a melhor introdução ao assunto da Parte III.
- Maes, R. — *Physically Unclonable Functions*. Para o experimento da Fase 6.

Ferramentas e cores usados

- **NEORV32** — processador RISC-V em VHDL, `github.com/stnolting/neorv32`
- **secworks/aes** e **secworks/sha256** — cores criptográficos para a Fase 2
- **openFPGALoader** — gravação por JTAG

Este documento acompanha o repositório `hsm-a7`. A Parte V reflete o estado ao fim da Fase 1, validada em hardware. As Partes I a IV são independentes do projeto e valem por si.